Claude Chat & Cowork Masterclass

Zero to Hero — From Everyday Chat to Agentic Desktop Automation

14	94	Zero to Hero
Sections	Lessons	Outcome

Act 1 — Chat	Sections 1–7 · Chat mastery, prompting, connectors, Projects, Artifacts, Skills
Bridge	Section 8 · Quality control and responsible use
Act 2 — Cowork	Sections 9–14 · Agentic desktop, automation, Dispatch, Computer Use, Live Artifacts

SECTION 01 · ACT 1 — CHAT

Foundations, Models & Safe Setup

Section 1 of 14

Section 1: Foundations, Models & Safe Setup

Before you build any habit, workflow, or automation with Claude, you need to understand what you're actually working with. Not at a technical level, but at a practical one. What does Claude do well? What does it get wrong? Which model should you choose, and why does that choice matter? How does memory work, and what does Claude actually remember between sessions? And perhaps most importantly: what are your responsibilities as the person reviewing its output?

This section answers all of that. It's the foundation everything else in the course rests on.

Why this section matters

Most people start using Claude the same way they start using a search engine: they type something in, read the result, and move on. That works fine for casual use. It stops working the moment you start relyi

Section 1 — Foundations, Models & Safe Setup

What Claude Is (And Why It's Not a Search Engine)

What Claude Is (And Why It's Not a Search Engine)

If your mental model for Claude is "fancy Google," you're going to be disappointed, confused, or both. The two tools do fundamentally different things, and understanding that difference is the first step to using Claude well.

Why this matters

Every day, people ask Claude questions it can't reliably answer, and ignore the things it does better than almost anything else, because they're treating it like a search engine. They ask for today's news. They ask for exact stock prices. They ask for the current version of a software package. Then they get a confident-sounding answer that's out of date or wrong, and they either trust it (bad) or dismiss Claude entirely (also bad, just for different reasons).

The fix isn't to use Claude less carefully. It's to understand what kind of tool you're actually using.

The idea in plain English

A search engine retrieves documents. You type a query, it finds web pages that match, and it shows you links. The information lives somewhere on the internet; the search engine's job is to find it and rank it.

Claude works entirely differently. It was trained on a large body of text up to a knowledge cutoff date. That training shaped a model that can reason, draft, explain, summarize, compare, translate, and generate text. When you ask Claude something, it doesn't go out to the web and look it up. It generates a response based on patterns and knowledge baked in during training.

This means Claude is, in a real sense, always speaking from memory. A very large, well-organized memory, but memory nonetheless. And like human memory, it can be wrong, outdated, or confidently mistaken.

The practical implications:

- Claude cannot tell you what happened in the news this morning
- Claude can help you draft a response to a difficult email in minutes
- Claude cannot reliably cite a specific source unless it's been given one
- Claude can synthesize a complex topic into a clear explanation
- Claude does not know your company's internal documents unless you share them
- Claude can reason about those documents once you do share them

How this works in Claude Cowork

In Cowork, Claude gets connected to your files, email, calendar, and other tools through connectors. This changes the picture in an important way: Claude is no longer limited to its training data. It can read your actual inbox, your actual calendar, your actual documents, and reason over that real, current information.

But the underlying mechanism is still the same. Claude reads what you give it, reasons over it, and generates output. It doesn't browse the web independently (unless a specific connector or tool enables that). It doesn't check facts against live sources by default. It works with whatever is in the context window.

This is why, later in the course, you'll learn to give Claude well-structured inputs and review its outputs carefully, especially when it's working with your real data and potentially taking actions on your behalf.

Practical example

Say you're preparing for a quarterly business review. You ask Google: "quarterly business review template." You get links to templates and articles. You click around, copy something, and adapt it.

You ask Claude: "I'm preparing a QBR for a mid-size SaaS company. The audience is our executive team. We've had a solid quarter on revenue but missed on net retention. Draft an agenda and a one-paragraph framing statement that acknowledges the retention issue without leading with bad news."

Claude doesn't give you a link. It gives you the agenda and the paragraph. You edit it. You use it. The difference isn't speed; it's the kind of work being done. Search retrieves. Claude generates and reasons.

Workflow design notes

Because Claude generates rather than retrieves, it can produce plausible-sounding text that is factually wrong. This is sometimes called hallucination, though the more accurate description is "confident generation without grounding." Claude doesn't know it's wrong. It generated what seemed most likely given its training.

The practical design principle: treat Claude's output as a strong first draft produced by a smart, well-read assistant who hasn't checked the news in a while and doesn't have access to your files unless you've shared them. Review before you send. Verify facts that matter. Supply current or proprietary information yourself.

This isn't a flaw you work around. It's a property of the tool you design for.

Try this in Claude

Open a new Claude conversation and send this prompt:

> *"I want to understand how you work. Without accessing the internet, summarize what you know about [a topic you're an expert in]. Then tell me what you wouldn't be able to reliably answer about that topic."*

Pay attention to two things: how accurate the summary is for something you know well, and how honest Claude is about its own limitations. This gives you a calibration point for how much to trust Claude in your domain.

Pro tips

- When you need current facts, supply them. Paste in the relevant data, article excerpt, or document. Claude reasons over what you give it.
- Don't ask Claude to "look up" anything. Ask it to reason, draft, summarize, compare, or explain instead.

- If you need web search, use a connector or tool that provides it explicitly. Don't assume Claude has live access.
- The more specific your question, the more useful the output. Vague inputs produce generic responses.
- Claude works best when you treat it as a thinking partner, not an oracle.

Quick summary

Claude is a language model that reasons and generates, not a search engine that retrieves. It works from trained knowledge, not live web access. It can help you think, draft, explain, and synthesize at a level most tools can't match, but it cannot tell you what happened this morning, and it doesn't know your business unless you tell it. Design your workflows accordingly. [redacted]

Section 1 — Foundations, Models & Safe Setup

Where Claude Excels (And Where It Will Let You Down)

Where Claude Excels (And Where It Will Let You Down)

Knowing what a tool does well is useful. Knowing where it quietly fails is essential. Claude has genuine strengths that most professionals underuse, and predictable failure modes that catch people off guard every time. This lesson maps both so you can deploy Claude where it helps and stay skeptical where it doesn't.

Why this matters

The most expensive Claude mistakes don't come from obvious errors. They come from plausible-sounding output that's wrong in subtle ways: a date that's slightly off, a statistic that's close but fabricated, a legal principle that's nearly right but not quite. People trust it because it sounds authoritative. They act on it. Then something goes wrong.

The other cost is missed opportunity. People who treat Claude as a drafting tool and nothing more are leaving most of its value on the table. Claude can do far more than polish sentences. Understanding its real strengths changes how ambitiously you use it.

The idea in plain English

Claude's strengths cluster around language, reasoning, and structure. Its failure modes cluster around facts, numbers, and anything that requires access to current or external information.

Where Claude is genuinely strong:

Drafting and rewriting. Claude is fast, consistent, and good at matching tone. It can draft a client email, a board memo, a job posting, or a product brief in a fraction of the time it would take to write from scratch. It can rewrite your draft to be more concise, more formal, or more persuasive on request.

Summarization. Give Claude a long document, thread, or transcript, and it will pull out the key points accurately and quickly. It handles legal documents, research papers, meeting notes, and dense reports well. This is one of its most reliable use cases.

Explanation and teaching. Claude is an excellent explainer. It can take a complex concept and render it clearly for a non-expert audience. This makes it useful for onboarding materials, training content, FAQs, and customer-facing documentation.

Reasoning through problems. Claude can walk through a decision, weigh tradeoffs, identify risks, and structure a recommendation. It's not infallible here, but for many professional decisions it gives you a thoughtful sounding board that's available instantly.

Structuring unstructured input. You give Claude a brain dump, a set of rough notes, or a disorganized list. It returns a structured document. This is surprisingly useful and underused.

Translation and adaptation. Not just language translation. Claude can translate ideas between audiences: take a technical spec and rewrite it for a business audience, take a regulatory document and surface the parts that matter to a specific team.

Where Claude fails:

Precise facts, especially recent ones. Claude has a training cutoff. It doesn't know what happened last week. Even within its training data, it can confuse specifics: wrong years, wrong figures, misattributed quotes. If the fact matters, verify it independently.

Arithmetic and calculation. Claude can reason about math, but it makes arithmetic errors. It's better than it used to be, and worse than a calculator. For anything financial or quantitative, verify the numbers.

Citations. Claude will sometimes invent sources. It generates plausible-sounding references that don't exist. Never use a Claude-supplied citation without checking that the source is real.

Your internal information. Claude doesn't know your company, your clients, your policies, or your history unless you tell it. It will fill gaps with plausible-sounding generic content if you don't provide the specifics.

Predicting outcomes. Claude can reason about probability and risk, but it is not a forecasting tool. Treat its predictions as structured thinking prompts, not forecasts.

A practical map

Task	Claude reliability	Notes
Drafting emails and documents	High	Review tone and specifics
Summarizing long text	High	Works well with supplied documents
Explaining complex concepts	High	Verify technical accuracy in specialized domains
Structuring rough notes	High	Useful and underused
Recent news or events	Low	Training cutoff applies
Precise statistics	Low	Verify independently
Citations and references	Low	Often fabricated; always check
Arithmetic	Medium	Use a calculator for anything that matters
Your internal business data	N/A	Claude only knows what you share

Practical example

A consultant uses Claude to prepare for a client workshop on supply chain risk. She pastes in a 40-page industry report and asks Claude to summarize the top five risks and flag any that apply to her client's industry. Claude produces a sharp summary in two minutes.

She then asks Claude: "What was the global average lead time for electronics components in Q3 last year?" Claude gives a number with confidence. She doesn't verify it. It ends up in her slide deck. At the workshop, a client asks for the source. She can't provide one because the number was fabricated.

The summary: reliable. The specific statistic: not. Same session, same tool, very different reliability depending on the type of task.

Workflow design notes

Design Claude into workflows based on where it's strong. Use it to draft, summarize, explain, and structure. Build in human verification for any output involving specific facts, numbers, dates, or citations that will be published or acted upon.

In Cowork, as Claude starts doing more autonomous work, these failure modes matter more. An automated email drafted with a fabricated statistic, sent without review, is a problem. The solution is to identify which parts of a workflow carry factual risk and build review steps around them.

Try this in Claude

Send this prompt:

> *"I need to write a short briefing document on [a topic relevant to your work]. Before you draft it, tell me: which parts of this topic are you confident about, and where should I verify your output before using it?"*

This builds a useful habit: asking Claude to flag its own uncertainty before you commit to anything.

Pro tips

- Always supply source material for fact-sensitive tasks. Don't ask Claude to recall facts from training when you can paste in the actual document.
- For anything you'll share externally, read it as if you're the skeptical recipient. Claude's confident tone is not the same as accuracy.
- Use Claude for the thinking and drafting. Use your own judgment and primary sources for verification.
- The more specific and bounded the task, the more reliable the output. "Draft an agenda for a 45-minute team retrospective" is better than "help me with my team."

Quick summary

Claude is strong at drafting, summarizing, explaining, reasoning, and structuring. It is unreliable on precise facts, recent events, arithmetic, and citations. Knowing the difference is not a caution label; it's the operating manual. Design your use of Claude around its actual strengths, and build verification into any workflow where accuracy matters.

Section 1 — Foundations, Models & Safe Setup

Choosing the Right Claude Model (And Why It Actually Matters)

Choosing the Right Claude Model (And Why It Actually Matters)

Picking the wrong Claude model for a task is like using a sledgehammer to hang a picture frame. The job gets done, sort of, but you've wasted effort and probably left marks you didn't intend. Model selection isn't a technical decision reserved for developers. It's a practical workflow choice that affects the quality of your results and how much time you spend getting there.

Why this matters

Claude isn't one thing. It comes in distinct variants, each with different strengths, response styles, and costs in terms of speed and credits. Most users default to whatever model is pre-selected and never think about it again. That works fine for low-stakes tasks. It's a real liability when you're doing complex, ambiguous, or high-stakes work.

Getting model selection right means your outputs are better calibrated for the task. You stop getting overly elaborate answers to simple questions, and you stop getting surface-level responses to problems that actually require depth.

The idea in plain English

At the time of this course, Claude's flagship model is **Opus** (currently Opus 4). Below it sits **Sonnet**, the mid-tier workhorse. There's also **Haiku**, the fastest and lightest option, useful for quick tasks where speed matters more than nuance.

Here's the practical frame:

Sonnet is your default for well-scoped, specific tasks. If you know exactly what you want and the request is clear, Sonnet handles it well and quickly. Think of it as the capable colleague who executes a clear brief without needing a lot of hand-holding.

Opus is built for tasks where Claude needs to figure out what you actually want. Ambiguous briefs, interpretive work, complex reasoning, nuanced writing, multi-layered problems, and situations where the "right answer" isn't obvious going in. Opus takes more time and costs more, but for genuinely hard work it earns the difference.

Haiku is for lightweight, repetitive tasks where you need speed and volume rather than depth. Not typically what you'd reach for in professional knowledge work, but worth knowing about.

How this works in Claude

Model selection typically appears as a dropdown or selector in the chat interface before you start a conversation. In Claude's web app, it appears at the top of a new chat. In Cowork, you can set a

default model for a workspace or change it per session.

The model you select applies to that conversation. If you start with Sonnet and find the task is more complex than expected, you can start a new conversation with Opus rather than trying to wrestle a better answer out of the wrong model.

Practical example

Here's the same task handled by different model choices:

Task A: "Draft a follow-up email to a client who confirmed they'll renew their contract. Keep it warm, brief, and include a note about the onboarding call we scheduled for Thursday."

This is well-scoped and specific. You know the output you want. Sonnet is the right choice. Opus would produce a similarly good email, but you'd be spending more for no meaningful difference.

Task B: "We've had three high-performing team members leave in the past six months. I'm not sure whether it's a management issue, a compensation issue, or something cultural. Help me think through how to diagnose this and what questions I should be asking."

This is ambiguous. The problem isn't well-defined. The "answer" isn't a single output but a thinking process. Opus is the right choice here. It will engage with the complexity, ask clarifying questions internally as it reasons, and give you something that actually helps you think rather than a generic HR framework you could have Googled.

A decision framework

Task type	Recommended model	Why
Clear brief, specific output	Sonnet	Fast, accurate, no depth needed
Drafting from a template or example	Sonnet	Well-scoped by definition
Summarizing a document you provide	Sonnet	The content is given; reasoning load is low
Ambiguous problem requiring judgment	Opus	Figuring out the right question is half the work
Complex writing with voice and nuance	Opus	Tone, interpretation, and craft require it
Multi-step reasoning or analysis	Opus	Depth of reasoning matters
Quick reformatting or extraction	Haiku or Sonnet	Speed over depth

Workflow design notes

One common mistake is using Opus for everything because it feels safer. In practice, this leads to slower responses, higher credit usage, and sometimes overly elaborate answers to simple questions. Opus will write three paragraphs about why an email should be brief when Sonnet would have just written the brief email.

The inverse mistake is defaulting to Sonnet for everything and being puzzled when Claude gives you a shallow answer to a hard question. Sonnet isn't underperforming; it's optimized for a different job.

For automated workflows in Cowork, model selection matters especially because you're not in the loop to course-correct. A scheduled task running on the wrong model might produce technically correct but poorly calibrated output. When setting up automation, think carefully about the type of task and choose the model accordingly.

Try this in Claude

Pick two tasks from your current workload: one that's clearly specified, and one that's genuinely ambiguous or interpretive. Run the specific task through Sonnet and the ambiguous one through Opus. Note the difference in how each model approaches the problem. This gives you a calibration point for your own work patterns.

Pro tips

- When in doubt on a complex task, use Opus for the first pass to establish framing and structure, then use Sonnet for follow-up drafting or refinement within that structure.
- If Opus gives you an answer that seems overly cautious or hedged, it may be picking up on genuine ambiguity in your request. Tighten the brief before retrying.
- In Cowork, check the model setting whenever you set up a new scheduled task or workflow. The default may not be the right choice for what you're automating.
- Model capabilities improve over time. The best practice is to revisit your default choices periodically rather than set and forget.

Quick summary

Sonnet handles clear, well-scoped tasks efficiently. Opus earns its place on ambiguous, interpretive, or complex work where Claude needs to reason about what you actually want. Choosing correctly isn't a technical detail; it's a professional judgment that affects output quality. Match the model to the task, and you'll get better results with less friction.

How Claude Memory Works (And How to Actually Control It)

Claude remembers things about you. That sentence sounds either reassuring or alarming depending on who you are, and both reactions are reasonable. Memory makes Claude progressively more useful over time. It also raises real questions about what's being stored, how it's used, and whether you can trust it. This lesson covers all of it.

Why this matters

Without memory, every Claude conversation starts from zero. You reintroduce yourself, explain your role, re-state your preferences, and provide context that you've already given a dozen times. It's repetitive and friction-heavy for anyone using Claude regularly.

With memory working well, Claude builds a lightweight model of who you are and how you work. It knows you prefer concise output. It knows you're a product manager at a mid-size SaaS company. It knows you're working on a Q3 launch and that you've already discussed the positioning. That context carries forward, making interactions faster and outputs more relevant.

Memory is now available on all plan tiers, not just paid accounts. That makes this a feature worth understanding regardless of how you're using Claude.

The idea in plain English

Claude's memory works through automatic summarization. After conversations, Claude (and the underlying system) identifies information that seems worth retaining: facts about you, preferences you've expressed, ongoing projects you've mentioned, instructions you've given. That information gets stored as a memory that Claude can reference in future conversations.

This is different from Claude "reading all your old chats." It's more like Claude keeping a rolling set of notes about you, structured around what's useful to carry forward.

You can view exactly what Claude has stored about you. You can edit it. You can delete individual memories or clear everything. The controls live in Settings, under a Memory section.

Memory is also not infallible. Claude might store something inaccurate, outdated, or lower-priority than you'd expect. It might miss something you thought was important. Managing memory

occasionally, not obsessively, is a reasonable practice.

How this works in Claude

In the Claude interface, navigate to Settings and look for the Memory section. There you'll see a list of stored memories, each presented as a short statement or note. Examples of what you might see:

- "Works in product management at a technology company"
- "Prefers bullet-point summaries over long paragraphs"
- "Currently leading a product launch scheduled for Q3"
- "Prefers formal tone in client-facing documents"

Each memory can be edited or deleted individually. You can also add memories manually if there's something specific you want Claude to carry forward that it hasn't picked up on its own.

Memories apply across conversations. When you start a new chat, Claude has access to your stored memories and can use them to inform responses without you having to re-explain.

Practical example

A marketing director uses Claude heavily for content and communications. In her first few weeks, she keeps repeating the same context: her company sells enterprise software, her audience is IT procurement teams, and she prefers active voice and direct language.

After a few conversations, Claude's memory captures most of this. Now when she starts a new chat asking Claude to draft a case study, Claude already knows the audience, the tone preference, and the industry context. She doesn't have to front-load every request with three paragraphs of background. The effective length of her prompts drops by half.

She also notices that Claude stored a slightly wrong detail about her role. She edits it in Settings. The correction propagates to future conversations.

What to store, what to skip

Not everything should be in memory. Here's a rough guide:

Worth storing	Not worth storing
Your role and industry	One-off project details
Consistent tone and format preferences	Specific client names (see privacy note below)
Ongoing high-priority projects	Details that will be outdated in weeks
Writing style rules you always apply	Anything sensitive or confidential

Worth storing	Not worth storing
Recurring workflow preferences	Task-specific context for a single conversation

Workflow design notes

Memory is most valuable for persistent preferences and stable context. It is least useful for ephemeral project details that will be irrelevant in a month. Review your stored memories every few weeks. Prune anything outdated. Add anything Claude has missed that would genuinely improve future interactions.

A practical governance note: be thoughtful about what business information ends up in memory. Client names, deal details, and confidential project specifics probably don't belong in a persistent memory store. Keep that kind of context in the conversation itself, or in a Claude Project where you have more control. Memory is for your stable working context, not for sensitive business data.

In Cowork, memory interacts with folder-level and global instructions, which you'll cover in Section 9. The short version: memory captures what Claude learns about you automatically; instructions are what you tell Claude explicitly at the project or workspace level. Both matter, and they work together.

Try this in Claude

Open Settings and look at your current memory. Ask yourself three questions:

- Is everything here accurate?
- Is anything missing that Claude should know about how you work?
- Is anything stored that you'd rather Claude not carry forward?

Edit, add, or delete as needed. This takes five minutes and meaningfully improves the quality of future interactions.

Then, in your next Claude conversation, notice whether Claude applies your stored context without being asked. If it does, memory is working. If it doesn't, check whether the relevant memory is stored accurately.

Pro tips

- After any conversation where you establish a new preference or working style, check Settings to see if it was captured. If not, add it manually.
- Don't let memory become a dumping ground for everything. Lean toward fewer, higher-quality memories over a long, cluttered list.
- If you share a Claude account with colleagues, be aware that memories reflect your individual usage. Shared accounts can produce unexpected memory contamination.

• Memory can be entirely disabled in Settings if you prefer a clean slate for every conversation. Some users working with sensitive information choose this.

Quick summary

Claude memory auto-summarizes past conversations and stores facts, preferences, and context about you. It's available on all tiers, controlled from Settings, and editable at any time. Used well, it makes Claude progressively more useful by reducing repetitive context-setting. Used carelessly, it can store inaccurate or sensitive information you didn't intend to persist. Check it, shape it, and prune it periodically.

Section 1 — Foundations, Models & Safe Setup

Finding and Using Your Past Claude Conversations

Finding and Using Your Past Claude Conversations

Most people treat Claude like a search bar: ask, get an answer, close the tab. What they leave behind is a growing archive of thinking, drafts, decisions, and context that they'll never use again. That's a lot of wasted work. Your Claude conversation history is searchable, referenceable, and genuinely useful if you know how to work with it.

Why this matters

Over time, if you use Claude regularly, you accumulate something valuable: a record of how you've thought through problems, what outputs you've produced, what context you've shared, and what decisions you've made with Claude's help. This isn't just a log. It's institutional memory about your own work.

Being able to find a specific conversation, extract a draft you produced three weeks ago, or recall the framing you used for a similar problem in the past makes Claude more useful as a long-term tool rather than a series of disconnected interactions.

It also saves time. "I know I worked through this with Claude before" is a usable signal, not just a vague memory, when you know how to search.

The idea in plain English

Claude stores your conversation history and makes it searchable through the sidebar. You can search by keyword, browse by recency, or look for conversations tied to specific topics or projects.

Within a conversation, you can scroll back through the full thread, copy outputs, and reference earlier points in the same session by continuing the conversation rather than starting a new one.

You can also reference a past conversation explicitly by starting a new chat and pasting in context from the old one. Claude doesn't natively link conversations together unless you use Projects (covered in Section 5), but you can manually carry forward anything relevant.

How this works in Claude

In the Claude interface, the left sidebar shows your recent conversations, each labeled by an auto-generated title based on the conversation content. You can search this list using the search bar at the top of the sidebar.

Tips for finding what you're looking for:

- Search for a distinctive word or phrase you used in the conversation, not a generic topic
- If you remember roughly when the conversation happened, you can scroll to that time period
- Conversations tied to Projects appear in the Project view rather than the general history

Once you've found a conversation, you can open it, read it, copy sections, or continue it by sending a new message. Continuing an existing conversation preserves all the context Claude has from that thread.

Practical example

A business analyst has a standing weekly task: prepare a summary of the department's key metrics for the leadership meeting. She used Claude to set up a format for this three months ago and wants to use the same structure.

Instead of rebuilding the format from memory, she searches her history for "metrics summary" and finds the conversation where she worked it out. She copies the format, starts a new chat, pastes it in as context, and says: "Use this format to help me draft this week's version with the following numbers." Two minutes instead of twenty.

This is a straightforward workflow, but it compounds. The more deliberately you use Claude, the more useful your history becomes as a searchable library of your own work.

Workflow design notes

A few things that improve the usefulness of your history:

Name conversations deliberately when it matters. Claude auto-generates titles, but they're often generic. For important or reusable conversations, rename the thread to something you'll

recognize later. (Right-click or use the edit option on the conversation title in the sidebar.)

Don't start new conversations for work that's continuing. If you're iterating on a document across multiple sessions, continue the same conversation rather than opening a new one. This keeps all the context in one place and makes it findable.

Use Projects for work that spans many conversations. If you're working on a specific client engagement, research project, or ongoing initiative, Projects give you a structured container for conversations, documents, and instructions. History search works across your general library; Projects give you dedicated spaces. Both have their place.

Know what history does and doesn't preserve. Your conversation text is preserved. The model's reasoning within a conversation is not separately stored. If you asked Claude to work through a complex analysis, the answer is there but the intermediate steps only exist within that conversation thread.

Try this in Claude

Search your conversation history right now for a topic you've discussed with Claude in the past month. Find the most useful output from that conversation. Ask yourself: is this something you'd want to reuse or build on?

If yes, either rename that conversation to something memorable, or pull the relevant content into a Claude Project for easier future access (you'll set that up in Section 5).

If you can't find the conversation because the search doesn't surface it clearly, note what you searched for and why it didn't work. That's useful calibration for how to structure future conversations so they're more findable.

Pro tips

- When you finish a significant Claude conversation, spend thirty seconds renaming it to something descriptive. Future-you will thank present-you.
- For reusable formats, templates, or frameworks you develop with Claude, keep a short master document where you paste the key outputs. This is faster than re-searching history every time.
- If you're continuing work from a previous session, start the new conversation with a brief recap: "In a previous conversation, we established X. I want to continue from there." This re-establishes context without requiring Claude to search its memory.
- History is personal. If you're working on a shared Claude account or workspace, you may not see each other's conversations depending on how the account is set up.

Quick summary

Your Claude conversation history is searchable and reusable. Search by keyword, continue existing conversations rather than starting new ones for ongoing work, and rename important threads so they're findable later. Paired with Projects (coming in Section 5), your history becomes a functional library of your own thinking and outputs rather than a graveyard of one-off interactions.

Privacy and the Review Habit: Two Things You Can't Skip

There are two practices that separate people who use Claude well from people who eventually get burned by it. The first is being deliberate about what you share. The second is reviewing what Claude produces before you act on it. Neither takes much time. Both prevent the kind of mistakes that are embarrassing at best and genuinely harmful at worst.

Why this matters

Claude is a powerful tool that processes everything you give it. That's what makes it useful. It's also what makes it worth thinking carefully about before you paste in a client's financial records, your company's unreleased product roadmap, or an employee's performance review.

On the output side, Claude is fluent, confident, and fast. Those qualities make its mistakes easy to miss. A factual error delivered with authority reads the same as a factual truth. A draft email with a subtle tone problem gets sent if no one reads it carefully first. The habit of review is cheap insurance against a long list of avoidable problems.

And in Act 2 of this course, when Claude moves from producing text you read to taking actions on your behalf, the stakes go up further. The review habit you build now will carry directly into that higher-stakes context.

Privacy: what to share and what to protect

Claude is developed by Anthropic. Conversations you have through Claude's consumer products may be used, subject to Anthropic's current privacy policy, to improve models. Enterprise and API customers have different data handling arrangements. If you're using Claude in a business context and data handling matters to your organization, check the current policy and your plan terms

rather than assuming.

The practical principle: treat Claude like a capable outside contractor who is excellent at the work but not bound by your company's confidentiality agreements unless you've arranged that explicitly.

Think carefully before sharing:

- Client names, deal terms, or confidential business information
- Personal data about employees, customers, or individuals (health information, financial details, performance records)
- Unreleased product plans, pricing strategy, or competitive intelligence
- Legal communications with privilege implications
- Anything covered by an NDA or regulatory requirement

What you can share without much concern:

- General descriptions of a problem without identifying details ("a client in retail" rather than the client's actual name)
- Your own work and thinking
- Public information, general industry context, published documents
- Sanitized versions of documents where identifying details have been removed

A useful test: before pasting something into Claude, ask whether you'd be comfortable if that content were visible to someone outside your organization. If not, either anonymize it or find a different approach.

The review habit: build it now

Claude's output should be treated as a well-researched first draft from a smart, fast, occasionally overconfident assistant. That framing tells you the appropriate response: read it, check it, and edit it before you use it.

The review habit has three components:

Read it as the recipient would. Don't read your draft email with the knowledge of what you meant. Read it as if you just received it. Does it say what you intended? Does the tone land correctly? Is anything ambiguous or likely to be misread?

Check the facts that matter. For any specific claim, number, date, name, or citation that will be acted on, verified independently. Claude can be wrong on these with complete confidence. It's not trying to mislead you; it simply generates plausible-sounding content and doesn't always know where the plausible stops and the accurate begins.

Ask whether the output actually answers what you needed. Claude sometimes produces something technically responsive to your prompt that isn't actually what you needed. This is more common with vague prompts, but it happens with clear ones too. A quick check: does this output move my actual goal forward, or did Claude answer a slightly different question than the one I was

asking?

Workflow design notes

The stakes of review scale with the consequence of being wrong. A Claude-drafted internal brainstorm note needs light review. A client-facing proposal with specific claims needs careful review. A legal document or regulatory filing needs professional review by a qualified person, regardless of how good Claude's draft is.

As you move through this course, the review habit applies to increasingly consequential outputs:

- In Chat sections: review text before sharing or sending
- In the Bridge section (Section 8): build formal review processes for outputs that will be published or distributed
- In Cowork sections: review before authorizing automated actions, because automated errors can compound faster than manual ones

Think of it as a sliding scale: the more consequential and irreversible the action, the more deliberate the review should be.

A privacy rule for memory

One specific note worth calling out: Claude's memory feature, which you covered in the previous topic, can store context across conversations. Be intentional about what ends up there. Business-sensitive details, client-specific information, and anything confidential probably shouldn't persist in a general memory store. Keep that kind of context within a specific conversation or Project where you have more control over its scope and lifespan.

Try this in Claude

Draft a short version of your personal privacy rules for using Claude. It can be simple: a list of categories of information you will and won't share, and a note about what level of review you'll apply to different types of output.

This becomes the privacy section of your mini project (the Claude Setup and Safety Profile). Having it written down means you'll actually follow it, and it's a useful artifact if you're ever asked by a manager or security team how you're using AI tools.

A starter template:

> ***I will not share with Claude:*** *[specific categories for your context]* ***I will treat Claude's output as a draft and verify before use in:*** *[circumstances where accuracy matters]* ***I will apply additional human review before:*** *[high-stakes actions or published outputs]*

Pro tips

• Anonymize before you paste. "Our largest retail client" is usually sufficient context. You don't need to name them.
• If you work in a regulated industry (finance, healthcare, legal), check with your compliance team before using consumer Claude tools for work involving regulated data. Enterprise agreements and on-premises deployments exist for exactly these situations.
• The review habit is easier to maintain if you build it into your workflow explicitly. "I'll read this once before I send it" is a rule you'll follow. "I'll review it if something feels off" is not.
• Don't conflate reviewing with rewriting everything from scratch. The goal is to catch errors and adjust tone, not to undo Claude's work.

Quick summary

Be deliberate about what you share with Claude: treat it like a capable outside contractor without a confidentiality agreement, and keep sensitive business and personal data out of general conversations. Build the habit of reviewing Claude's output before you act on it: read it as the recipient would, check any facts that matter, and confirm it answers the question you actually asked. These two practices protect you and the people your work touches, and they become more important as Claude takes on more autonomous work later in this course.

SECTION 02 · ACT 1 — CHAT

Section 2 of 14

Section 2: Prompting & Iteration

If Section 1 was about understanding what Claude is, Section 2 is about learning how to talk to it well. The gap between a mediocre Claude response and a genuinely useful one is almost always in the prompt, not the model. This section teaches you to close that gap deliberately and consistently.

Why this section matters

Most people write prompts the way they write a text message: quickly, loosely, with the assumption that the other party will fill in the gaps. That works with people who share your context and can ask follow-up questions. With Claude, vague inputs produce generic outputs, and generic outputs require more back-and-forth to fix than a well-structured prompt would have cost upfront.

Good prompting isn't a technical skill. It's a communication skill. The underlying principles are the same ones that make any written brief effective: be clear about who you are, what you need, what context is relevant, and what good looks like.

What you'll learn

This section covers five topics:

Role, task, context, and output format. The four-part structure that turns a vague request into a clear brief. Role tells Claude who it's working for and what perspective to take. Task specifies what you want done. Context provides the relevant background. Output format tells Claude what the finished product should look like. Mastering this structure improves every prompt you write.

Adding examples and constraints. The fastest way to align Claude's output with your expectations is to show it what you want. Examples anchor tone, structure, and quality in a way that descriptions alone rarely achieve. Constraints (what to avoid, what to keep under a certain length, what format not to use) are equally important: they prevent Claude from doing plausible things you don't want.

Asking for tables, outlines, and checklists. Structured output requests are underused. When you need information organized rather than narrated, asking for a specific format produces a much more usable result.

Using follow-up questions. A single prompt rarely produces a finished result. Knowing how to refine, redirect, and build on Claude's initial response through follow-up is how you move from first draft to something genuinely useful.

Critique and revision loops. Claude can critique its own output and improve it on request. Building a deliberate revision loop into your workflow produces better results than accepting the first response or rewriting manually from scratch.

The mini project

You'll take a weak, poorly-specified prompt (one you've actually used or one provided as an exercise) and improve it through each element: role, context, examples, output format, and a revision pass. The before-and-after comparison is the deliverable, and it's a practical demonstration of exactly how much prompt quality affects output quality.

How to approach this section

Each topic builds on the previous one. Start with the four-part structure, then layer in examples and constraints, then practice requesting specific formats, then work on the follow-up and revision skills. By the end, prompting well should feel less like trial-and-error and more like a learnable, repeatable process.

The payoff compounds. Every Claude interaction in the rest of this course benefits from stronger prompting habits.

Section 2 — Prompting & Iteration

The Four-Part Prompt: Role, Task, Context, and Output Format

The Four-Part Prompt: Role, Task, Context, and Output Format

Most Claude prompts fail not because the model is bad but because the brief is incomplete. You would not hand a new colleague a sticky note and expect a finished report. You'd explain who they are in this context, what you need, what they need to know to do it, and what the finished thing should look like. That same discipline applied to Claude prompts produces dramatically better results.

Why this matters

Vague prompts produce responses that are technically responsive but practically useless. "Help me with my presentation" is a prompt. "Write the executive summary slide for a board presentation on Q2 performance, written for a financially sophisticated audience, in five bullet points of no more than fifteen words each" is a brief. The second one takes thirty seconds longer to write and saves thirty minutes of revision.

The four-part structure is not a formula you have to follow rigidly for every message. It's a checklist that helps you notice when you're leaving something important out.

The idea in plain English

Role tells Claude what perspective or expertise to bring to the task. It answers the question: who is Claude being right now?

Examples: - "You are an experienced HR consultant." - "You are a plain-language editor." - "Act as a skeptical CFO reviewing this proposal."

Role shapes tone, vocabulary, and the assumptions Claude brings to the work. A response from "an experienced HR consultant" reads differently from a response from "a friendly generalist assistant." Both might answer your question, but only one is calibrated to your actual need.

Task is what you want done. It should be specific and action-oriented.

Examples: - "Review this job description and identify any language that might discourage qualified candidates." - "Rewrite this paragraph in plain English for a non-technical audience." - "List the five strongest objections a CFO would raise to this proposal."

Context is the relevant background Claude needs to do the task well. It includes information about your situation, constraints, audience, existing work, or anything else that would affect what a good response looks like.

Examples: - "The audience is senior executives who are not technical. They have fifteen minutes and want conclusions, not methodology." - "We are a 40-person professional services firm in the UK." - "The tone of our existing documentation is formal but accessible."

Output format specifies what the finished product should look like: length, structure, format, what to include, what to omit.

Examples: - "Return a numbered list of no more than six items." - "Write this as a three-paragraph memo, not longer than 200 words." - "Format as a markdown table with columns for Risk, Likelihood, and Mitigation."

How this works in Claude

Claude responds to these elements whether you label them explicitly or embed them naturally in your prompt. Both of these work:

Labeled version:

> *Role: You are a plain-language editor. Task: Rewrite the paragraph below. Context: The reader is a first-time homebuyer with no legal background. Format: Match the original paragraph length. Avoid jargon.*

Natural version:

> *Rewrite this paragraph as a plain-language editor would for a first-time homebuyer with no legal background. Keep it roughly the same length and avoid jargon.*

The natural version is often faster and reads better. The labeled version is useful when you're learning the structure, when prompts are complex, or when you want to be especially precise.

Practical example

Before (weak prompt):

> *Write a bio for our new VP of Marketing.*

After (four-part prompt):

> *You are a professional copywriter specializing in executive communications. Write a 100-word professional bio for our new VP of Marketing, Sarah Chen. Context: Sarah joins us from a Series B fintech startup where she led a team of twelve and grew inbound pipeline by 40% in eighteen months. She's joining a 200-person B2B software company. The audience for this bio is our website, LinkedIn, and conference programs. Tone: confident and warm, not corporate. Format: one short paragraph, 90-110 words, third person.*

The second prompt takes forty-five extra seconds to write. It produces a usable draft on the first try. The first prompt will require three or four back-and-forth exchanges to get to the same place.

Workflow design notes

The four-part structure is especially valuable for recurring tasks. If you write the same type of prompt repeatedly, build a template that pre-fills role, context, and format, leaving only the task variable to fill in each time. This is the foundation of Skills and reusable instructions, which you'll build in Section 7.

For automated workflows in Cowork, complete prompts are not optional. Claude has no opportunity to ask clarifying questions mid-run. Every piece of context that might affect the output needs to be in the prompt upfront. The habit of complete briefs you build now pays compounding dividends in Act 2.

Try this in Claude

Take a prompt you've written recently that produced a mediocre result. Rebuild it using the four-part structure. Add role, check that the task is specific, add relevant context, and specify the output format. Run both versions and compare the output quality.

Pro tips

- If Claude's response is generic, the most likely cause is insufficient context. Ask yourself: what would a smart colleague need to know to do this task well?
- For creative or persuasive tasks, role is often the highest-leverage element. "Write a pitch" and "Write a pitch as a seasoned venture capitalist who has heard a thousand pitches" produce noticeably different outputs.
- Output format is the most commonly skipped element. Specifying format prevents Claude from making layout decisions you'll have to undo.
- You don't need all four elements every time. A simple task with obvious context doesn't need a full brief. Use the structure when you notice your prompts are producing outputs you have to heavily revise.

Quick summary

The four-part prompt structure, role, task, context, and output format, turns vague requests into clear briefs. It's not a rigid formula but a checklist for noticing what's missing. Complete prompts produce better first drafts, reduce revision time, and become the foundation of reusable templates and automated workflows.

Section 2 — Prompting & Iteration

Examples and Constraints: The Fastest Way to Align Claude with What You Actually Want

Examples and Constraints: The Fastest Way to Align Claude with What You Actually Want

Words describing what you want are useful. An actual example of what you want is better. And a clear statement of what you don't want is often the most underrated prompt element of all. Examples and constraints are not optional polish. They're the fastest route to outputs that match

your expectations on the first try.

Why this matters

Claude has seen an enormous amount of text during training. When you describe a style, a format, or a type of output, Claude picks the most common interpretation of that description. Its interpretation and yours may not match. An example removes the ambiguity. A constraint narrows the space before Claude starts generating, which is far more efficient than correcting the output after.

The practical benefit: fewer revision rounds, more usable first drafts, and prompts that consistently produce what you need even when you hand them to someone else or build them into an automated workflow.

The idea in plain English

Examples show Claude what you mean. They anchor tone, length, structure, and quality in a concrete way that descriptions can't fully achieve.

You can provide examples of: - The style or voice you want ("write in a tone similar to this excerpt") - The structure you want ("format each entry like this sample entry") - The quality level you want ("here's a good version and a weak version; match the good one") - What good looks like for your specific context ("here are three subject lines we've used that performed well")

You can also use negative examples: "don't write like this" is as useful as "write like this," sometimes more.

Constraints tell Claude what not to do, what limits to respect, and what rules apply. They prevent Claude from making reasonable default choices that are wrong for your situation.

Common useful constraints: - Length limits ("no more than 150 words," "fit on one slide") - Format restrictions ("no bullet points," "no headers," "plain prose only") - Content restrictions ("do not include pricing," "avoid mentioning competitors by name") - Tone restrictions ("professional but not stiff," "direct, not aggressive") - Scope limits ("focus only on the Q2 results, not full-year context")

How this works in Claude

Include examples inline in your prompt, before or after your main request. Claude will pick up on them and apply them.

Using a style example:

> *Write a product update email to our customers. Match the tone and length of this example: [paste example]. Constraints: no more than 200 words, no exclamation points, do not mention the previous version's bugs.*

Using a structural example:

> *Create three case study summaries using this structure: [paste one completed case study as a template]. Each summary should be 80-100 words. Do not include client names.*

Using negative examples:

> *Write a performance review summary. Avoid this kind of vague language: "John is a team player who always brings his best." Be specific about outcomes and behaviors.*

Practical example

A content team needs to produce ten LinkedIn posts for a founder's account over the month. Without examples or constraints, they'd get ten posts that are technically correct but generic in tone.

Instead, the prompt includes: - Three examples of the founder's actual high-performing posts - A constraint against hashtags ("we don't use them") - A constraint on length ("100-150 words, one idea per post") - A note on voice ("first person, direct, occasionally self-deprecating, never salesy")

The output is far closer to what the founder would actually say. Editing time drops from twenty minutes per post to three.

Workflow design notes

Examples and constraints are especially powerful in templates and reusable prompts. When you build a recurring workflow, embedding examples in the template means every run benefits from those anchors, not just the runs where you remember to include them.

One practical note on length: very long examples can push other important context out of Claude's attention in a long prompt. Use the most illustrative excerpt rather than pasting an entire document when brevity will serve. Two clear sentences of example text are often more useful than ten paragraphs.

For automated Cowork workflows, constraints are particularly important because there's no human in the loop to catch Claude doing something technically correct but contextually wrong. Lock down the behaviors that matter with explicit constraints, and Claude's automated outputs will require less cleanup.

Try this in Claude

Find a recent Claude output you had to significantly revise. Identify the element that was off: was it tone? Length? Structure? Format? Write a constraint that would have prevented that problem. Then rerun the prompt with that constraint added and compare the results.

Bonus: find or create a strong example of the output type you needed. Add it to the prompt as an anchor and run it again. Note how much the quality shifts with the example in place.

Pro tips

- The best examples come from your own past work. A strong piece of writing you've already produced is worth more as an anchor than any generic description of quality.
- Constraints work better when they're specific. "Keep it concise" is not a constraint. "No more than 100 words" is.
- Don't over-constrain. If you specify every element, Claude becomes a formatter rather than a collaborator. Leave room for Claude to fill in where its judgment is actually useful.
- Negative examples are underused. If you have a clear sense of what bad looks like in your context, show Claude and say "not this."

Quick summary

Examples anchor Claude's output to your actual expectations. Constraints prevent Claude from making default choices that are wrong for your context. Both are more efficient than revising bad output after the fact. Embed them in recurring prompt templates and automated workflows to make quality consistent rather than variable.

Section 2 — Prompting & Iteration

Asking for Structured Output: Tables, Outlines, and Checklists

Asking for Structured Output: Tables, Outlines, and Checklists

Claude defaults to prose. Left to its own devices, it will give you paragraphs. Sometimes that's exactly what you want. Often, it isn't. Knowing when and how to ask for structured output, and specifically what kind of structure, produces results that are immediately usable rather than requiring a reformatting step before you can do anything with them.

Why this matters

Prose is the right format for explanations, narratives, and analysis. It's the wrong format for comparing options, tracking steps, prioritizing lists, or presenting information to an audience that needs to scan rather than read. When you ask for the wrong format, you get an answer that's technically correct but practically awkward. You either reformat it manually or use a less-than-ideal output. Neither is a good use of your time.

The larger point: output format is a decision, not an afterthought. Making it explicitly, as part of your prompt, consistently improves the usability of Claude's responses.

The idea in plain English

Claude can produce several types of structured output on request. The most commonly useful ones for professional work:

Tables are ideal for comparisons, options analysis, feature matrices, risk registers, and any situation where you're mapping one dimension against another. Ask for a table when you need to compare multiple things across consistent attributes.

Outlines are ideal for planning documents, presentation structures, meeting agendas, report frameworks, and any deliverable that has a hierarchy of ideas. An outline gives you the skeleton; you (or Claude, in a follow-up) fill in the content.

Checklists are ideal for processes, review steps, launch readiness, audit criteria, and anything that needs to be done in sequence or verified item by item. A checklist is also a useful way to convert a complex procedure into something a team can actually follow.

Numbered lists work for rankings, sequential steps, or any ordered set where the sequence matters.

Bullet lists work for unordered sets of equivalent items: key points, options, features, considerations.

JSON or structured data formats are useful if the output will be processed programmatically or imported into another tool. (Less common for non-technical workflows, but worth knowing.)

How this works in Claude

State the format explicitly in your prompt. Don't hint at it; name it.

Requesting a table:

> *Create a comparison table with these five project management tools as rows and these four criteria as columns: pricing model, maximum users on the free tier, offline access, and native Gantt chart support.*

Requesting an outline:

> *Create a detailed outline for a 30-minute onboarding presentation for new sales hires. Include section headings and two to three sub-points per section. The presentation should cover: our product, our ICP, our sales process, and common objections.*

Requesting a checklist:

> *Write a pre-launch checklist for a SaaS product going live to a waiting list of 500 early adopters. Include categories for technical readiness, customer communication, support preparedness, and monitoring.*

Practical example

A project manager needs to present three vendor options to her leadership team. She asks Claude: "Compare these three vendors."

Claude returns three paragraphs, one per vendor, each with similar information presented in a different order and with different emphasis. The leadership team has to read all three paragraphs carefully to extract the comparison. The PM reformats it into a table herself.

She reruns the prompt: "Create a comparison table with vendors as rows and these attributes as columns: annual cost, implementation timeline, support model, integration with Salesforce, and references available." Claude returns a clean table. She pastes it directly into her deck.

Same information. Different format request. One required manual work; the other produced a directly usable output.

Workflow design notes

Structured output requests are particularly valuable in Cowork automations and scheduled tasks. When Claude is generating a weekly report or a daily briefing without a human in the loop, receiving a well-structured table or checklist is far more useful than receiving unstructured prose that needs interpretation.

Specify the structure in your automation prompts as precisely as you would in a one-off conversation. The output will be consistent across every run.

One practical note: very complex tables with many columns can sometimes be rendered inconsistently in markdown. If you're building something that will be pasted into a document or tool, verify the table renders correctly in your target format.

Try this in Claude

Pick any topic where you currently receive prose from Claude and think about whether it would be more useful as structured output. Rewrite the prompt specifying the exact format: a comparison table, a numbered checklist, or a hierarchical outline. Compare the usability of the two outputs.

A useful starter: "Convert this [existing document or list I'll paste] into a checklist organized by [category]." This is often faster than asking Claude to generate content from scratch and immediately shows the value of format control.

Pro tips

- Combine format requests with constraints: "A table, no more than six rows, focusing only on the three most important attributes" produces cleaner output than an unconstrained table request.
- Outlines are underused as a first step for long documents. Ask for an outline first, approve or edit the structure, then ask Claude to expand each section. This produces much better long-form output than asking for the full document in one shot.
- If you need a checklist that will be used repeatedly, ask Claude to format it in a way that's copy-paste ready for your specific tool (Notion, Confluence, a Word doc).
- Tables work especially well when you specify the exact column headers upfront. Claude will organize its thinking around those headers rather than deciding what categories to use.

Quick summary

Asking for structured output is a specific skill that prevents unnecessary reformatting work. Tables work for comparisons. Outlines work for hierarchical planning. Checklists work for processes and verification. Numbered and bulleted lists work for ordered and unordered sets. Name the format explicitly in your prompt and Claude will produce something immediately usable rather than something you have to restructure before you can act on it.

Section 2 — Prompting & Iteration

Using Follow-Up Questions to Get Where You Actually Need to Go

Using Follow-Up Questions to Get Where You Actually Need to Go

A single prompt almost never produces a finished result. That's not a flaw; it's how good collaborative thinking works. The skill isn't writing one perfect prompt. It's knowing how to continue a conversation productively until you have something genuinely useful. Follow-up questions are where most of the real work happens.

Why this matters

People who treat each Claude interaction as a one-shot transaction consistently get lower-quality output than people who treat it as a conversation. The first response is a starting point. What you do next determines whether you end up with something excellent or something mediocre that you'll have to heavily revise.

The follow-up skill is also what separates active use of Claude from passive use. Passive: read the first response, accept or reject. Active: shape, redirect, push back, deepen. Active produces dramatically better results.

The idea in plain English

Follow-up questions fall into a few categories. Knowing which type to use in a given situation is the core skill.

Refinement: You liked the direction but need adjustments. "Make the tone more direct." "Shorten this by half." "Remove the bullet points and write it as prose." "The third point is weak; make it more concrete."

Redirection: The response went the wrong way. "That's not quite what I meant. I need X, not Y." "Ignore the previous framing; the actual situation is..." "Start over with this constraint in mind."

Deepening: You want more on a specific point. "Expand on the second section." "Give me three specific examples of what that looks like in practice." "Walk me through the reasoning behind that recommendation."

Challenging: You want Claude to pressure-test its own output. "What are the weaknesses in this argument?" "What would a skeptic say about this plan?" "Where could this go wrong?"

Redirecting scope: The response was at the wrong level of detail or in the wrong direction. "Too tactical; I need the strategic framing." "Too abstract; give me a concrete example." "I already know the background; get to the recommendation."

Extending: Building on a good response to produce the next piece. "Now write the executive summary that introduces this analysis." "Using that framework, evaluate this specific case."

How this works in Claude

Claude maintains context within a conversation. Each follow-up builds on what came before. You don't need to re-paste earlier content; Claude remembers it. You can refer to specific parts of its previous response directly: "In the second paragraph, the claim about X is too strong; soften it."

This context window does have limits in very long conversations, and Claude may eventually lose track of details from early in a thread. For extended work, periodic brief summaries help: "We established earlier that our target audience is X and our constraint is Y. With that in mind..."

Practical example

A communications manager needs a crisis response email after a service outage. She starts with a basic prompt and gets a reasonable draft, but it's too apologetic and doesn't clearly communicate the timeline.

Follow-ups: 1. "The tone is too apologetic. We take responsibility but don't want to sound like we're groveling. Rewrite with confidence." 2. "Add a specific timeline: the issue began at 2pm ET, was identified at 3:30pm, and resolved at 5:45pm. Make that timeline clear." 3. "The closing paragraph feels generic. Replace it with something that specifically addresses customers who had meetings or demos during the outage window." 4. "Read this back to me as if you're a customer who received it and tell me whether it restores trust or makes things worse."

Each follow-up tightens the draft. By the fourth exchange, she has something she's confident to send. The first draft was never going to get her there; the conversation did.

Workflow design notes

In Cowork automated workflows, follow-up conversations don't exist in the same way. The prompt runs, Claude responds, and that's the output. This is why investing in prompt quality upfront matters so much for automation: you can't course-correct mid-run.

For interactive Chat work, however, the ability to follow up is a feature to use, not a limitation to apologize for. Budget multiple exchanges for anything that matters. A five-turn conversation producing excellent output is more efficient than one prompt producing acceptable output that you then revise manually.

One practical note: if you find yourself making the same follow-up corrections repeatedly across different conversations, those corrections should be baked into your base prompt as constraints. "Always be direct, not apologetic" as a standing instruction is more efficient than adding it as a follow-up every time.

Try this in Claude

Take any recent Claude conversation where you accepted the first response without following up. Reopen it and challenge the output: "What's the weakest part of what you just produced?" or "What would someone who disagrees with this conclusion say?" Note whether the subsequent exchange produces something more useful than what you originally settled for.

Pro tips

- If a response is in the wrong direction entirely, it's faster to redirect with "actually, start over and approach it this way" than to try to fix a bad draft through incremental corrections.
- "What am I missing?" is a reliably useful follow-up for any analysis or recommendation.

• "Steelman the opposing view" is useful when Claude has produced a one-sided argument and you want to understand the other side.
• When a follow-up doesn't produce what you expected, consider whether your follow-up itself was clear enough. The same four-part structure principles apply to follow-ups.

Quick summary

Follow-up questions are where productive Claude conversations happen. Use them to refine, redirect, deepen, challenge, and extend initial responses. Knowing which type of follow-up to use and when is the core skill. For automated workflows, front-load this thinking into the initial prompt since there's no opportunity to course-correct mid-run.

Section 2 — Prompting & Iteration

Critique and Revision Loops: Using Claude to Improve Claude's Own Output

Critique and Revision Loops: Using Claude to Improve Claude's Own Output

One of the more useful and underused Claude techniques is asking Claude to critique something it just produced. Claude can identify weaknesses in its own output, apply a different evaluative lens, and revise against specific criteria. When you build this into your workflow deliberately, you consistently arrive at better results than accepting the first draft.

Why this matters

The first thing Claude produces is shaped by its interpretation of your prompt. That interpretation is often slightly off, and the output reflects it. Critique and revision loops catch that misalignment before it becomes your problem to fix manually.

More practically: professional writing, business analysis, and strategic communications benefit from multiple passes. The critique pass forces a different evaluative mode. It catches things that a direct revision prompt might miss. And asking Claude to argue against its own output produces a level of rigor that single-pass generation rarely achieves.

The idea in plain English

A revision loop works in stages:

Stage 1: Generate. Ask Claude to produce the initial output. Don't over-specify in this stage; let Claude show you what it would produce with the information you've given.

Stage 2: Critique. Ask Claude to evaluate what it just produced against specific criteria. "What are the three weakest parts of this?" or "Read this as a skeptical [audience type]. What would they find unconvincing?" or "Does this actually answer the question I asked, or does it answer an easier version of it?"

Stage 3: Revise. Use the critique to drive a targeted revision. "Rewrite it, fixing the issues you identified." Or, if you have your own critique to add: "In addition to your own notes, also [your specific change]. Now revise."

Stage 4 (optional): Second opinion. "Now read the revised version with the same critical eye. What's still weak?"

You don't always need all four stages. For low-stakes work, one critique-and-revise cycle is often enough. For high-stakes outputs, two or three passes are worth the investment.

How this works in Claude

Claude is reasonably good at self-critique when given a clear evaluative frame. Vague critique prompts ("is this good?") produce vague assessments. Specific frames produce useful ones.

Useful critique frames: - "Read this as [specific audience]. What would they find missing, unconvincing, or off-putting?" - "Does this have a clear recommendation, or does it hedge?" - "What claims in this document are not supported by the evidence provided?" - "Is the structure logical, or does it bury the main point?" - "Rate the opening paragraph on: hook quality, clarity, and whether it makes the reader want to continue." - "What would a good editor cut from this?"

After the critique, you can either ask Claude to revise directly, or take the critique as input and write your own revised prompt with the issues addressed.

Practical example

A senior analyst needs to produce a strategic memo recommending one of three market expansion options for her leadership team.

She asks Claude to draft a recommendation. The draft is thorough but buries the recommendation on page two and spends too much time on methodology that the leadership team doesn't need.

She runs a critique prompt: "Read this memo as a time-pressed C-suite executive. What's wrong with it structurally, and what information is here that they don't need?"

Claude identifies: the recommendation should lead, not follow the analysis; the methodology section is three times longer than necessary; and the risk section doesn't include a mitigation for

the highest-priority risk.

She then asks: "Revise the memo with the recommendation leading, cut the methodology section to two sentences, and add a mitigation for the market entry risk." The revised version is exactly what she would have produced herself after reading Claude's first draft and rewriting it, except she spent ten minutes instead of sixty.

Workflow design notes

Critique loops are most valuable for outputs that will be shared externally, presented to decision-makers, or used as the basis for action. For internal rough notes and quick summaries, the overhead may not be worth it.

For Cowork automations, you can build critique into a multi-step workflow: step one generates the output, step two runs a structured critique prompt against it, step three produces a final revised version. This is overkill for daily routine tasks but powerful for scheduled reports or documents that need consistent quality without human review.

Build your critique criteria based on what you've actually had to fix in the past. The best critique prompts come from remembering the specific mistakes Claude has made in your context.

Try this in Claude

Take any Claude output from today or this week that you edited significantly. Paste the original output back into Claude and say: "Tell me what's weak about this and why you think I edited it." Then say: "Now produce a revised version that addresses those issues."

Compare the revised version to your own edited version. Note where Claude's revision and your edits converged, and where they diverged. The divergences tell you where your own judgment adds value beyond what Claude can self-correct.

Pro tips

- Use role-based critique frames for audience-specific work: "Read this as a lawyer looking for liability exposure" or "Read this as a technical evaluator who cares about implementation feasibility."
- If Claude's self-critique is too gentle or general, push it: "Be more critical. What are the genuine weaknesses, not just the minor improvements?"
- Critique prompts also work for your own writing, not just Claude's. "Here's a document I wrote. Read it as a critical editor and tell me what's not working."
- When you find yourself running the same revision request repeatedly (always shortening, always removing hedging language, always making the recommendation clearer), add those as constraints to your base prompt. The critique loop teaches you what your prompts are missing.

Quick summary

Critique and revision loops produce better output than single-pass generation. Ask Claude to evaluate its output against specific criteria, then revise based on the findings. The critique frame matters: vague asks produce vague critiques; specific evaluative lenses produce actionable ones. For high-stakes outputs, two passes are almost always worth the extra five minutes. For repeated tasks, let the critique results inform your base prompt to reduce the need for loops over time.

SECTION 03 · ACT 1 — CHAT

Section 3 of 14

Section 3: Voice, Files & Everyday Knowledge Work

Section 3: Voice, Files & Everyday Knowledge Work

Claude's value isn't limited to what you can type into a chat window. This section extends Claude into two of the most common ways professionals actually work: capturing ideas through voice, and making sense of documents they already have.

Why this section matters

The friction of switching to Claude often lies not in Claude itself but in the input methods. People don't think "I'll use Claude for this" when they're rushing between meetings, talking through an idea on the way to the office, or staring at a 60-page PDF they need to understand before tomorrow. This section removes that friction by showing how Claude works with the ways you already think and the materials you already have.

What you'll learn

Using voice on desktop and mobile. Claude supports voice input across devices. You can dictate directly rather than type. This is faster for many people and opens Claude up to moments where typing isn't practical.

Turning speech into notes, plans, and drafts. The real skill is not transcription but transformation. A rough verbal brain dump can become structured meeting notes, a project brief, a decision document, or a draft email. Claude handles the translation from spoken language to organized written output.

Uploading files and documents. Claude accepts PDFs, text files, images, and more. This lets you bring existing material into Claude's context window and work with it directly rather than re-describing or summarizing it yourself before asking questions.

Summarizing PDFs, notes, and transcripts. Long documents become short, useful summaries. Meeting transcripts become action item lists. Research papers become executive summaries. Claude handles the compression.

Extracting action items, risks, and follow-up questions. Beyond summarization, Claude can extract specific types of information from documents: what needs to be done, what could go wrong, what questions remain unanswered. This turns reading into working.

The mini project

You'll dictate a rough idea by voice, add a supporting document or set of notes, and have Claude produce three outputs: a summary of the combined material, an action list, and a follow-up plan. The deliverable demonstrates how voice capture and document processing can compress hours of knowledge work into minutes.

How to approach this section

Voice and file use are practical, not conceptual. The best way to learn them is to try them on real material you actually have. If you have a document on your desk right now that you need to process, use it. If you have a meeting coming up that you'd normally rush to take notes on, practice the voice capture workflow first.

By the end of this section, Claude should feel like a natural part of how you process information, not a separate step you have to remember to take.

Section 3 — Voice, Files & Everyday Knowledge Work

Using Voice with Claude: Faster Input, Less Friction

Using Voice with Claude: Faster Input, Less Friction

Typing is not the only way to work with Claude. Voice input works across Claude's desktop and mobile interfaces, and for many tasks it's significantly faster and more natural than typing. The barrier between thinking and acting shrinks considerably when you don't have to translate thoughts into keystrokes before Claude can work with them.

Why this matters

A lot of Claude's potential value is lost not because Claude can't do the task, but because using it requires switching contexts, sitting down at a keyboard, and composing a prompt from scratch. Voice removes at least two of those three barriers. You can talk to Claude while walking to your next meeting, capture an idea while it's still fresh, or dictate a rough draft of something that would take much longer to type.

This is not about replacing thoughtful prompting with rambling speech. It's about making Claude accessible in more moments and reducing the friction cost of starting.

The idea in plain English

Claude's web app and mobile app both support voice input. On mobile, the microphone button in the message input area lets you dictate rather than type. On desktop, your operating system's voice dictation tools feed text into the Claude input field, and some browsers support native voice-to-text input directly.

Dictation for Claude works like dictation for any text field: you speak, it transcribes, and your words appear as the prompt. You can review and edit before sending, or send the transcription directly for quick tasks.

The key shift in mindset: you don't need to speak in perfectly formed sentences. Claude handles loosely structured voice input well. A rambling description of a problem, a stream-of-consciousness brain dump, or a series of half-formed thoughts are all workable inputs. Claude's job is to make sense of them.

How this works in Claude

On mobile (iOS and Android): Open the Claude app. In the message input area, tap the microphone icon. Speak your prompt. Review the transcription. Tap send. Claude responds in text, which you can read, copy, or share.

On desktop: Use your operating system's built-in dictation feature (Windows voice typing with Win+H, or macOS Dictation in System Settings) to dictate into the Claude message input field. Alternatively, browsers like Chrome support voice input on some platforms.

In Cowork: Voice input works in the same way through the chat interface. Dictated prompts trigger Cowork's full capability set, including file access, connectors, and scheduling.

Practical example

A product manager is commuting and has twenty minutes before her next call. She has to prepare a quick status update for her team. Instead of waiting until she's back at her desk, she opens Claude on her phone and dictates:

"Hey, I need to put together a quick team status update. We're three weeks into the sprint, we've shipped the new dashboard feature, the analytics integration got delayed by about a week because of a data schema issue, and we're still on track for the overall release. Can you write up a short status update, maybe four or five bullet points, that I can paste into Slack?"

She reviews the transcription, sends it, and Claude returns a clean status update in thirty seconds. She posts it to Slack before the call starts. Total time: three minutes instead of ten.

Workflow design notes

Voice input is particularly effective for: - Quick tasks you'd otherwise postpone ("I'll do that when I'm at my desk") - First drafts of messages, updates, or documents where the raw content matters more than perfect structure - Brain dumps where you have a lot of information in your head that needs organizing - Reviewing and responding to items while away from your desk

It's less effective for: - Tasks requiring precise formatting or structured prompts where typing is faster than speaking formatting instructions - Long prompts with specific technical requirements where review is critical - Situations where speaking aloud isn't practical

One quality note: review transcriptions before sending for anything important. Voice recognition makes errors, especially on proper nouns, technical terms, and unusual words. A thirty-second review prevents the occasional embarrassing transcription mistake.

Try this in Claude

Pick a task you currently delay because it requires sitting down to type something: a quick update, a note to a colleague, a draft for something you've been putting off. Do it by voice instead, right now, on your phone. Note whether the friction reduction changes when you would have done it.

Pro tips

- Speak in chunks: short sentences are transcribed more accurately than long, complex ones.
- Don't worry about "ums" and "uhs"; Claude filters them when generating a response.
- For recurring voice workflows (daily updates, quick memos), save the corresponding Claude prompt as a template so you only need to dictate the variable content.
- On mobile, the Claude app keeps context across voice and text inputs in the same conversation. You can dictate a rough description and then type a more precise follow-up refinement.

Quick summary

Voice input makes Claude accessible in more moments and reduces the friction cost of getting started. It works on mobile and desktop, handles loosely structured speech, and is particularly valuable for quick tasks you'd otherwise postpone. Review transcriptions before sending anything important, and use voice most for capturing ideas and first drafts rather than for prompts that require precision.

Section 3 — Voice, Files & Everyday Knowledge Work

Turning a Voice Dump Into Something Usable

Turning a Voice Dump Into Something Usable

Dictating to Claude is useful on its own. But the real value comes from the transformation: raw, unstructured speech becoming organized notes, a project brief, a decision document, or a polished draft. Claude's ability to handle messy verbal input and return clean structured output is one of its most underappreciated capabilities.

Why this matters

Human speech is not structured. When you think out loud, you loop back, contradict yourself, use vague references ("that thing we talked about"), jump between ideas, and leave sentences unfinished. Transcribing that verbatim produces something that's technically your words but practically unusable.

Claude doesn't transcribe. It interprets, organizes, and produces something your audience can actually use. The cognitive gap between "thoughts in my head" and "document on paper" has never been smaller.

The idea in plain English

The workflow is simple. You speak a rough version of your content into Claude. You tell Claude what kind of output you want. Claude produces it.

The transformation can be as light or as heavy as the material requires:

Light transformation: "Clean this up and fix the grammar." The structure stays; Claude just makes it readable.

Medium transformation: "Organize this into three sections: background, recommendation, and next steps." Claude imposes structure on loosely organized speech.

Heavy transformation: "I rambled about a project for three minutes. Extract the key decisions, open questions, and action items." Claude identifies signal in noise.

The specific output types most commonly useful in professional work: - Meeting notes from a verbal recap after a meeting - Project brief from a voice description of a project - Decision document from a rambling thinking-out-loud session - Email or message draft from a verbal explanation of what you want to communicate - Task list from a verbal dump of everything on your mind

How this works in Claude

Tell Claude both what the input is and what you want from it.

Example prompt (dictated or typed):

> *"I just had a really long call with a client. Here's what happened: [voice dump of key points]. Turn this into a meeting summary with sections for key discussion points, decisions made, and action items with owners."*

Example prompt for a project brief:

> *"I have this idea for a new feature we should build. Let me just talk through it: [voice dump]. Take what I said and write it up as a one-page project brief with sections for problem, proposed solution, success metrics, and open questions."*

Claude will organize the material, fill in structural gaps, and produce something that looks like it was deliberately written, not dictated carelessly.

Practical example

A consultant finishes a two-hour client workshop. She has scribbled notes, half-formed thoughts, and a vague memory of several decisions that were made. Instead of typing up meeting notes for an hour, she records a five-minute voice memo in the car: "Okay so the big things were... we decided to go with Option B for the org design, but there were still concerns from the operations team about the transition timeline. The CEO wants a 30-day plan from us by next Friday. There were also like three things the team said were non-negotiable: keeping the current reporting structure for at least six months, making sure the two senior managers are promoted as part of the transition, and getting HR buy-in before anything is announced. Oh, and someone mentioned budget but we didn't resolve that."

She pastes the transcription into Claude: "Turn this into meeting notes with three sections: key decisions, non-negotiables identified by the client, and action items with owners and deadlines."

Claude returns clean, organized meeting notes in fifteen seconds. She reviews, adds one detail she missed, and sends them to the client. What would have been a 45-minute documentation task took ten minutes.

Workflow design notes

Voice-to-document workflows have a few quality considerations worth managing:

Context completeness: Claude can only work with what you say. If your verbal dump omits critical context, the output will too. Make a habit of including: who was involved, what decision was being made, what the time frame is, and any constraints that should shape the output.

Specificity of transformation instructions: "Write this up as notes" is vague. "Write this as a client-ready meeting summary with action items formatted as: [owner] will [action] by [deadline]" is specific. The more specific the output format, the more directly usable the result.

Review for accuracy: Claude may smooth over ambiguity in your speech in a way that's clear but slightly off. Read the output as if you're the recipient, not the person who knows what you meant.

Try this in Claude

After your next meeting or call, take five minutes and dictate a verbal recap of what happened: key points, decisions, outstanding questions. Don't try to be organized; just say what you remember. Then send it to Claude with a specific transformation instruction. Compare the result to what you would have written manually.

Pro tips

- For recurring meeting types (weekly team syncs, client calls, one-on-ones), build a template for the transformation prompt. Always produce the same format, so the output is always ready to paste directly where it needs to go.
- If you're driving or otherwise unable to review before sending, add "flag anything that seems unclear or ambiguous in the source" to your transformation request. Claude will note gaps in your verbal input.
- Voice memos from your phone can be transcribed with your phone's built-in transcription or a third-party tool, then pasted into Claude if the Claude mobile voice input isn't available or convenient.
- Long verbal dumps work better when you give Claude organizing instructions upfront rather than asking it to figure out structure on its own.

Quick summary

Dictating to Claude and asking for a transformation produces organized, usable documents from messy verbal input. Meeting notes, project briefs, decision documents, and draft communications are all achievable from a five-minute voice dump. The quality of the output depends on the completeness of the verbal input and the specificity of the transformation instruction. For recurring meeting types, build a reusable transformation template.

Section 3 — Voice, Files & Everyday Knowledge Work

Uploading Files: Bringing Your Documents Into Claude's Context

Uploading Files: Bringing Your Documents Into Claude's Context

Describing a document to Claude and then asking questions about it is like explaining a painting to someone who isn't in the room. You're doing most of the work and introducing your own interpretive bias before Claude even gets involved. The better approach is to hand Claude the document directly. File upload puts your actual material in front of Claude so it can work with what's really there.

Why this matters

The most common underuse of Claude in professional settings is not giving it the source material. People describe the contents of a report instead of uploading it. They summarize a contract instead of sharing it. They paraphrase a client brief instead of pasting it in. Every layer of translation between the original document and Claude's context introduces error, limits depth, and makes Claude's responses less precise.

Uploading files removes that friction. Claude reads what's actually there, not your description of it.

The idea in plain English

Claude accepts a range of file types: PDFs, text files, Word documents, images, spreadsheets, and more. When you upload a file, its content enters Claude's context window and Claude can read, reference, and reason about it directly.

This means you can: - Ask questions about a specific document ("What does Section 4 say about termination clauses?") - Ask Claude to summarize, extract, or reorganize the document's content - Ask Claude to compare multiple documents you upload - Use the document as context for a task ("Draft a response to this RFP") - Ask Claude to find specific information without reading the whole thing yourself

The context window has a size limit, which means very long documents may be partially processed depending on the tier and model. For most professional documents, PDFs up to several hundred pages, this is not a practical constraint. For very large document sets, either summarize in stages or use Projects (covered in Section 5) to manage the scope.

How this works in Claude

In the chat interface, look for the attachment icon (a paperclip or plus sign, depending on the interface version) in the message input area. Click it, select your file, and it uploads. Once uploaded, it appears in the conversation as a reference that persists throughout that thread. You can then type your prompt or question as you normally would.

File uploads work in both Chat and Cowork. In Cowork, Claude can also access files directly from your connected folders rather than requiring you to upload them explicitly each time.

Supported file types include: PDF, DOCX, TXT, CSV, XLSX, images (PNG, JPG, and others), and plain text in various formats. If a file type isn't directly supported, copying the text content and pasting it works as a fallback.

Practical example

An HR manager receives a 30-page employee handbook from a newly acquired company. She needs to identify everything that conflicts with her own company's policies before integration. Reading both documents manually would take most of a day.

She uploads the acquired company's handbook and her own policy document to Claude in the same conversation. She asks: "Compare these two documents and identify any areas where the policies conflict or are meaningfully different. Focus on compensation, termination, and leave policies."

Claude reads both, identifies fifteen specific areas of divergence, and formats them as a comparison table with the relevant excerpts from each document. The review that would have taken hours takes twenty minutes.

Workflow design notes

A few practical considerations for file uploads:

File quality matters. Scanned PDFs where the text hasn't been OCR-processed are read as images rather than text. Claude can handle this, but text extraction quality is lower. When possible, use text-based PDFs rather than scanned images.

Sensitive content considerations. As covered in Section 1, think before uploading documents that contain confidential business information, client data, or personally identifiable information. The same privacy principles apply here.

Context window management. If you're uploading multiple large documents, prioritize what's most relevant. Too many long documents in a single context can dilute Claude's attention across them. For complex multi-document work, organize by topic and work through them in focused sessions.

Referencing across sessions. File uploads persist within a conversation but not across conversations. If you'll need the same document across multiple sessions, consider a Claude Project where uploaded reference material stays available.

Try this in Claude

Find a document you need to process this week: a report to summarize, a contract to review, meeting notes to organize, or an email thread to make sense of. Upload it to Claude and ask one specific question about it rather than a general "summarize this." Notice how much more precise the response is when Claude is working from the actual document rather than your description of it.

Pro tips

- For long PDFs, tell Claude which section to focus on if you only need part of it. "Focus on Section 3, which starts on page 14" prevents Claude from spreading attention across the whole document when you only need one part.
- You can upload multiple files in the same conversation and ask Claude to work across them. This is especially useful for comparison tasks.
- Images of whiteboards, hand-drawn diagrams, or handwritten notes can be uploaded as images and Claude will interpret their content.
- When uploading a document for a specific task, include the task in the same message as the upload rather than uploading first and asking later. This gives Claude context to apply immediately.

Quick summary

Uploading files puts your actual documents in front of Claude instead of your description of them. This produces more precise responses, enables direct extraction and comparison, and removes a layer of interpretive translation. Use file uploads for any task where you have relevant source material rather than rebuilding that material as text in your prompt.

Section 3 — Voice, Files & Everyday Knowledge Work

Summarizing Long Documents: PDFs, Notes, and Transcripts

Summarizing Long Documents: PDFs, Notes, and Transcripts

Reading everything is not a strategy. It's a way of slowly falling behind. Claude's summarization capability is not about laziness; it's about directing attention to the right level of detail at the right time. Knowing when and how to use Claude for summarization is one of the highest-leverage productivity habits available to knowledge workers.

Why this matters

The volume of text that passes through a professional's week is staggering: reports, proposals, research papers, meeting transcripts, email threads, policy documents. Reading all of it carefully is neither realistic nor valuable in most cases. The question is not "should I read this?" but "what do I need from this, and at what level of detail?"

Claude answers that question efficiently. A 50-page industry report becomes a five-point summary in two minutes. A two-hour meeting transcript becomes a one-page brief. A research paper becomes a three-paragraph plain-language explanation. You then decide whether the summary is sufficient or whether you need to read the original more closely for specific sections.

The idea in plain English

Summarization is not a single task. It breaks into several subtypes, each with a different use case:

Executive summary: The main points and conclusions, written for someone who needs the big picture without detail. Best for decision-makers and stakeholders who need to be informed but not briefed in depth.

Key takeaways: A bulleted list of the most important points. Best for situations where people need to know what the document contains without reading the narrative.

Plain-language translation: Converting technical, legal, or academic text into language a non-specialist can understand. Best for documents written for a different audience than the one you're serving.

Audience-specific summary: Extracting and presenting only the parts relevant to a specific reader or use case. A 100-page vendor contract summarized for "what does this say about data handling and liability" is more useful than a general summary.

Timeline and chronology: Reconstructing the sequence of events from a document. Best for incident reports, legal filings, and narrative accounts.

Decision summary: What decisions were made, by whom, on what basis. Best for meeting transcripts and board minutes.

Each type can be requested explicitly, which produces much better output than asking Claude to "just summarize" without specifying what kind of summary you need.

How this works in Claude

Upload the document or paste its content. Then specify the type of summary you want, the target length, and the intended audience.

Example prompts:

For a research paper:

> *"Summarize this paper in plain English in under 300 words for a business audience with no academic background. Focus on the practical implications, not the methodology."*

For a meeting transcript:

> *"Summarize this meeting transcript as a decision summary. What decisions were made, who owns each one, and what were the key discussion points that led to each decision?"*

For a long contract:

> *"I'm the buyer in this contract. Summarize the terms that are most relevant to me: payment, delivery timelines, cancellation terms, and any clauses that limit my rights or expose me to liability."*

For a lengthy report:

> *"Extract the five most important findings from this report. Present each finding as a one-sentence statement followed by two to three sentences of supporting detail."*

Practical example

A compliance officer needs to process three regulatory guidance documents released last quarter before a team meeting the following morning. The documents are 35, 47, and 28 pages respectively. She has two hours.

She uploads all three documents to Claude in the same conversation: "I need to brief my team on the key changes in these three guidance documents. For each document, produce a summary with: one paragraph overview, a bulleted list of key changes that will affect our operations, and any deadlines or compliance dates mentioned."

Claude produces a three-part brief in under a minute. She reviews it, adds a few context notes of her own, and uses it as the basis for her meeting agenda. The documents that would have consumed her evening are processed in twenty minutes.

Workflow design notes

Summarization quality depends heavily on how clearly you specify what you need. The two most important variables:

Purpose: Who is this summary for, and what will they do with it? A summary for a CEO is different from a summary for a project manager. A summary to decide whether to read something in full is different from a summary to brief a team.

Specificity: What aspects of the document matter? A general summary of a 100-page report spreads Claude's attention evenly. A targeted summary ("focus on sections 3, 7, and the appendices") gives you more depth where it counts.

For documents that will be summarized regularly (weekly reports, board minutes, research digests), build a prompt template that specifies the exact format and focus areas. Consistency in the summary format makes the summaries easier to use over time.

Try this in Claude

Find a long document you have to process but haven't yet: a report, a paper, a set of meeting notes. Upload it. Send two different summary requests: one general ("summarize the key points") and one specific ("summarize this for [specific audience] focusing on [specific aspects]"). Compare the usefulness of the two outputs.

Pro tips

- Ask Claude to "identify what the document doesn't cover" alongside the summary. This is often as useful as the summary itself for research and due diligence purposes.
- For very long documents, work in stages: summarize each section, then ask Claude to synthesize the section summaries into an overall brief.
- Meeting transcripts often contain a lot of noise. Tell Claude explicitly: "Focus on decisions and action items, not discussion." This cuts through the back-and-forth and surfaces what matters.
- If a summary seems incomplete, it may be because the document exceeded Claude's context window. Ask specifically about sections you think were missed.

Quick summary

Summarization is not a single task. Choose the right type: executive summary, key takeaways, plain-language translation, audience-specific summary, or decision summary. Specify the audience, the length, and the focus. Quality improves dramatically when you tell Claude what you need from the document rather than asking it to summarize generically. Build summary templates for documents you process regularly.

Section 3 — Voice, Files & Everyday Knowledge Work

Extracting What Matters: Action Items, Risks, and Follow-Up Questions

Extracting What Matters: Action Items, Risks, and Follow-Up Questions

Summarization tells you what happened. Extraction tells you what to do about it. The ability to pull specific types of information out of a document, rather than just compressing it, turns reading from a passive activity into an active workflow input. This is where Claude earns its place in everyday knowledge work.

Why this matters

After a meeting, a report, or a long document, there are usually three things you need to act on: what was decided or agreed upon, what could go wrong, and what still needs to be answered. These things are rarely labeled clearly in the source material. They're embedded in discussion, buried in paragraphs, or implied rather than stated.

Finding them manually requires reading carefully, taking notes, and organizing your observations. Claude can do that extraction directly, and do it in seconds, so you can move from reading to acting faster.

The idea in plain English

Extraction is targeted information retrieval: you tell Claude what type of information you need and it pulls those specific items from the document.

The three most commonly useful extraction types in professional work:

Action items: What needs to be done, by whom, and by when. From meeting transcripts, these are often buried in discussion. From project documents, they may be scattered across sections. Claude can pull them out and format them consistently.

Risks: What could go wrong, what the potential consequences are, and whether mitigations have been identified. Risk extraction is valuable from proposals, plans, contracts, and research. It finds the "however" and "but" and "assumes" hidden in documents that are otherwise optimistic.

Follow-up questions: What remains unresolved, unclear, or unanswered. This is particularly useful after meetings that ended without conclusions, after reviewing proposals with gaps, and after reading research that raises more questions than it answers.

You can also extract other specific types: commitments, dependencies, timelines, open decisions, named parties and their roles, financial figures, compliance requirements, and more.

How this works in Claude

Be explicit about what you're extracting and how you want it formatted.

Extracting action items from a meeting transcript:

> *"From this meeting transcript, extract all action items. Format each as: [Person responsible] - [Action] - [Deadline if mentioned or 'No deadline specified']."*

Extracting risks from a project plan:

> *"Identify all risks mentioned or implied in this document. For each risk, note: the risk, who or what it affects, and whether the document includes any mitigation. If no mitigation is mentioned, note that."*

Extracting follow-up questions from a research document:

> *"After reading this research report, list the five most important questions that are either left unanswered or that a critical reader should ask before relying on its conclusions."*

Combined extraction:

> *"From this board meeting transcript, produce three lists: decisions made (with who approved each), action items (with owner and deadline), and open questions that need resolution before the next meeting."*

Practical example

A strategy analyst reviews a 25-page competitive analysis prepared by an external consultant. The document is well-written but dense, and she needs to take it to her leadership team with a clear set of recommended actions and concerns.

She uploads the document and sends three extraction prompts:

- "List every recommendation this document makes, in the order it appears."
- "Identify every risk or threat the document mentions, whether explicitly labeled as such or embedded in the analysis."
- "What assumptions does this analysis make that the report itself does not validate?"

The first extraction gives her a crisp recommendation list for her leadership presentation. The second surfaces eight risks she hadn't noticed in her initial read. The third reveals three assumptions that, if wrong, would undermine the main recommendation.

She walks into the leadership meeting knowing the document better than she would have from three hours of reading alone, and she has questions prepared that demonstrate real analytical depth.

Workflow design notes

Extraction works best when you're precise about the type of information you want and the format you need.

Common mistakes: - Asking for "action items and risks and questions and dependencies" in one pass often produces a muddled result. Separate prompts for each extraction type produce cleaner outputs. - Asking Claude to extract without providing the document in context, relying instead on a description of the document, produces much lower quality results. - Accepting extracted items

without reviewing: Claude may occasionally pull something that isn't quite the type of item you asked for. A quick review catches edge cases.

Building extraction into recurring workflows: If you regularly process the same type of document (weekly reports, client calls, board minutes), build a standard extraction prompt and reuse it. The consistency of format makes the outputs easier to use over time and easier to compare across periods.

Try this in Claude

Take any document you've recently processed: a meeting summary, a proposal, a report. Upload it to Claude and run a three-part extraction: action items, risks, and open questions. Compare what Claude surfaces to what you took away from reading it yourself. Note anything Claude found that you missed, and anything it missed that you caught.

Pro tips

- "What does this document assume to be true?" is one of the most useful extraction prompts for any analytical or strategic document. Most documents embed significant assumptions invisibly.
- Ask Claude to flag items that are mentioned but unresolved: "Identify anything that was raised in this document but not concluded." This is different from extracting only items that are clearly unresolved.
- For contracts, "what obligations does this create for me that are easy to miss?" is more useful than a general risk extraction.
- Extraction prompts pair well with follow-up: once you have the extracted list, ask Claude "which of these action items is most time-sensitive?" or "which risk would be most costly if it materialized?"

Quick summary

Extraction pulls specific types of information from documents: action items, risks, follow-up questions, assumptions, commitments, and more. It turns reading into working by surfacing what needs to happen, what could go wrong, and what still needs answering. Use specific extraction prompts with clear formatting instructions. For recurring document types, build reusable extraction templates to make the output consistent and immediately actionable.

SECTION 04 · ACT 1 — CHAT

Section 4 of 14

Section 4: Connectors — Email, Calendar & the Connector Ecosystem

Section 4: Connectors — Email, Calendar & the Connector Ecosystem

Claude becomes significantly more useful when it can see what you're actually working with. Connectors give Claude access to your real email, your real calendar, and the real data from the tools you use every day. This section covers what that looks like in practice, starting with the two connectors most professionals use first.

Why this section matters

Without connectors, you describe your situation to Claude. With connectors, Claude reads your situation directly. The difference is the same as the difference between telling a colleague "I have a lot of meetings this week" and letting them look at your calendar. One produces generic advice; the other produces specific, actionable help.

Email and calendar connectors are the starting point for most professionals because those two tools contain most of the context that shapes a workday: what needs a response, what's coming up, what was decided, who said what to whom. When Claude has access to that context, it can act as a genuine operational assistant.

What you'll learn

Searching email with natural language. Ask Claude to find specific messages, threads, or information across your inbox. "Find all emails from the procurement team about the Q3 vendor review" is faster than digging through search filters manually.

Summarizing email threads. Long email threads condensed into a clear summary of what was discussed, what was decided, and what needs a response. This is one of the highest-value connector use cases for professionals managing a heavy inbox.

Drafting replies from inbox context. Claude reads the thread and drafts a reply informed by the full context of the conversation, your preferences, and any relevant information in your inbox. Not a generic reply but one grounded in what was actually said.

Finding free time and scheduling time blocks. Claude reads your calendar and identifies open windows. You can ask for specific configurations: "Find a two-hour block this week for deep work, avoiding mornings before 9am."

Analyzing calendar patterns against goals. Where is your time actually going? Claude can look across your calendar and surface how your time is allocated versus how you've said you want to spend it.

Preview: the same connectors power Dispatch in Act 2. The connectors you set up here carry into Cowork. When you use Dispatch to trigger tasks from your phone, the email and calendar connectors you configured are what Claude reaches for. This section is laying groundwork you'll use throughout the course.

The mini project

You'll design a "personal chief of staff" workflow: a structured sequence where Claude reviews your inbox and calendar, identifies your priorities for the day or week, drafts replies to time-sensitive messages, and recommends schedule changes based on what you've said your goals are. This is a practical, immediately usable workflow for anyone managing significant email and calendar volume.

How to approach this section

Connectors require a one-time setup: you authorize Claude to access your email and calendar through the connector settings. The authorization is scoped, meaning Claude gets read (and sometimes limited write) access to specific tools, not your entire digital life. Take the setup step seriously: read what permissions you're granting before confirming.

Once connected, the workflows in this section become available. Start with one, the email summary or the scheduling assistant, and see how it changes your daily routine before adding others.

Section 4 — Connectors: Email, Calendar & the Connector Ecosystem

Searching Your Inbox with Natural Language

Searching Your Inbox with Natural Language

Email search is broken by design. The standard search interface assumes you remember keywords from the subject line or body of a message. Natural language search assumes you

remember what the conversation was about. Those are very different things, and for most of the emails that matter, what you remember is the topic, not the exact wording.

Claude with an email connector changes this. You describe what you're looking for in plain language and get the messages back.

Why this matters

Retrieval failure is a real productivity tax. You know the email exists. You remember roughly when it arrived and what it was about. But you can't find it because you don't remember the right keyword to search for. You spend five minutes searching, don't find it, give up, and either ask the sender to resend or reconstruct the context from memory.

This happens multiple times a day for people managing significant email volume. Natural language search eliminates most of it.

The idea in plain English

With an email connector active, Claude can search your inbox on your behalf using the description you provide. You're not limited to exact keyword matches. You can describe the content, the sender relationship, the time frame, or the topic, and Claude will find the relevant messages.

This works because Claude understands the intent behind your description, not just the literal words. "The email from the legal team about the indemnification clause" and "the thread where someone raised concerns about our liability for data breaches" might surface the same message.

Beyond retrieval, Claude can also search to answer questions: "Has anyone emailed me about project delays this week?" is a search that returns an answer, not just a list of messages.

How this works in Claude

With the email connector enabled, you can ask Claude directly in Chat or Cowork:

- "Find all emails from Sarah Chen in the last 30 days."
- "Did anyone follow up on the proposal I sent to the Meridian account last month?"
- "Search my inbox for anything related to the office lease renewal."
- "Find emails where someone is waiting for a response from me."

Claude searches your inbox, returns the relevant results, and presents them in the chat. You can follow up with additional refinement ("those results don't look right, try searching for..." or "show me only the ones from external senders").

The connector typically reads your email but does not delete, send, or modify messages unless you have a write-enabled connector and explicitly ask it to take an action.

Practical example

A business development manager is preparing for a call with a prospective client. She wants to review everything that's been exchanged over the past six months. The company name is "Ashford Solutions" but she's not sure she's been consistent about using it in subject lines.

She asks Claude: "Find all email threads related to Ashford Solutions, including any where they might be referred to by their founder's name, James Ashford, or where someone from their domain @ashfordsolutions.com appears."

Claude surfaces twelve threads across six months, including two where the subject line didn't mention the company name at all. She reviews them in five minutes and walks into the call with full context. The manual search would have taken twenty minutes and probably missed the threads without the company name in the subject.

Workflow design notes

A few considerations for email search:

Privacy scope: Email connectors have access to everything in your connected inbox. Be thoughtful about which account you connect. If your work email contains sensitive personnel matters or confidential communications, consider whether that's the right inbox to connect, and review the permissions granted to the connector.

Search quality depends on your inbox quality: If your inbox is thousands of unread messages with no organization, search results may be less precise. Claude can still find things, but the more organized your inbox, the better the signal-to-noise ratio.

Search as a step in a workflow: Email search is most useful not as a standalone query but as the first step in a sequence. Find the thread, then ask Claude to summarize it, then ask Claude to draft a reply. These compose naturally in a single conversation.

Try this in Claude

Connect your email (if you haven't already) and run a search for a specific email you know exists but had trouble finding recently. Use a description rather than keywords. Then run a second search for "emails where someone is waiting on something from me." Note whether the results surface anything you'd forgotten about.

Pro tips

- Date-bounding your searches improves precision: "in the last two weeks" or "between March and May" helps Claude narrow the scope when you have a high-volume inbox.

• If the first search doesn't find what you want, refine with additional context rather than accepting the result. "Not quite. The email I'm thinking of had an attached document and came from someone outside the company."
• Use search to audit your own response backlog: "Find emails I received more than a week ago that I haven't replied to." This is uncomfortable but useful.

Quick summary

Natural language email search lets you find messages by describing what they were about rather than remembering exact keywords. With an email connector, Claude searches on your behalf and returns results based on your intent, not just keyword matches. Use it for retrieval, for inbox audits, and as the first step in a summarize-draft-send workflow.

Section 4 — Connectors: Email, Calendar & the Connector Ecosystem

Summarizing Email Threads: Getting to the Point Without Reading Everything

Summarizing Email Threads: Getting to the Point Without Reading Everything

An email thread with 47 messages is not a conversation. It's a document that accreted over time through people who had slightly different information at each step. Reading it top to bottom to understand the current state is a reasonable choice if you have nothing else to do. It's a terrible choice if you're busy.

Claude can read the whole thread and tell you what you actually need to know.

Why this matters

Email threads grow in ways that make them increasingly hard to parse: tangential points, people who reply-all when they should reply-one, corrections to corrections, subject lines that no longer match the content, and the occasional message from someone who clearly didn't read the previous three replies. Getting current on a long thread is surprisingly time-consuming for the actual information density it contains.

Summarization extracts signal from that noise. You get the current state, the key decisions, the outstanding questions, and any action items directed at you, without reading every message.

The idea in plain English

With an email connector, Claude can access a specific thread and summarize it according to your instructions. The summary can be:

Status summary: Where things stand right now. What's been agreed, what's still in discussion, what's blocked.

Decision log: What decisions were made in this thread, by whom, and when.

My-role summary: What's being asked of me specifically, and what (if anything) I've already committed to.

Chronological summary: The key developments in order, useful for understanding how a situation evolved.

Catch-me-up summary: You've been out. You rejoin a thread. You need to know what happened while you were gone.

The type of summary worth requesting depends on why you're looking at the thread and what you're about to do with it.

How this works in Claude

With an email connector, you can either ask Claude to find and summarize a thread in one step, or search first and then ask for a summary of the results.

Direct thread summary:

> *"Summarize the email thread with the subject 'Q3 Marketing Budget Review.' Tell me what's been decided, what's still open, and whether anything needs a response from me."*

Post-search summary:

> *(After searching) "Summarize the thread you just found. Focus on the current status and any action items."*

Catch-up summary:

> *"I was on leave for ten days. Summarize the email threads where my input is pending or where significant decisions were made without me."*

Practical example

A senior manager returns from a week-long offsite to find 200+ unread emails. Rather than reading through the inbox chronologically, she asks Claude: "I've been away for a week. Summarize the five most important email threads that involve me directly. For each one: what happened, is anything pending from me, and what's the most recent message?"

Claude identifies and summarizes the five threads that most need her attention. In fifteen minutes, she has a clear picture of what requires her response and what has already been handled by others. She goes from "drowning in inbox" to "clear on what to do next" in a fraction of the time it would have taken to read everything.

Workflow design notes

Thread summarization works best when you pair it with a clear action instruction afterward. Summary alone is passive. Summary plus "now draft a reply" is active. Summary plus "now tell me which threads are most time-sensitive" is strategic.

One practical note: thread summaries sometimes lose nuance. If the details in a thread are genuinely important (a negotiation, a legal matter, a sensitive personnel issue), read the thread yourself rather than relying entirely on the summary. Use summarization to triage, and read in full when the stakes warrant it.

For high-volume inboxes, a daily or weekly thread-summary routine can replace a significant portion of email reading time. A scheduled Cowork task (covered in Section 10) can automate this: Claude reads your inbox each morning and surfaces the threads that need attention. You review the summaries, decide what to act on, and proceed. This is one of the most commonly useful automations professionals build with Cowork.

Try this in Claude

Find the longest email thread in your inbox right now. Ask Claude to summarize it with: current status, any decisions made, and what (if anything) is waiting on you. Then ask: "Is there anything in that thread I should have noticed but might have missed?"

Pro tips

- After a thread summary, ask "who else has been most active in this thread and what's their position?" This gives you a quick stakeholder map before you reply.
- "Tell me the most recent development in this thread" is a useful shortcut when you just want the latest update, not the full history.
- Thread summaries are useful for preparing for calls. "Summarize my email history with this person before our call in an hour" gives Claude-produced context without the manual email archaeology.

Quick summary

Email thread summarization compresses long, messy threads into clear status updates, decision logs, and action items. Pair it with search to get current on any thread without reading every

message. Use it for inbox triage after time away, for pre-call preparation, and as the first step in a reply workflow. For recurring inbox management, it becomes even more valuable as a scheduled automation.

Drafting Replies From Inbox Context

Writing email replies is one of the most time-consuming, cognitively interruptive parts of professional work. You stop what you're doing, load the context of the thread, figure out what the right response is, draft it, review it, and send it. Multiply that by twenty messages a day and you've spent hours on a task that often requires more working memory than it does actual judgment.

Claude with an email connector collapses the context-loading and drafting steps. You describe what you want to say; Claude writes it.

Why this matters

The cognitive cost of email is usually higher than the time cost. Switching into "email mode" multiple times a day fragments attention and makes it harder to do deep work. If Claude can handle the drafting, you stay in decision mode, not writing mode. You review, approve, and send rather than compose.

This is different from a generic email template. Claude reads the actual thread, understands the actual context, and drafts a reply that's responsive to what was specifically said, not a generic version of what you might want to say.

The idea in plain English

With an email connector, Claude can read a thread and draft a reply based on your instructions. You tell Claude what you want to communicate and any specific requirements, and Claude produces a draft that:

- Addresses the specific points raised in the most recent message
- Maintains the appropriate tone for the relationship and context
- Reflects the history of the thread, not just the last message
- Includes whatever you've told Claude you need to say

You review the draft, edit as needed, and send. The composition work is done; you're in editing mode, which is faster and less mentally taxing.

How this works in Claude

Basic draft:

> *"Draft a reply to the most recent email in the Ashford Solutions thread. I want to confirm the meeting for Tuesday at 2pm and let them know I'll have the proposal ready beforehand."*

Tone-specific draft:

> *"Draft a reply to this vendor. They're asking for a longer payment timeline. I want to be firm that our standard terms are 30 days but leave the door open for an exception if they explain their situation. Professional but direct."*

Complex situation draft:

> *"I need to reply to this email from my manager who's asking why the project is delayed. The real reason is that we're waiting on legal review that was delayed by their office, not ours. Draft a reply that's factual and professional without being defensive, and that makes the dependency clear without assigning blame."*

Declining or deflecting:

> *"Draft a polite but clear decline to this speaking invitation. I don't want to commit time this quarter but I'd like to leave the door open for next year."*

Practical example

A sales director receives a frustrated email from a client who has experienced a service outage. The client is asking for a credit, an explanation, and a commitment that it won't happen again. This requires a careful reply, and writing it badly could escalate the situation.

She asks Claude: "Draft a reply to this client email. Read the thread for context. I want to: acknowledge their frustration genuinely, not just formally; explain what happened at a high level without making excuses; confirm they'll receive a credit for the affected period; and give them confidence this has been addressed without making a specific guarantee I can't keep."

Claude drafts a reply that hits all four points, maintains a warm and professional tone, and is specific enough about the credit and the resolution to sound credible. She adjusts two sentences, adds a personal note she wants to include, and sends it. The reply that might have taken thirty minutes of careful drafting took ten minutes with review.

Workflow design notes

Reply drafting requires trust calibration. Claude will produce a good draft, but the draft reflects Claude's interpretation of what you want to say and how you want to sound. For routine replies, this is usually accurate enough to edit quickly. For high-stakes replies (to executives, in conflict situations, with legal implications), budget more review time and read carefully before sending.

Tone consistency is a real consideration. Claude's default professional tone may not match your voice. If you have a strong personal communication style, you may want to add tone notes: "I write formally but never stiffly" or "I use short sentences and avoid corporate language." Storing these preferences in your Claude memory or in a Project instruction makes every reply draft more consistent.

An important operational note: review before sending. This is basic, but worth stating explicitly. A Claude-drafted reply that goes out with a wrong assumption, an incorrect fact, or an off tone is your problem. The reply-drafting workflow saves drafting time, not review time.

Try this in Claude

Find a pending email reply that you've been putting off because it's complicated or requires care. Share the thread context with Claude and describe what you want to say. Review the draft Claude produces. Note how much editing it needs compared to starting from blank.

Pro tips

- "What should I say in response to this email?" is a useful pre-draft prompt. Claude will often surface considerations you hadn't thought of before drafting.
- For recurring reply types (meeting confirmations, project updates, vendor negotiations), build reply templates with Claude and save them. You customize the variables, Claude doesn't have to generate from scratch.
- After sending a particularly well-drafted reply, note what instructions produced it. That's your prompt template for similar situations.

Quick summary

Drafting replies from inbox context combines email reading with writing assistance. Claude reads the thread, understands the situation, and drafts a reply based on your instructions. The value is in collapsing context-loading and composition into a single step, leaving you in editing and decision mode. Review carefully before sending, calibrate for high-stakes situations, and store tone preferences to make drafts more consistently aligned with your voice.

Finding Free Time and Protecting It

Knowing you have a packed week is not the same as knowing when you have open time and how much. Most professionals have a vague sense of their calendar load but a poor picture of where actual working time exists between commitments. Claude with a calendar connector makes the invisible visible: real open time, identified and described, ready to be allocated deliberately.

Why this matters

Calendar blindness is a real phenomenon. You know you're busy. You don't know whether you have a two-hour uninterrupted block on Thursday or whether Thursday is already fragmented into pieces too small for focused work. The difference matters: knowing where your deep work windows are lets you schedule cognitively demanding tasks into them rather than doing them reactively in whatever time is left.

Claude's calendar search saves the five-to-ten minutes of visual scanning you'd otherwise do. More importantly, it can surface patterns and opportunities you'd overlook: "You have three open hours on Tuesday if you move your 2pm recurring call, which has been rescheduled twice anyway."

The idea in plain English

With a calendar connector, Claude can read your calendar and answer questions about it in natural language.

Time-finding queries: - "Where do I have at least two uninterrupted hours this week?" - "What's my first available 30-minute slot tomorrow afternoon for a call?" - "Do I have any back-to-back meeting blocks longer than three hours that I could break up?"

Time-protection queries: - "I need to block three hours this week for deep work. What's the best slot based on what's already on my calendar?" - "Add a focus block to my calendar on Friday morning." - "When was the last week I had a full day without meetings?"

Load analysis queries: - "How many hours of meetings do I have scheduled this week?" - "What percentage of my working week is in meetings versus free time?" - "Which days have the most fragmented schedules?"

How this works in Claude

Calendar connector queries work in the same way as email queries. You ask in natural language and Claude returns the answer based on your actual calendar data.

If your connector has write access, Claude can also create calendar blocks on your behalf. You'd say: "Block two hours on Thursday morning for project planning and label it accordingly." Claude creates the event. You confirm or review.

If your connector is read-only, Claude tells you where the time is and you create the block yourself.

Example prompts:

> *"I need to find time for a 90-minute strategy session with my team this week or next. We need a time that works for all of us. I know their calendars aren't shared with me, but find the times that work for me and I'll check with them."*
>
> *"I've been in meetings for three weeks straight. Find the day this week with the fewest commitments and block it as a buffer day."*
>
> *"What's the latest I can schedule a one-hour call this Friday and still have an hour before my 5pm commitment?"*

Practical example

An operations director notices that her weeks feel progressively more reactive and less strategic. She asks Claude: "Look at my calendar for the last four weeks. What percentage of my time has been in scheduled meetings versus open time? And is there a pattern to when my open time typically falls?"

Claude reviews her calendar and reports: "Your average meeting load has been 62% of working hours over the past four weeks. Open time tends to cluster on Tuesday afternoons and Friday mornings. Wednesday and Thursday are consistently your most blocked days."

With that information, she makes a deliberate decision: Tuesdays and Fridays are protected for deep work. Mondays are for administrative tasks. She asks Claude to look for recurring meetings she could decline or shorten. Claude identifies two standing meetings she hasn't initiated and haven't produced action items in three weeks.

That conversation takes fifteen minutes and reshapes how she structures her work for the month.

Workflow design notes

Calendar management with Claude is genuinely useful, but the value is in acting on what you find. Finding open time is step one. Deciding what to put there, or what to protect it from, is the step that matters.

The most productive use of calendar-connected Claude is periodic calendar reviews: once a week, ask Claude to summarize your upcoming week, identify whether you have time for your actual priorities, and flag any scheduling conflicts or overloads before they become day-of problems.

Write access for calendar scheduling is powerful but warrants care. An event created by Claude in the wrong slot, or with the wrong attendees, is a problem to fix. If you're new to write-enabled calendar integrations, start with read-only and create events manually based on Claude's suggestions until you're confident in the workflow.

Try this in Claude

Connect your calendar and run three queries: where is your next open two-hour window, what is your meeting load percentage this week, and which day has the most fragmented schedule. Use that information to make one deliberate calendar decision, even if it's just blocking twenty minutes for something you've been putting off.

Pro tips

- Ask Claude to tell you your "real" availability, not just technical open slots: "Exclude early mornings and lunch blocks; what open time do I actually have this week?"
- Combine calendar and email: "Find a time for a 30-minute call with [person] and draft an email suggesting it." This is one of the cleanest connector-combination workflows.
- Use calendar analysis to audit recurring meetings: "List every recurring meeting on my calendar and for each one, note how often it's been rescheduled or cancelled." This is often illuminating.

Quick summary

Calendar-connected Claude finds free time, analyzes calendar load, and (with write access) creates blocks on your behalf. The value is in making invisible open time visible and allocating it deliberately rather than reactively. Combine with email to schedule calls. Use periodic calendar reviews to stay ahead of load problems before they compound.

Section 4 — Connectors: Email, Calendar & the Connector Ecosystem

Analyzing Calendar Patterns Against Your Actual Goals

Analyzing Calendar Patterns Against Your Actual Goals

Your calendar is a record of where your attention went. Most people look at their calendar forward (what's coming up) but rarely backward (where did my time actually go, and does that match what I said my priorities were?). Claude with a calendar connector makes that backward analysis fast and specific enough to act on.

Why this matters

There's usually a significant gap between how people say they want to spend their time and how they actually do. Not because of bad intentions, but because individual scheduling decisions accumulate into patterns that no one actively chose. Twenty minutes here, a recurring call there, a meeting that could have been an email but wasn't. By the end of the quarter, you've spent three times as much time in status updates as in the strategic work you said was your priority.

Seeing that gap clearly is the first step to narrowing it. Claude turns calendar analysis from a manual, tedious task into a quick, specific one.

The idea in plain English

Calendar pattern analysis compares how your time has actually been allocated to some reference point: your stated priorities, an ideal time allocation, a role description, or simply your own intuition about what your job requires.

The kinds of questions this answers:

Allocation analysis: "How much of my week is in each category of work? Strategy, operations, people management, administrative tasks, external meetings?"

Pattern detection: "What time of day and week am I most frequently in meetings versus free? Is my deep work time actually protected or just theoretically protected?"

Goal gap analysis: "I said I wanted to spend 30% of my time on strategic initiatives. How much am I actually spending?"

Meeting audit: "Which meetings am I attending that might not require my presence? Are any of my recurring meetings consistent candidates for delegation or removal?"

Trend analysis: "Is my meeting load increasing, decreasing, or stable over the past quarter?"

How this works in Claude

With a calendar connector, you ask Claude to analyze your calendar over a defined period and return a structured analysis.

Allocation query:

> *"Look at my calendar for the past month and categorize my meetings by type (internal sync, external client, one-on-ones, large group). Give me a rough percentage breakdown."*

Goal gap query:

> *"I'm supposed to spend about 40% of my time on business development. Based on my calendar for the last six weeks, how close am I to that target?"*

Meeting audit:

> *"Which recurring meetings have been rescheduled or cancelled most frequently? And which ones do I attend where I'm not the organizer and where I don't have a speaking role listed in the agenda?"*

Trend query:

> *"Compare my average weekly meeting hours for the last month versus the month before. Am I trending up or down?"*

Practical example

A newly promoted VP wants to ensure her time allocation reflects her new level of responsibility. In her previous role, about 80% of her time was in execution. In her new role, she's been told she should be 50/50 between execution and leadership. She asks Claude to analyze her calendar for the past six weeks.

Claude reports: she's at roughly 70% execution meetings and 30% leadership activity (one-on-ones, strategy reviews, stakeholder management). The allocation hasn't actually shifted as much as she'd assumed.

The analysis also surfaces that she's still attending three weekly operational syncs that she was running in her previous role but that could now be delegated. Those three meetings represent four hours a week.

She delegates the meetings, reclaims the time, and asks Claude to set a recurring Friday reminder to check her allocation against target. The review that would have required a manual calendar audit took ten minutes.

Workflow design notes

Calendar pattern analysis is a periodic practice, not a one-time query. Monthly or quarterly reviews are most useful: they give enough data for patterns to emerge and enough time to act before another full period passes.

The categories you use for analysis matter. Broad categories ("internal" and "external") produce broad insights. Specific categories that map to your actual work (business development, team management, operational review, strategic planning) produce more actionable ones. Take a few minutes to define your categories before running the analysis; Claude will use them consistently.

One limitation to be aware of: Claude analyzes what's on your calendar, which reflects scheduled commitments. Time spent on email, deep work, and other non-calendar activities doesn't appear in this analysis. For a complete picture, combine calendar analysis with a manual estimate of how unscheduled time is being used.

Try this in Claude

Define three to five categories that represent how you'd ideally divide your working time. Ask Claude to analyze your calendar for the past four weeks and categorize your meetings into those buckets. Compare the actual allocation to your intended allocation. Note the category with the biggest gap and identify one specific change you could make this week to narrow it.

Pro tips

- "What meetings could I stop attending without losing important information?" is a useful prompt after an allocation review. Claude will look for meetings where you're a passive participant.
- Combine calendar analysis with a goal review: paste in your written goals (from a planning document, a performance review, or your own notes) and ask Claude to assess whether your calendar reflects them.
- If you use time-blocking for categories of work (deep work, admin, calls), ask Claude to check whether the blocks are being respected or routinely overridden by scheduled meetings.

Quick summary

Calendar pattern analysis compares how your time is actually allocated against how you intended to allocate it. With a calendar connector, Claude makes this analysis fast enough to do regularly. Define meaningful categories, run the analysis monthly, and use the gap between actual and intended to drive specific calendar changes rather than vague intentions to "manage time better."

Section 4 — Connectors: Email, Calendar & the Connector Ecosystem

Preview: How These Connectors Scale Into Cowork

Preview: How These Connectors Scale Into Cowork

The email and calendar connectors you've set up in this section don't stop being useful when you move to Cowork. They become the engine for the more powerful workflows in Act 2. This lesson gives you a brief map of where you're going, so the patterns you're building now are recognizable when they show up later in a larger context.

Why this matters

One of the common confusions for people moving from Chat to Cowork is thinking of them as completely different tools. They're not. Cowork is Chat with additional capabilities: file system access, scheduled tasks, Dispatch (remote triggering from your phone), and computer use. The connectors are the same. The underlying Claude model is the same. The difference is scope and autonomy.

Understanding that continuity means the investment you make in configuring and using connectors now pays dividends in Act 2 without starting over.

The Chat connector workflow

In Chat, the connector workflow is interactive and manual:

- You ask Claude to search, summarize, or analyze
- Claude uses the connector to retrieve the relevant data
- Claude returns the result in the chat
- You act on it (review, reply, schedule, delegate)

This is useful and significantly faster than doing the same tasks manually. But it requires you to be present for each step.

How the same connectors work in Dispatch

Dispatch (covered fully in Section 11) is how you trigger Cowork tasks from your phone while your desktop runs them in the background. The email and calendar connectors you configured here are exactly what Dispatch reaches for when you send a task from mobile.

A Dispatch task like: "Review my inbox, summarize the three most urgent threads, draft replies for any that just need a confirmation, and leave the drafts ready for me to review when I'm back at my desk" uses:

- Your email connector (to read the inbox and threads)
- Claude's drafting capability (to produce the reply drafts)
- Your file system (to save the drafts if needed)

You sent that instruction from your phone in thirty seconds. Claude executed it on your desktop while you were in a meeting. You came back and the drafts were waiting.

How connectors power scheduled tasks

Scheduled tasks (Section 10) run on a timer without any input from you. A daily morning briefing task that summarizes your email, checks your calendar for the day, and surfaces anything time-sensitive runs every morning at 7am because you set it up once. The connector access it needs is the same email and calendar access you authorized in this section.

The connector ecosystem beyond email and calendar

Email and calendar are the starting point, but the connector ecosystem extends further. Depending on your work, useful connectors might include:

- Slack or Teams (for communication context and messaging)
- Google Drive or OneDrive (for document access and creation)
- CRM tools like Salesforce or HubSpot (for customer and deal data)
- Project management tools like Asana, Linear, or Jira
- Analytics tools for data retrieval
- Web search for current information

Each connector extends what Claude can see and act on. The principle is always the same: Claude reasons over whatever is in its context, and connectors expand what that context can include.

What this means for setup decisions now

The connectors you configure, and the permissions you grant, persist into Cowork. A few things worth thinking about now:

Scope of access: Grant the access you'll actually use. Broad access to every inbox, every calendar, every drive creates a larger surface area to manage. Start with the tools you use every day and expand deliberately.

Read vs. write permissions: Read access lets Claude see; write access lets Claude act. Write-enabled connectors (creating calendar events, sending emails, creating documents) are more powerful and require more care. Start with read, add write when you have a specific workflow that needs it and you trust your review process.

Testing the workflow before automating it: The interactive Chat workflow is a useful proving ground before you automate it. If the email summarization doesn't work reliably in manual Chat sessions, it won't work reliably in a scheduled automation either. Get the workflow right interactively first, then automate it.

Try this in Claude

Think about one task you do manually every day that involves email or calendar and that follows a consistent pattern. Draft the Cowork instruction you would eventually give for that task. You don't need to set it up yet; just write the instruction clearly enough that you could hand it to someone else and they'd know what to do. This is the prompt you'll use when you set up your first scheduled automation in Section 10.

Quick summary

The email and calendar connectors from this section are the same connectors that power Dispatch, scheduled tasks, and automated workflows in Act 2. The investment in setting them up and learning to use them interactively in Chat compounds when those connectors are accessed autonomously in Cowork. Start with read access, prove out the workflows manually before automating, and think ahead to which recurring tasks you'd eventually like to run without being present.

SECTION 05 · ACT 1 — CHAT

Section 5 of 14

Section 5: Projects as Your Workspace

Section 5: Projects as Your Workspace

A conversation with Claude is a session. A Project is a workspace. The distinction matters more than it sounds: sessions end, workspaces persist. This section is about structuring ongoing work in Claude so that the context, documents, and instructions you've carefully assembled don't have to be rebuilt from scratch every time you return.

Why this section matters

The single biggest efficiency loss for regular Claude users is re-establishing context. Every new conversation starts cold. You re-explain your company, your project, your preferences, and the background you've already covered. For one-off tasks, that's acceptable. For ongoing work, it's a significant and unnecessary tax.

Projects solve this. They're persistent containers where you store documents, instructions, and reference material that Claude can access across many conversations. You set it up once, and the context is there every time.

What you'll learn

What Claude Projects are. A Project is a named workspace where you can store files, instructions, and reference context that persists across sessions. Claude treats everything in a Project as available background for any conversation started within it.

Projects as context folders. The core concept: a Project is not a task or a deliverable. It's the context container for a category of work. Research projects, client engagements, content creation, personal planning, each can have its own Project with its own relevant materials.

Adding documents, PDFs, and reference material. Uploading files to a Project makes them permanently available without re-uploading each session. A client brief, a style guide, a set of company policies, reference materials that Claude needs for a broad category of tasks can live in the Project.

Writing Project instructions. Instructions are the standing directions Claude follows for every conversation in a Project. Your preferences, constraints, tone guidelines, and any context that would otherwise have to be repeated can be written once and applied automatically.

Projects vs. Artifacts: persistent inputs vs. things you produce. Projects hold inputs (documents, instructions, context). Artifacts are outputs (things Claude produces that you can save and use). Understanding this distinction prevents confusion about what goes where.

Using Projects for research, content, and collaboration. The practical applications: a research project with source documents, a content project with a style guide and brief templates, a client project with background documents and relationship notes.

The mini project

You'll build a functional Project for a real category of ongoing work: a client engagement, a research topic, a content type you produce regularly. The deliverable is a configured Project with uploaded reference material, written instructions, and at least one conversation demonstrating that the context carries forward correctly.

How to approach this section

Projects are best understood by using them rather than reading about them. Set one up for something real. The concepts become clear once you see how much simpler a Claude interaction becomes when the context is already there.

By the end of this section, the idea of starting every Claude conversation from zero should feel unnecessarily inefficient.

Section 5 — Projects as Your Workspace

What Claude Projects Are

What Claude Projects Are

A Project is a persistent workspace in Claude. It holds documents, instructions, and reference material that remain available across multiple conversations. If you've been rebuilding context from scratch every time you open a new Claude chat, Projects are the solution.

Why this matters

Every Claude conversation has a context window: the space available for everything Claude can "see" while generating a response. In a standard chat, that window starts empty. You fill it with your prompt, any files you upload, and the conversation history so far.

In a Project, part of that context window is pre-filled with materials you've loaded into the Project. Every conversation you start within the Project begins with Claude already knowing what you've stored there. Your documents, your instructions, your preferences: all available without repeating yourself.

For anyone doing ongoing work with Claude, rather than purely one-off tasks, this changes the character of every interaction.

The idea in plain English

Think of a Project as a shared folder and briefing document combined. You put relevant materials in the folder (documents, PDFs, reference text). You write a briefing document (the Project instructions) that tells Claude what it needs to know about this work and how you want it to operate. Then every conversation you have within that Project benefits from both.

A Project has three main components:

Files: Documents and reference materials you upload to the Project. These persist across sessions and are available to Claude in every conversation within the Project without re-uploading.

Instructions: A standing brief that Claude reads at the start of every Project conversation. It can include context about the work, preferences, constraints, personas, and any other standing directions.

Conversations: The chats you have within the Project. Each conversation has access to the Project files and instructions. The conversations themselves are separate threads; they don't share context with each other, but they all share the Project context.

How this works in Claude

In the Claude interface, Projects appear in the left sidebar, usually above or alongside your conversation history. You create a new Project by clicking the "New Project" option, giving it a name, and then configuring it: uploading files and writing instructions.

Once a Project exists, you start new conversations within it by opening the Project and creating a new chat from there. That conversation has access to everything in the Project.

You can have multiple Projects running simultaneously. A client account, a research area, a content type, and a personal planning space can all exist as separate Projects with their own documents and instructions.

Practical example

A market research analyst works on competitive intelligence for three different product lines. Each product line has: a set of market reports, competitor analysis documents, a style guide for

deliverables, and specific instructions about what clients care about.

Before Projects, she rebuilt that context in every conversation: pasting in the relevant reports, explaining the context, restating her preferences. This took 10-15 minutes before she could start any substantive work.

After setting up three Projects (one per product line), each stocked with the relevant documents and a clear instruction set, she opens the relevant Project and starts working. The context is there. She moves from opening Claude to producing useful output in two minutes instead of fifteen.

The productivity gain isn't just time: it's the cognitive cost of rebuilding context, and the errors that come from occasionally forgetting to include something important.

What Projects are not

Projects are not for storing conversations you want to find later. Conversation history lives in your general history, not in Projects.

Projects are not task management or project management tools. They don't have deadlines, assignees, or statuses. They're context containers, not work trackers.

Projects are not a way to make Claude remember things permanently. They store what you explicitly put in them. If you want Claude to carry forward something it learned in a conversation, you either save it to the Project files manually or rely on Claude's memory feature (covered in Section 1).

Try this in Claude

Create one Project right now. Name it for something you work on regularly: a client, a topic, a type of deliverable. Don't fill it yet; just create it and look at the interface. Where would you upload files? Where do you write instructions? You'll build it out in the next few topics.

Pro tips

- Name Projects clearly and specifically. "Client Work" is a folder. "Meridian Account - Q3 2026" is a Project.
- Start with the instructions before uploading files. Writing the instructions first helps you think clearly about what materials the Project actually needs.
- If a Project starts to feel cluttered with outdated files, archive the old ones rather than leaving them. Irrelevant context can dilute Claude's attention on what's current.

Quick summary

A Claude Project is a persistent workspace that holds documents, instructions, and reference material available across multiple conversations. It solves the problem of rebuilding context every session. Each Project has files, instructions, and the conversations you have within it. Set up Projects for ongoing work where the same context applies repeatedly.

Section 5 — Projects as Your Workspace

Projects as Context Folders: Organizing Your Work

Projects as Context Folders: Organizing Your Work

The word "project" carries management connotations: timelines, deliverables, milestones. Claude Projects are simpler than that. They're context folders. The question to ask when deciding whether something needs a Project is not "is this a project?" but "is this a category of work where the same context applies across many conversations?"

Why this matters

The way you organize your Projects directly affects how useful they are. Too few Projects, and you're mixing unrelated contexts (client A's documents appearing in conversations about client B). Too many Projects, and you're maintaining more structure than it's worth. The right framing, Projects as context folders rather than projects in the management sense, makes the organizational decisions much simpler.

The idea in plain English

A context folder holds everything Claude needs to know to do a specific category of work well. It answers the question: "What would a new colleague need to read before jumping into this work?"

Useful categories for context folders:

By client or account. Each client has different background, different preferences, different deliverables, and often different constraints. A context folder per client means Claude is always oriented to the right account without you having to specify.

By content type. If you produce the same type of content repeatedly (weekly reports, monthly newsletters, customer case studies, product briefs), a context folder for each type can hold style guides, example outputs, and standing instructions.

By topic or domain. Research in a specific area benefits from a folder with source documents, background reading, and any constraints on how the research should be used.

By role or function. Some professionals maintain a folder for each major function of their job: one for communications, one for strategy, one for operations. Each has different reference materials and different standing instructions.

By project (actual projects). Time-bounded initiatives with specific deliverables: a product launch, a market entry, a proposal.

How to decide what goes in a folder

The test: if you started a new conversation without this context, would you have to re-explain something? If yes, that something belongs in the Project.

Common candidates: - Background documents (company overview, product description, org structure) - Reference materials (research, reports, data sources) - Style guides and tone guidelines - Example outputs ("Claude, write a case study like this one") - Standing constraints ("never mention competitor names," "always include a CTA") - Relationship notes ("this client responds well to direct language, not diplomatic softening")

Not every category of work needs a Project. For truly one-off tasks, the overhead of Project setup is not worth it. Use Projects for work you return to repeatedly.

How this works in Claude

When you create a Project in Claude, you're essentially making a folder. You give it a name, upload the documents that belong to it, and write instructions for how Claude should behave within it.

The files in the folder are available as context in every conversation you start within that Project. You don't have to specify "use the documents in this folder"; Claude reads them automatically. You can reference them explicitly ("based on the style guide in this Project") or let Claude apply them implicitly.

Practical example

A content strategist manages a B2B technology client. She creates a Project called "Hartwell Technologies — Content." In it she puts: - The client's brand voice guide - Their most recent annual report (for context on company direction) - Six examples of well-received content pieces from the past year - A one-page brief on their target audience and key messages - Instructions: "All content should be aimed at IT leaders at mid-market companies. Avoid jargon. Use active voice. Keep sentences under 20 words where possible."

Now every content conversation for this client starts with Claude already knowing the brand, the audience, the examples, and the constraints. She doesn't re-explain any of it. She just describes the piece she needs: "Write a LinkedIn article about cloud migration." Claude produces something that sounds like the client, not like generic tech content.

Workflow design notes

Context folders require occasional maintenance. Documents go stale: a brand guide from two years ago may not reflect current positioning. An example set that worked well in the past may not be the right benchmark now. Build a habit of reviewing Project contents when you notice Claude's output drifting from what you expect, or on a simple time schedule (quarterly for most Projects is sufficient).

The folder structure is also not permanent. If a Project grows unwieldy, split it. If a Project is rarely used, archive it or delete it. Context folders are tools, not monuments.

Try this in Claude

Look at the work you've done with Claude in the past two weeks. Identify the category that required the most context-rebuilding. That's your first Project. Define what folder it is, list the three to five documents that should be in it, and draft one sentence describing what Claude should know going in. You'll build this out in the upcoming topics.

Pro tips

- A Project for "my work style and preferences" is useful even if you don't have recurring client or content work. It's the one folder that applies everywhere: your tone preferences, formatting rules, and general working style.
- If you work with a team, shared Projects (if your plan supports them) let multiple people work from the same context without each person rebuilding it independently.
- For a new client engagement, start by creating the Project before the first meeting. Set it up as empty and add materials as they come in. The habit of adding to the Project means you don't have to reconstruct it later.

Quick summary

Projects are context folders, not project management systems. The question is not "is this a project?" but "is this a category of work where the same context applies repeatedly?" Organize by client, content type, topic, or function. Maintain the folders by removing outdated materials. The investment in setup pays off every time you open a conversation and the context is already there.

Adding Documents and Reference Material to a Project

The documents you add to a Project are what make it genuinely useful rather than just an organizational container. Getting this right, knowing what to include, how to structure it, and how to keep it current, is what separates a Project that saves you time from one that requires maintenance without delivering much.

Why this matters

A Project with no files is just a named chat folder with a standing instruction. A Project with the right files is a context-rich workspace where Claude can produce calibrated, relevant output without you having to supply the background every time. The files are the "read this first" stack that a new colleague would need before contributing meaningfully. Curate them accordingly.

The idea in plain English

Any document you would need to consult or reference while doing this category of work is a candidate for the Project.

High-value additions:

Background documents: Anything that explains the subject matter, the client, the product, or the context. Company overviews, product briefs, market summaries.

Reference materials: Source documents, research reports, data sets, regulatory frameworks, technical specifications. The raw material Claude might need to draw on.

Style and tone guides: If Claude is producing written content, examples and guidelines shape the output more effectively than descriptions. Include samples of what "good" looks like.

Constraints documents: What Claude should never do in this context. Topics to avoid, formats not to use, approvals required before something goes out.

Templates: If you produce consistent types of deliverables, including templates tells Claude what the finished product should look like. Claude fills them out rather than inventing a structure from scratch.

Lower-value additions:

Very long documents that are only partially relevant: A 200-page annual report where you only care about the strategy section takes up context space. Extract and upload the relevant section instead.

Outdated materials: An old version of a document that contradicts the current version creates confusion. Keep only the current version.

Everything as a hedge: It's tempting to upload everything related to a topic just in case. Resist this. Relevant, curated context produces better output than a sprawling archive Claude has to sort through.

How this works in Claude

When you're in a Project, look for the "Add to Project" or file upload option in the Project interface. You can upload files directly (PDF, DOCX, TXT, CSV, and other supported formats), or add text content manually.

Files you add to a Project: - Are available in all conversations within that Project - Do not need to be re-uploaded each session - Can be removed or replaced when they become outdated - Contribute to the context window that Claude works from

One practical note: the Project context window has limits. If you add many large documents, Claude may not be able to give full attention to all of them simultaneously. For very large document sets, consider splitting into multiple Projects or working with targeted extracts rather than complete documents.

Practical example

A consultant runs a Project for a six-month client engagement. Over the course of the project, she adds to the folder progressively:

Week 1: The client's signed statement of work, the onboarding deck, and a one-page brief she wrote summarizing the engagement.

Week 3: The first research report she produced (as an example of the expected output style), and a constraints document specifying what the client has said is out of scope.

Week 6: Meeting notes from the key strategic decisions made so far, and an updated brief reflecting the scope change agreed in week four.

At each point, she removes or annotates documents that are superseded. The Project folder always represents the current state of the engagement, not its entire history.

Because the folder is maintained this way, any conversation she starts within the Project is grounded in current context. Claude doesn't reference the original scope that changed in week four; it knows the revised scope.

Workflow design notes

The most useful discipline for Project document management is the "update on change" habit: whenever something significant changes in the work (a scope revision, a new style requirement, an updated reference), update the Project before the next session. This is a thirty-second task that prevents hours of confusion later.

Naming documents clearly within a Project also matters. "Brand Guide v3 Final.pdf" tells Claude (and you) more than "Document1.pdf." Claude can reference documents by name in its responses, which is more useful when the names are descriptive.

Try this in Claude

For the Project you started thinking about in the previous topic, identify the three to five documents that should be in it. Check whether you have current versions. Upload them. As you do, ask yourself: is there anything in this document I'd want Claude to ignore? If so, note that in your Project instructions.

Pro tips

- For confidential documents, remember the privacy principles from Section 1. Uploading a document to a Project doesn't change where it goes; treat it with the same care as any other upload.
- Consider adding a "Project README" as a text document at the top of your folder: a brief summary of what the Project is for, what documents are included, and any important context that doesn't live in the instruction set.
- When you update a document (new version of a style guide, updated research), delete the old version rather than leaving both. Duplicate versions of the same document create ambiguity.

Quick summary

The documents you add to a Project are what give it power. Include background materials, reference content, style guides, constraints, and templates. Keep the folder curated: current, relevant, and free of outdated or contradictory materials. Maintain it with an "update on change" habit rather than doing a quarterly cleanup. The folder should always reflect the current state of the work, not its full history.

Writing Project Instructions That Actually Work

Project instructions are the standing brief Claude reads at the start of every conversation in a Project. Done well, they eliminate a category of repetitive prompting and make every interaction more immediately useful. Done poorly, they're either ignored (too vague) or a source of friction (too rigid). This lesson covers how to write instructions that reliably improve outputs without getting in the way.

Why this matters

Instructions are the most powerful per-conversation time-saver in a Project. Every constraint, preference, context note, or standing direction that lives in the instructions doesn't need to be in your prompts. A style guide instruction applied once to every conversation is more reliable than a style reminder that you sometimes remember to include.

They also encode your judgment. The best Project instructions capture decisions you've already made: the tone this client expects, the format this report always takes, the topics that are always out of scope. That decision doesn't need to be made again for each conversation.

The idea in plain English

Project instructions can include several types of content:

Context: Background information that applies to all work in this Project. Who the client is, what the product does, what the goals of the engagement are.

Persona or role: How Claude should approach this work. "Act as an experienced marketing consultant" or "take the perspective of a junior analyst writing for a senior audience" changes the frame for every response.

Tone and style: How Claude should sound. "Direct and concise, no corporate filler" or "formal but accessible, appropriate for a regulatory audience."

Format preferences: What outputs should look like by default. "Always use bullet points for lists, never numbered lists unless sequence matters." "Keep responses under 300 words unless asked for more."

Standing constraints: What Claude should never do in this context. "Do not name competitors." "Do not include pricing assumptions." "Always flag when something requires legal review."

Reference notes: Key facts Claude should always have in mind. "The client's primary market is the UK and Ireland, not North America." "The product is in beta; do not describe it as production-ready."

How this works in Claude

When you open a Project's settings, you'll find the instructions field. Write your instructions there as plain text. They don't need to be formatted in any special way; Claude reads them as a standing brief.

A useful structure for Project instructions:

```
About this project:
[One paragraph of context: what this work is, who it's for, what the goals are]

How to approach this work:
[Role, tone, and style guidance]

Format defaults:
[Any output format preferences that apply across all conversations]

Standing constraints:
[Things Claude should always or never do in this context]

Key facts to keep in mind:
[Reference notes that should inform every response]
```

This is a template, not a requirement. Instructions can be as short as two sentences or as detailed as a full page, depending on the complexity of the work.

Practical example

A PR agency creates a Project for a financial services client. Their instructions read:

About this project: This Project supports content and communications work for Clearwater Capital, a London-based wealth management firm serving high-net-worth individuals. Their key differentiator is personalized service and long-term relationship focus, not product breadth.

Tone and approach: Write with authority but warmth. Avoid financial jargon unless the audience is clearly specialist. Never use hyperbole about returns or performance. The compliance team reviews all external content, so flag anything that makes specific financial claims or predictions.

Format defaults: For client-facing content: short paragraphs, no bullet points, formal register. For internal briefings: bullet points are fine, concise is better than comprehensive.

Standing constraints: Do not mention competitors by name. Do not make specific return or performance claims. Do not use the word "guaranteed" in any context. Always end client communications with the regulatory disclaimer prompt: [reminder to add disclaimer].

Key facts: The firm manages approximately £2.4bn in assets. Primary client base is 40-65, predominantly professionals and business owners. UK and Ireland only; no international operations.

Every conversation in this Project starts with Claude knowing all of that. No prompt in this Project needs to include any of it.

Workflow design notes

Instructions should be treated as living documents. Review and update them when: - The work changes significantly (new scope, new constraints) - You notice Claude consistently doing something you don't want (add a constraint) - A standing preference you've been including in every prompt belongs in instructions instead

Instructions can also be tested in a lightweight way: start a conversation in the Project with a simple request and check whether Claude applies the instructions correctly. If it doesn't, the instructions may be ambiguous or conflicting.

One important note: instructions have a length limit. Very long instruction sets may be partially applied or may leave less room for document context and conversation content. Keep instructions focused. If you find yourself writing a long instruction document, ask whether some of it belongs in a reference file instead.

Try this in Claude

Write the first version of instructions for the Project you've been building. Start with three elements: one sentence of context, one sentence of tone guidance, and one standing constraint. That's a minimum viable instruction set. Run a test conversation and check whether Claude applies all three. Expand from there.

Pro tips

- Instructions written as "always do X" and "never do Y" are clearer and more reliably applied than instructions written as "try to X" or "prefer Y."
- If you're unsure what to put in instructions, run a few conversations without them first. Note everything you include in prompts that you repeat across multiple sessions. Those are your

instruction candidates.

• For shared Projects (where multiple team members have access), instructions also serve as a shared briefing for how Claude should behave in this context. Write them to be useful for anyone on the team, not just yourself.

Quick summary

Project instructions are standing directions Claude applies to every conversation in the Project. They can include context, persona, tone, format preferences, constraints, and key facts. Write them clearly using "always/never" language, keep them focused, and treat them as living documents to update as the work evolves. A good instruction set eliminates the need to repeat context in every prompt.

Section 5 — Projects as Your Workspace

Projects vs. Artifacts: Inputs vs. Outputs

Projects vs. Artifacts: Inputs vs. Outputs

Projects and Artifacts are both things Claude produces and manages, but they serve fundamentally different functions. Mixing them up creates confusion about where work lives and how to access it. Understanding the distinction cleanly makes your Claude workspace significantly easier to navigate.

Why this matters

Once you're using Claude regularly for real work, you end up with a lot of things: source documents, reference materials, drafts, polished outputs, ongoing conversations, one-off responses. Without a clear mental model for what goes where, everything ends up in a pile. You lose track of where to find things, and you don't take advantage of how Claude actually stores and manages different types of content.

The Projects vs. Artifacts distinction is the key structural concept that organizes your Claude workspace.

The idea in plain English

Projects are inputs. A Project holds everything Claude needs to do a category of work: the documents, the instructions, the reference context. These are things you put into Claude to inform

what it produces. They persist because you want them to be available across many conversations.

Artifacts are outputs. An Artifact is something Claude produces: a document, a report, a mini-app, a visualization, a piece of code. Artifacts are things Claude creates that you can view, save, edit, and use. They persist so you can return to them, share them, or continue working on them.

The flow goes: Project (context in) → Conversation (Claude works) → Artifact (output out).

A concrete analogy: a Project is the brief, the source documents, and the style guide you hand a writer. An Artifact is the article they produce.

How Artifacts work in Claude Chat

When Claude produces a substantial piece of output (a document, a piece of code, a data visualization), it often appears as an Artifact in a separate panel alongside the conversation. You can: - View it rendered (not just as raw text) - Copy it - Edit it directly - Expand on it through follow-up conversation - Share it (where sharing is supported)

Artifacts are interactive outputs. They're designed to be used and refined, not just read.

In Chat, Artifacts are the outputs you produce within a conversation. They persist as long as the conversation does, and can be revisited by returning to that conversation.

In Cowork (covered in Act 2), Artifacts can be live and connected to data sources, refreshing automatically. That's a more advanced capability covered in Section 13.

The practical distinction

When you're deciding where something goes, ask one question: is this something I bring to Claude, or something Claude produces for me?

Type	Where it lives	Examples
Background document	Project (input)	Client overview, style guide, policy document
Reference research	Project (input)	Industry report, competitor analysis
Standing instructions	Project (input)	Tone guide, constraints, persona
Draft document	Artifact (output)	Proposal draft, report, email
Data visualization	Artifact (output)	Chart, dashboard
Mini tool	Artifact (output)	Interactive calculator, tracker

Type	Where it lives	Examples
Finished content	Artifact (output)	Blog post, case study, presentation outline

Practical example

A consultant uses a single Project for a client engagement. The Project holds: the client brief, the contract scope, an example of a past deliverable they liked, and her standing instructions for tone and format.

In each conversation within that Project, she produces Artifacts: a discovery document from the first session, a risk register from the second, a strategy memo from the third. Each Artifact is the output of that conversation, shaped by the Project context she built.

The Project doesn't change much over the course of the engagement (she updates it when scope changes). The Artifacts accumulate as the work progresses.

When she needs to reference the strategy memo in a later conversation, she either opens the conversation where it was produced, or pastes the relevant section into the current conversation. The distinction between where she put inputs (Project) and where she finds outputs (conversation Artifacts) makes this navigation natural.

Workflow design notes

One practical consideration: finished outputs sometimes become future inputs. A strategy memo produced as an Artifact in one conversation might be added to the Project as a reference document for future conversations. That's a deliberate transition: something that was an output (Artifact) becomes part of the standing context (Project file).

This is intentional, not a contradiction of the model. As work evolves, things shift from outputs to inputs. A first draft becomes a reference example. An initial report becomes background for the next one. The Project-Artifact distinction describes the current role of each piece, not a permanent categorization.

Try this in Claude

Open a Project conversation and ask Claude to produce a document of some kind. When Claude produces it as an Artifact, note where it appears and how you can interact with it. Then ask yourself: is there any case where this Artifact would become a reference document you'd add to the Project? Identify one output type where you'd regularly promote outputs to Project inputs.

Pro tips

- Don't clutter your Project with every output. Only add Artifacts back to the Project when they'll genuinely serve as reference for future conversations.
- In long engagements, create a lightweight "deliverables log" in the Project: a text document listing the Artifacts you've produced, where to find them, and their status (draft, approved, delivered). This is faster than hunting through conversation history.

Quick summary

Projects hold inputs: documents, instructions, and reference context that inform Claude's work. Artifacts are outputs: documents, visualizations, and tools Claude produces. The flow is inputs in, work happens, outputs out. Some outputs later become inputs, but the distinction helps you navigate where to find things and understand what's persistent context versus produced deliverable.

Section 5 — Projects as Your Workspace

Using Projects for Research, Content, and Collaboration

Using Projects for Research, Content, and Collaboration

The same Project concept applies across very different types of work. Research projects look different from content production projects, which look different from collaborative team projects. This lesson looks at how Projects adapt to three common professional use cases so you can recognize which pattern fits your situation.

Why this matters

Projects are general-purpose tools, but the way you configure them varies significantly by use case. Knowing the patterns that work for research, content, and collaboration prevents you from starting from scratch each time. It also helps you identify when a Project is the right tool versus when a simpler arrangement (a single conversation, a document, a shared file) would serve better.

Research Projects

A research Project is a workspace for sustained inquiry into a topic: competitive intelligence, market research, policy analysis, academic literature review, due diligence.

What the Project holds: - Source documents (papers, reports, articles) - Key excerpts and summaries compiled so far - A research brief or question set defining the scope - Instructions about analytical approach and output format

How conversations typically work: Each conversation adds to the research or explores a specific angle. You upload new sources as you find them. You produce Artifacts: summaries, synthesis documents, comparison tables, argument maps.

What to watch for: Research Projects can accumulate a lot of documents quickly. Periodically prune sources that are superseded or less relevant, and create a synthesis document that captures the current state of your knowledge. Otherwise, the Project becomes an archive rather than an active workspace.

Content Production Projects

A content Project is a workspace for producing a consistent type of output repeatedly: blog posts, newsletters, case studies, marketing copy, thought leadership pieces.

What the Project holds: - Style guide and tone guidelines - Example outputs (the best examples of what you've produced before) - Brand voice notes, vocabulary lists, forbidden phrases - Brief templates for the type of content - Instructions covering audience, format, and quality criteria

How conversations typically work: Each conversation produces one piece of content. You bring the brief (a prompt or topic), Claude applies the style and format from the Project context, and the output is consistently aligned with your voice and standards.

What to watch for: Content Projects degrade slowly as your style evolves. The examples you added six months ago may no longer represent your current standard. Refresh the example set quarterly, or whenever you notice Claude's outputs feeling slightly off. Add new examples of your best work as you produce it.

Collaborative Projects

A collaborative Project is a shared workspace for a team working on the same category of work. Multiple people use the same context, instructions, and reference materials.

What the Project holds: - Shared briefing documents that the whole team needs - Team conventions and shared style guidelines - Reference materials everyone accesses - Instructions that represent team decisions, not individual preferences

How conversations typically work: Each team member starts conversations within the shared Project. They all start with the same context, which produces more consistent outputs across the team without requiring repeated briefings.

What to watch for: Shared Projects require someone to own the maintenance. Agreed-on conventions need to be captured in the instructions. Outdated documents need to be removed before they create confusion. Assign ownership of the Project before launching it, not after problems emerge.

A word on Projects vs. one-off conversations

Not every piece of work needs a Project. Projects are worth setting up when: - You'll return to the same context multiple times - The context-building overhead for each session is significant - Consistency across outputs matters

Projects are not worth setting up when: - The task is genuinely one-time - The context is simple enough to include in a prompt - The work is over in one or two sessions

The right answer for any given work type is usually apparent after you've rebuilt the context twice. The second time you paste in the same documents and explain the same background, that's the signal to create a Project.

Practical example

A communications team at a technology company runs three Projects:

"Exec Comms": Holds the CEO and CPO voice guides, past speeches, company messaging framework, and investor relations guidelines. Used for board presentations, earnings calls, and internal all-hands content.

"Product Marketing": Holds the product positioning document, feature glossary, competitive analysis, and example product briefs. Used for launch communications, sales enablement, and product documentation.

"Customer Stories": Holds the case study template, approved quotes, four example case studies, and instructions on how to handle attribution and review. Used for all customer story content.

Three separate context folders. Three distinct types of work with different audiences, styles, and constraints. Each Project produces consistent output without having to re-explain the brand, the audience, or the format.

Try this in Claude

Map your own work onto the three patterns: research, content production, and collaboration. Which fits most of what you do with Claude? Design a Project for one real category of your work using that pattern as your guide. What goes in the folder? What do the instructions say? What does a typical conversation in that Project look like?

Pro tips

- For research Projects, create a "key findings" document in the Project that you update after each significant session. It becomes the running synthesis of what you've learned.
- For content Projects, add a "what not to do" example alongside your positive examples. Showing Claude both ends of the quality spectrum gives it a clearer target.
- For collaborative Projects, run an onboarding conversation: "Explain back to me what you know about this Project based on the instructions and documents." This surfaces misinterpretations before they affect real work.

Quick summary

Projects adapt to different types of work. Research Projects hold source documents and analytical frameworks; they require periodic pruning as research grows. Content Projects hold style guides and examples; they require refresh as standards evolve. Collaborative Projects hold shared context for teams; they require designated ownership and maintenance. Identify which pattern fits your work, configure accordingly, and start with the simplest version that removes the context-rebuilding problem.

SECTION 06 · ACT 1 — CHAT

Section 6 of 14

Section 6: Artifacts, Interactive Visuals & Claude Design

Section 6: Artifacts, Interactive Visuals & Claude Design

This section is where Claude stops producing text you read and starts producing things you use. Artifacts are outputs you can interact with: mini apps you can click, charts that respond to your data, mockups you can share, decks you can present. Claude Design extends this further into branded visual outputs.

Why this section matters

Text responses are useful. Interactive outputs are usable. The difference becomes concrete the first time you paste a data set into Claude and receive a working chart instead of a paragraph describing what a chart might show. Or when you need a quick landing page mockup and get a rendered, shareable HTML page rather than a written description of one.

This section covers the full spectrum of what Claude can produce as an Artifact, how to get there, and how to review and refine the outputs so they're production-quality rather than impressive-but-rough.

What you'll learn

What Artifacts are and how they differ from chat responses. A chat response is text in the conversation. An Artifact appears in a separate panel, is rendered (not just displayed as raw code or text), and can be interacted with, copied, or extended. Understanding when something should be an Artifact versus a chat response shapes how you prompt.

Creating mini apps and editable tools. Claude can produce functional web-based tools: calculators, trackers, decision frameworks, interactive comparisons. These run in the browser without installation or code knowledge.

Visualizing frameworks and processes. Diagrams, flowcharts, process maps, and visual frameworks rendered directly in Claude. Useful for explaining complex systems, mapping workflows, and communicating structure.

Inline interactive charts. Paste a set of numbers into Claude and ask for a chart. Claude produces an interactive, rendered chart in the chat. No spreadsheet, no external tool required.

Pitch decks, one-pagers, and landing-page mockups with Claude Design. Polished visual outputs for business communications and design contexts. Claude Design helps produce branded, visually structured outputs beyond plain text.

Reviewing and refining AI-generated designs. How to evaluate what Claude produces visually, what to ask for when it's not quite right, and how to iterate toward something genuinely usable.

Teaser: live artifacts as the Cowork upgrade. Static Artifacts are useful. Live Artifacts, covered in Section 13, connect to real data and refresh automatically. This section covers the foundation; Cowork extends it.

The mini project

You'll produce one polished, shareable output: an editable tracker, a clickable visual framework, a landing-page mockup, or a pitch-deck outline. The deliverable demonstrates that Claude's Artifact capability produces something genuinely usable, not just something impressive in a demo.

How to approach this section

Artifacts are best learned by producing them. The concepts are intuitive once you see what they look like. As you work through the topics, run the exercises on real material: actual data you want to chart, an actual process you want to map, a real product or idea you want to mock up.

The section ends with a design review lesson for a reason: impressive at first glance and genuinely production-quality are different things. Building the habit of evaluating and refining Artifacts is part of using them responsibly.

Section 6 — Artifacts, Interactive Visuals & Claude Design

What Artifacts Are (And When to Ask for One)

What Artifacts Are (And When to Ask for One)

A chat response is text. An Artifact is something Claude makes that you can use. The distinction sounds simple, but understanding it changes how you think about what to ask for and how to ask for it.

Why this matters

Most people use Claude in text mode by default: they send a message, they receive a message back. This is fine for explanations, drafts, and analysis. It's the wrong mode for anything that needs to be interactive, rendered, or handed to someone else in a usable form.

Artifacts are the bridge between "Claude explained how to do this" and "Claude built this for me." When you know to ask for an Artifact, you get something you can share, click, and act on, not just read.

The idea in plain English

An Artifact is a distinct output that Claude creates and presents in a separate panel alongside the conversation. It's rendered, meaning it displays as the finished thing rather than as the code or markup behind it.

Types of Artifacts Claude can produce:

Documents: Formatted text documents, reports, memos, briefs. When you ask for a long piece of writing, Claude often produces it as a renderable Artifact you can view cleanly and copy.

Code: Functional programs or scripts displayed in a code panel with syntax highlighting.

HTML pages: Rendered web pages. These can be simple formatted documents or interactive web apps with buttons, forms, and dynamic behavior.

Data visualizations: Charts, graphs, and visual data displays rendered in an interactive format.

SVG graphics: Scalable vector illustrations and diagrams.

Markdown documents: Structured text that renders with proper formatting, headers, and lists.

React components: Interactive interface components that run in the browser.

The key characteristic of an Artifact is that it renders and persists. You can return to it, share it (by copying the underlying code or content), and continue working on it through follow-up.

How this differs from a chat response

A chat response: - Lives in the conversation thread - Is text (or inline code) - Scrolls with the conversation - Is useful for reading, but not interactive

An Artifact: - Appears in a side panel - Is rendered as the finished output - Persists in that panel as you continue the conversation - Can be interactive (for HTML and React Artifacts) - Can be directly copied, downloaded, or extended

The practical difference: if you ask Claude to "write a report," it might produce a chat response. If you ask Claude to "create a formatted report document," it will often produce it as an Artifact you can view cleanly.

How this works in Claude

You don't need to use special syntax to trigger Artifacts in most cases. When Claude recognizes that a request calls for an Artifact (code, a rendered document, a visualization, an interactive tool), it produces one automatically.

You can also be explicit: "Create this as an Artifact" or "Make this an interactive HTML page" will direct Claude toward Artifact output.

When an Artifact appears in the panel, you can: - View it in the Preview tab (rendered) - View it in the Code tab (the underlying source) - Copy the content - Ask Claude to modify it by continuing the conversation

Practical example

A strategy consultant needs to explain a complex decision framework to her team. She could describe it in prose. She could paste bullet points. Or she could ask Claude for something more engaging.

She asks: "Create an interactive HTML page that shows our three-stage decision framework as a visual diagram. Each stage should be clickable and show a brief description when clicked."

Claude produces an Artifact: a rendered HTML page with three clickable stages, each revealing a description. It takes two minutes. She copies the HTML, shares it with her team via email, and they view it in any browser without installing anything.

The same framework as a chat response would have been a text description. As an Artifact, it's a usable team resource.

Workflow design notes

Not everything needs to be an Artifact. Use Artifacts when: - The output needs to be rendered, not just read - You need to share the output with someone outside the conversation - The output should be interactive (buttons, inputs, dynamic behavior) - The output is substantial enough to benefit from a clean separate view

Use chat responses when: - You need a quick answer, explanation, or analysis - The output will be incorporated into something larger - You're in the middle of a back-and-forth refinement

Artifacts are most powerful when you treat them as starting points, not final outputs. Ask for a first version, review it in the Preview pane, and iterate with follow-up requests until it's what you need.

Try this in Claude

Ask Claude to create a simple interactive tool: "Make an HTML page with a simple calculator for [some calculation relevant to your work]." Observe how Claude produces it as an Artifact, how it renders in the Preview tab, and how you can ask for modifications. This is the fastest way to understand the Artifact workflow.

Pro tips

- If Claude produces something as a chat response when you wanted an Artifact, add "create this as a rendered HTML Artifact" or "format this as a markdown Artifact" to your follow-up.
- Large Artifacts take slightly longer to generate than text responses. For complex interactive tools, be patient with the generation.
- The Code tab in the Artifact panel shows you the source. Even without coding knowledge, you can often see whether the Artifact is doing what you asked.

Quick summary

Artifacts are rendered, persistent outputs that appear in a side panel alongside your conversation. They're the right format for interactive tools, formatted documents, visualizations, and anything that needs to be shared or used rather than just read. Claude produces them automatically for appropriate requests; you can also ask explicitly. Treat them as starting points to iterate on, not final deliverables after a single generation.

Section 6 — Artifacts, Interactive Visuals & Claude Design

Creating Mini Apps and Editable Tools Without Writing Code

Creating Mini Apps and Editable Tools Without Writing Code

One of the more surprising capabilities in Claude's Artifact toolkit is its ability to produce functional, interactive mini applications. Not mockups or wireframes (though it does those too), but actual working tools: calculators, estimators, decision helpers, trackers. If you've never had reason to describe yourself as someone who "builds apps," this is the lesson that might change that.

Why this matters

A lot of the tools knowledge workers need don't exist in any software product, or exist in bloated form in products you'd have to pay for, configure, and maintain. A quick pricing calculator for a proposal. A resource allocation helper for a team. A decision matrix for evaluating vendor options. A scoring rubric for reviewing candidates.

These are small tools. Building them as spreadsheets takes an hour. Getting them built by a developer costs time and coordination. Claude can produce them as interactive HTML Artifacts in five to ten minutes, and they run in any browser with nothing to install.

The idea in plain English

Mini apps made by Claude are HTML files with embedded JavaScript and CSS. They run in the browser, look presentable, and work without any server, database, or software installation. You can open them on your laptop, share them as attachments, or embed them in an internal tool.

They're best suited for:

Calculators: Input a few numbers, get a result. Pricing estimators, ROI calculators, budget trackers, time estimators.

Decision tools: Input criteria and weights, get a ranked output. Vendor comparison tools, project prioritization matrices, hiring rubrics.

Checklists and process guides: Interactive step-by-step guides where checking off items changes the state of the page.

Simple data entry forms: Input fields that calculate or display results as you fill them in.

Reference tools: Searchable tables, filtered lists, quick-access reference guides.

They're less suited for: anything requiring a database, login, persistent storage across users, or integration with external systems. For those, you need actual software development.

How this works in Claude

Describe the tool you want and ask Claude to build it as an Artifact. Be specific about inputs, outputs, and any calculations or logic involved.

Example prompts:

> *"Build an HTML Artifact that is a project effort estimator. It should have input fields for: number of team members, hours per person per week, and total weeks. It should display the total effort in person-hours and give a traffic-light indicator (green/yellow/red) based on whether the effort exceeds 500, 1000, or 2000 person-hours."*
>
> *"Create an interactive vendor comparison tool. I should be able to enter up to five vendor names and rate each on five criteria (cost, support quality, implementation ease, scalability, references) on a scale of 1-10. The tool should calculate a weighted total score and rank the vendors."*

> *"Make a simple meeting prep checklist as an interactive HTML page. It should have a list of items to check off before a client presentation, organized into categories (logistics, content, tech setup). Completed items should be visually struck through."*

Practical example

A sales manager needs to help her reps quickly estimate deal profitability. Currently, they use a spreadsheet that was built three years ago, has broken formulas, and requires a desktop to access. She asks Claude to build a replacement.

She describes the inputs: deal value, discount percentage, cost of goods, implementation cost, and expected annual support cost. She describes the outputs: gross margin per deal, margin percentage, and a breakeven period in months.

Claude produces an HTML Artifact with clean input fields, real-time calculation as the rep types, and the outputs displayed clearly. She shares the HTML file with her team via Slack. Her reps bookmark it. No IT request, no spreadsheet maintenance, no installation required.

Workflow design notes

Mini apps produced by Claude are first drafts, not finished products. They'll need review and likely several rounds of refinement:

Test the logic: Fill in known values and check that the outputs match what you'd calculate manually. Claude can make mathematical errors, especially in complex formulas.

Test edge cases: What happens when a field is left empty? What happens with an unusually large or small number? Edge case failures are common and usually easy to fix with a follow-up.

Check the interface: Is it clear what each input is for? Are there labels, tooltips, or instructions needed? Is the output easy to interpret?

Consider the audience: If you're sharing this with colleagues, the interface needs to be intuitive without explanation. Ask Claude to add help text or improve labels if needed.

Try this in Claude

Think of a small calculation or decision you make regularly that currently requires a spreadsheet or a mental exercise. Describe it to Claude and ask for an interactive HTML tool. Test the output with real values. Note what needs refinement. Run two to three follow-up iterations to get it working correctly.

Pro tips

- Start simple and add complexity in iterations. A basic version with two inputs is easier to verify than a complex version with ten. Add features once the core logic is correct.
- If the tool has significant math, ask Claude to explain the calculation logic in a follow-up. This helps you verify whether it's doing what you intended.
- Mini apps can include visual design improvements: ask Claude to "make this look more professional" or "use a clean, minimal style with consistent fonts and spacing."
- If a mini app is genuinely useful, ask Claude to add a way to export or print the results. A "print to PDF" button is easy to add and makes the tool significantly more shareable.

Quick summary

Claude can build functional interactive tools as HTML Artifacts: calculators, decision helpers, checklists, and simple data entry forms. These run in any browser without installation and can be shared as files. Describe the inputs, outputs, and logic clearly. Test the first version carefully and iterate. For small recurring tools that live between "too simple for software" and "too annoying to rebuild as a spreadsheet every time," mini apps are often the right answer.

Section 6 — Artifacts, Interactive Visuals & Claude Design

Visualizing Frameworks and Processes

Visualizing Frameworks and Processes

Some ideas communicate better as pictures than as prose. A decision tree, a process flow, a strategic framework, a responsibility matrix. Describing these in text forces the reader to build the visual in their head, which is cognitive work they shouldn't have to do. Claude can produce rendered visual diagrams that communicate structure directly. No drawing tools, no design software, no waiting for a designer.

Why this matters

Knowledge workers spend real time trying to make structure visible: manually drawing process maps, building PowerPoint shapes, commissioning diagrams from design teams. Most of these don't need to be beautiful. They need to be clear. Claude produces clear structural diagrams quickly, which means you can externalize structure fast rather than letting complex processes stay in your head or your bullet points.

This is also useful for communication. A stakeholder who won't read a six-paragraph process description will often understand the same content in a simple flowchart.

The idea in plain English

Claude produces two main types of visual diagrams as Artifacts:

SVG diagrams: Scalable vector graphics rendered directly in the Artifact panel. These work well for static visuals: framework boxes, concept maps, labeled diagrams, organizational charts.

Mermaid diagrams: A text-based diagram syntax that Claude knows well and that renders in the Artifact panel. Mermaid supports flowcharts, sequence diagrams, entity-relationship diagrams, state diagrams, Gantt charts, and more.

For most professional visualization needs, Mermaid diagrams are the more capable and reliable path. They support complex logic and relationships that are harder to express in SVG alone.

Types of diagrams Claude produces well

Flowcharts: Decision trees, approval processes, customer journeys, onboarding flows. Any process with branching paths and decision points.

Sequence diagrams: Showing how different parties interact over time. Useful for describing API interactions, communication flows, escalation procedures.

Process maps: Step-by-step sequences without branching. Operations processes, standard workflows, procedure documentation.

Concept maps and frameworks: Strategic frameworks like SWOT, Porter's Five Forces, jobs-to-be-done, or custom frameworks you've developed.

Org charts: Team structures, reporting lines, stakeholder maps.

Gantt charts: Timeline-based project plans with phases and dependencies.

How this works in Claude

Describe the diagram you want. Include all the steps, decisions, parties, or elements that need to appear.

Example prompts:

> *"Create a flowchart for our customer onboarding process. Start with account creation, then email verification, then a decision point: did they complete the profile setup? If yes, proceed to product tour. If no, send a reminder email loop. After the product tour, end at 'active user.'"*
>
> *"Draw a sequence diagram showing how a support ticket flows through our team: customer submits ticket, first-line support triages it, either resolves it or escalates to tier-two, tier-two either*

resolves or escalates to engineering, engineering resolves and notifies the customer."

"Create a SWOT analysis framework as a clean four-quadrant diagram with these items: [your SWOT content]."

"Build a simple org chart showing our department structure: VP of Marketing at the top, three direct reports (Content, Demand Gen, Brand), and two people under Demand Gen."

Practical example

An operations lead needs to document a vendor approval process that currently lives only in institutional knowledge. Three different stakeholders have slightly different memories of how it works. She needs to get it written down and approved.

She describes the process to Claude in a detailed prompt, including all the decision points she knows about and the approximate steps in sequence. Claude produces a Mermaid flowchart as an Artifact in about thirty seconds.

She reviews it, identifies two missing steps and one decision point that should branch differently, and asks Claude to update it. She then shares the diagram in a Slack message for team review. Three colleagues suggest minor corrections. She makes two more iterations. By the end of the meeting, the approved process diagram is documented and shared. What would have taken hours of document formatting and Visio wrestling took thirty minutes.

Workflow design notes

Diagram quality depends on the quality of your description. Before asking Claude for a diagram, spend two minutes writing down all the elements: the steps, the decisions, the parties, the sequence. A complete description produces a usable first draft. A vague description produces something that's structurally correct but missing most of what you need.

Diagrams often need iteration. The first version identifies what you forgot to include, what you got out of order, and what's unclear in the structure you described. Treat the first draft as a thinking tool, not a finished product.

One limitation: very complex diagrams with many nodes and complex logic can become unreadable. If a diagram is getting crowded, consider breaking it into two diagrams: a high-level overview and a detailed version of the complex section.

Try this in Claude

Choose a process from your work that someone recently asked you to explain or document. Describe it to Claude step by step, including any decision points. Ask Claude to create a flowchart as an Artifact. Review the output and note what's missing or wrong. Make two follow-up refinements.

Pro tips

- For Mermaid diagrams, Claude can also show you the Mermaid code in the Code tab. This is useful if you need to embed the diagram in a tool (like Confluence or Notion) that accepts Mermaid syntax directly.
- Sequence diagrams are particularly underused for explaining complex multi-party processes. They're easier to read than narrative descriptions for anything involving multiple actors.
- If a diagram needs to be included in a formal document, ask Claude to export it or describe how to save it as an image. SVG diagrams can typically be saved directly from the browser.

Quick summary

Claude produces flowcharts, sequence diagrams, process maps, frameworks, org charts, and other visual diagrams as Artifacts. Describe the elements and structure clearly, treat the first draft as a starting point, and iterate. Mermaid syntax supports the widest range of diagram types and renders reliably. Use diagrams when structure needs to be communicated efficiently, especially to audiences who won't read lengthy process descriptions.

Section 6 — Artifacts, Interactive Visuals & Claude Design

Inline Interactive Charts: From Numbers to Visualization in Seconds

Inline Interactive Charts: From Numbers to Visualization in Seconds

You have data. You need a chart. Normally this involves opening a spreadsheet, formatting the data, selecting a chart type, adjusting the labels, and then either exporting an image or taking a screenshot. Claude skips all of that. Paste the numbers, describe what you want, and Claude produces a rendered, interactive chart as an Artifact. This is one of the most immediately useful and underused features available.

Why this matters

The friction of creating a chart is disproportionate to the value. Most charts produced for professional purposes are not data visualizations for a permanent dashboard. They're one-time displays: a quick trend line for a meeting, a bar chart to support a point in a document, a

comparison across options before a decision. These don't need to live in a spreadsheet forever. They need to exist for the next hour.

Claude produces them in under a minute, and they're interactive: you can hover over data points, see values, and the visual scales correctly without manual adjustment.

The idea in plain English

Paste a table of numbers, a list of values, or describe the data you have. Tell Claude what type of chart you want and what it should show. Claude produces an HTML Artifact with a rendered, interactive chart.

Chart types Claude handles well: - **Line charts:** Trends over time - **Bar and column charts:** Comparisons across categories - **Pie and donut charts:** Proportional breakdowns - **Scatter plots:** Relationships between two variables - **Stacked bar charts:** Composition over time or across categories - **Area charts:** Volume or magnitude over time

The chart renders in the Artifact panel. You can hover over it, see tooltips with values, and in some cases interact with the data points. It looks presentable for use in slides, documents, or screen shares.

How this works in Claude

Provide the data and specify the chart. The clearest approach is to paste the data first, then describe what you want to visualize.

Example prompts:

> *"Here's my data: Q1: 47,000 Q2: 52,000 Q3: 49,000 Q4: 68,000*
>
> *Create a line chart showing quarterly revenue with the values labeled. Title it 'Revenue by Quarter.'"*
>
> *"Create a bar chart comparing these five departments by headcount and average project completion rate: [paste table] Show both metrics as separate bars. Use blue for headcount and orange for completion rate."*
>
> *"I have this data about where our leads come from: Organic search: 34% Paid ads: 28% Referral: 18% Events: 12% Other: 8%*
>
> *Make a clean donut chart with these percentages and a legend."*

Practical example

A finance manager is preparing for a budget review meeting in two hours. She needs to show the team how actual spend tracks against budget across six departments. She has the numbers in a spreadsheet.

She pastes the two columns into Claude and says: "Create a grouped bar chart showing budget vs. actual for each department. Use green for under budget, red for over budget. Add a title: 'Q3 Budget vs. Actual.'"

Claude produces the chart. The manager screenshots it, drops it into her slide deck, and moves on to the next preparation task. No Excel chart formatting. No adjusting axis labels. No fussing with colors. Three minutes total.

Workflow design notes

Charts from Claude are not replacements for your permanent data infrastructure. They're working tools for in-the-moment communication. A dashboard you check daily should live in a proper analytics tool. A chart for this week's meeting can live in a Claude Artifact.

A few practical considerations:

Data accuracy is your responsibility. Claude charts the data you give it. If your numbers are wrong, the chart is wrong. Always verify the source data before producing a chart that will be shared or acted upon.

Visual design is functional, not polished. Charts from Claude are clean and readable. They are not as visually refined as charts produced by a professional data visualization tool or a skilled designer. For external presentations or formal reports, use them as references or first drafts; have them refined if the audience warrants it.

Asking for color and style changes is easy. If the default colors clash with your brand or the chart type isn't quite right, a simple follow-up ("change the colors to our brand blue and grey" or "make this a horizontal bar chart instead") takes seconds.

Try this in Claude

Find a set of numbers you've been meaning to visualize: monthly metrics, a budget breakdown, performance comparisons, or any simple data set. Paste the data into Claude and ask for the chart type that fits the message. Run at least one follow-up to adjust something: a color, a label, or the chart type. Note how quickly the visual came together compared to your usual process.

Pro tips

- For time-series data, always specify whether the x-axis represents dates, quarters, months, or generic periods. Claude will label it accordingly.
- If the chart looks cluttered, ask Claude to "simplify the chart: fewer data labels, cleaner gridlines, and reduce the number of colors to two."
- For data with outliers, ask Claude to note the outlier in the chart title or add an annotation. This prevents the outlier from distorting the visual interpretation.

• Charts produced as HTML Artifacts can be screenshotted directly, or the source HTML can be opened in a browser for a cleaner screenshot.

Quick summary

Paste data into Claude, describe the chart type and any formatting requirements, and Claude produces an interactive rendered chart as an Artifact in seconds. This is the right tool for charts needed quickly for meetings, documents, and decisions. Not a replacement for permanent analytics infrastructure, but a fast, practical tool for one-time data visualization. Verify your data before sharing; adjust colors and labels through simple follow-up requests.

Section 6 — Artifacts, Interactive Visuals & Claude Design

Pitch Decks, One-Pagers, and Landing Page Mockups with Claude Design

Pitch Decks, One-Pagers, and Landing Page Mockups with Claude Design

The gap between "I have an idea" and "I have something I can share with someone" used to require a designer, a presentation tool, an hour with templates, or all three. Claude compresses that gap. With Claude Design capabilities, you can produce visually structured pitch deck outlines, formatted one-pagers, and landing page mockups that are presentable and shareable from a standing start.

Why this matters

Ideas fail to get traction not because they're bad but because they're hard to evaluate without a concrete expression of them. A rough proposal in a Word doc gets half the consideration of the same content in a cleanly structured one-pager. A new product concept in a slide deck gets more serious attention than the same content in a long email.

Claude can't replace a professional designer for polished final work. But it can get you from concept to shareable draft in minutes rather than hours, and that draft is often good enough to get the feedback you need before investing in polish.

The idea in plain English

Pitch deck structures: Claude can produce a complete slide deck outline with content for each slide: problem, solution, market size, team, traction, ask. It can also produce this as a rendered HTML presentation (a simple web-based slideshow) that you can share as a link or file rather than attaching a PowerPoint file.

One-pagers: A single-page summary document designed to be read in under two minutes. Product one-pagers, executive summaries, project proposals, service descriptions. Claude produces these as formatted Artifacts that look like documents rather than chat responses.

Landing page mockups: HTML-rendered page layouts that show what a web page could look like. Useful for: pitching a digital product concept, getting stakeholder alignment on website direction, or creating a reference design before involving developers.

Visual formatting: Claude uses clean HTML and CSS to produce outputs that are visually structured: proper heading hierarchy, consistent spacing, professional color schemes, readable typography. The output looks made, not generated.

How this works in Claude

For any of these output types, describe what you're creating, who it's for, and what the key messages are. Specify that you want a polished Artifact.

Pitch deck:

> *"Create a pitch deck outline for [your product/idea] as an HTML presentation Artifact. The audience is angel investors. Include eight slides: Problem, Solution, How It Works, Market Opportunity, Business Model, Traction, Team, and Ask. Use a clean, professional dark theme. Here's the content: [your notes]."*

One-pager:

> *"Create a formatted one-pager Artifact introducing our consulting service to a mid-market CFO audience. Include: what the problem is, what we do, why it works, who we've worked with, and how to engage us. Professional, concise, suitable for emailing as a PDF."*

Landing page mockup:

> *"Create an HTML mockup for a SaaS product landing page. The product is [brief description]. The target audience is [description]. Include sections for: hero headline, three key benefits, a feature overview, social proof placeholder, and a call-to-action. Use a clean minimal style, white background, one accent color."*

Practical example

A consultant is pitching a new service offering to a potential partner. She has the concept clear in her head but nothing written down. She has a meeting in two hours.

She dictates a rough description of the offering to Claude (see Section 3), then asks Claude to turn it into a formatted one-pager as an Artifact. She describes the structure she wants, the audience (operational leaders at PE-backed companies), and the key messages.

Claude produces a formatted HTML Artifact in about ninety seconds. She previews it, adjusts two sections through follow-up requests, and screenshots it as a PDF for the meeting. The one-pager she arrives with looks considered and professional. The meeting goes differently than if she'd arrived with a blank slide.

Workflow design notes

These outputs are starting points, not final products. Know what stage of polish is appropriate for your situation:

For internal discussions: Claude's first or second draft is usually sufficient. The goal is to externalize the idea clearly, not to produce a finished document.

For external sharing (low stakes): A well-structured Claude draft with one or two refinement iterations is typically presentable. Review it as if you're the recipient.

For formal presentations or high-stakes communications: Use Claude's draft as a starting point. A professional designer, copywriter, or communications colleague should review and refine before the final version goes out.

The most common refinements after the first Artifact: - Adjusting the headline to be more direct or compelling - Changing the visual style (color, font weight, spacing) - Reordering sections to put the most compelling content first - Shortening content that's too dense for the format

Try this in Claude

Pick a concept you've been meaning to document: a new initiative, a service offering, an idea for improvement. Describe it to Claude in a couple of sentences and ask for a formatted one-pager Artifact. Review the output as if you're the intended recipient. What does it get right? What would you change? Make two follow-up refinements and then evaluate whether the revised version is something you'd actually use.

Pro tips

- For landing page mockups, specify the accent color and any brand-relevant details. "Use blue (#0057B7) as the accent color and keep the overall design minimal" produces more on-brand results than a generic color scheme.
- If you have existing branding guidelines, paste the key elements into the prompt. Claude applies them to the output.

- For pitch decks meant to be presented live, ask Claude to add speaker notes or talking points below each slide.
- One-pagers work best when they're actually one page. Ask Claude to "tighten this so the key messages fit on a single printed page without feeling crowded."

Quick summary

Claude Design capabilities produce pitch deck structures, formatted one-pagers, and landing page mockups as rendered HTML Artifacts. The output is visually structured and presentable, appropriate for internal reviews and early external sharing. Treat the first draft as a strong starting point, refine through follow-up, and apply professional review before using outputs for high-stakes external communications.

Section 6 — Artifacts, Interactive Visuals & Claude Design

Reviewing and Refining AI-Generated Designs

Reviewing and Refining AI-Generated Designs

An Artifact that looks impressive on first view is not necessarily an Artifact that's ready to use. Claude produces visually coherent outputs quickly, and that speed can make them feel more finished than they are. Building a reliable review habit for visual outputs prevents the specific category of mistake where something looks polished but communicates the wrong thing.

Why this matters

Text errors are usually obvious: wrong word, missing sentence, incorrect fact. Design errors are sneakier. A layout that looks clean may bury the most important information. A color choice that seems professional may be inaccessible to someone with color vision deficiency. A pitch deck that looks like a deck may be structured in a way that undermines the argument rather than supporting it.

Reviewing AI-generated visuals requires a different mental mode than reviewing text. You're evaluating not just what it says but how it communicates.

A review framework for AI-generated visual outputs

1. Does it communicate the right message at a glance? Look at the output for five seconds. What is the first thing your eye goes to? What's the first thing you understand? If that's not the

most important thing you want to communicate, the hierarchy is wrong.

2. Is the structure logical for the audience? Work through the output in sequence. Does each section follow logically from the previous one? Does the structure match how the intended reader needs to receive the information? A pitch deck for investors follows a different logic than one for customers. A one-pager for a CFO prioritizes differently than one for an operations lead.

3. Is anything misleading or overstated? Claude occasionally overstates, uses overly confident language, or makes claims that seem stronger than your actual position. Read every headline and data label as a skeptic. Does each claim hold up under scrutiny?

4. Is the visual design accessible? Check color contrast: is text readable against its background? Are key distinctions conveyed only through color (which some readers won't see), or also through shape, position, or label? For anything going to a broad audience, accessibility matters more than aesthetics.

5. Is the length and density appropriate for the format? A one-pager should be skimmable in two minutes. A landing page should drive to a clear action. A pitch deck should have one idea per slide. If any section is too dense, ask Claude to simplify it.

6. Does it match your voice and brand? The output was generated. Its default tone may not sound like you. Read it aloud and ask: would I actually say this? Where does it sound like AI?

How to refine effectively

Once you've reviewed the output, follow-up requests are the refinement tool. Be specific about what to change and why.

Structure changes:

> *"Move the benefits section before the features section. The reader needs to understand why this matters before they care about how it works."*

Tone changes:

> *"The headline is too hyperbolic. Change it to something more specific and credible: instead of 'Transform your operations,' say something about what the specific outcome is."*

Visual changes:

> *"The chart colors are too similar; use high-contrast colors that differentiate the categories clearly even in black and white."*

Density changes:

> *"This section has too many points. Keep only the three most important ones. Each point should be one sentence, not a paragraph."*

Credibility checks:

> *"Read the claims in this deck as a skeptical investor. Flag any that seem unsupported or overstated."*

Practical example

A product manager produces a feature announcement one-pager as a Claude Artifact. On first look, it's clean and well-structured. On closer review:

- The headline is generic ("Introducing Our New Reporting Feature") rather than benefit-focused
- The second paragraph buries the most important detail: this feature saves users two hours per week
- One of the screenshots is described as "best-in-class," which sounds like marketing speak the company wouldn't actually use
- The CTA at the bottom is weak: "Learn more" rather than something specific

She runs four targeted follow-up requests addressing each issue. The revised one-pager takes about eight minutes in total. The difference between the first draft and the final version is not visual; it's in how clearly and honestly it communicates.

Workflow design notes

Build the review habit before the polish habit. It's tempting to immediately ask Claude to "make this more visually polished" before checking whether the content and structure are right. Fix substance first, then aesthetics. Re-polishing content that will get rearranged wastes iterations.

For outputs that will be shared with external stakeholders, consider a two-stage review: your own review for structure and content, then a colleague's review for whether the message lands clearly to someone unfamiliar with the context.

Try this in Claude

Take any Claude-generated visual Artifact you produced in the previous lessons. Apply the six-point review framework above. Identify at least two things to change. Write targeted follow-up prompts for each change. Run them. Compare the before and after.

Pro tips

- "Read this as [specific audience]. What would they find confusing or unconvincing?" is often more useful than reviewing it yourself as the author.
- If a visual element doesn't render well, ask Claude to describe what it intended and whether there's a simpler way to achieve the same communication goal.

• For landing page mockups: read the page as if you've never heard of the product. By the end of the page, do you know what it does, who it's for, and what to do next?

Quick summary

AI-generated visual outputs require deliberate review using a different lens than text review. Check hierarchy, structure, claims, accessibility, density, and voice. Refine with specific targeted follow-up prompts. Fix substance before aesthetics. For external-facing outputs, add a colleague review before final use. The value of Claude Design is in speed of draft; the value of review is in ensuring that draft actually earns its place.

Section 6 — Artifacts, Interactive Visuals & Claude Design

Teaser: From Static Artifacts to Live Ones

Teaser: From Static Artifacts to Live Ones

Every Artifact you've produced in this section is static. It displays what it displayed when Claude made it. If the underlying data changes, the Artifact doesn't. That's fine for most of what Chat is used for. But it's a real limitation for dashboards, briefings, and any output you'd want to keep current without rebuilding it each time.

Section 13 removes that limitation. This lesson is a brief look ahead at what becomes possible.

The static Artifact constraint

An HTML chart of your Q3 metrics shows Q3 metrics. To see Q4, you rebuild the chart. An HTML dashboard showing your open tasks lists the tasks that existed when Claude generated it. To see new tasks, you rebuild the dashboard.

For one-time views, this is fine. For recurring views, this creates a maintenance problem: you're not checking a dashboard, you're rebuilding one every time you want to look at it.

What live Artifacts add

Live Artifacts (covered fully in Section 13) connect to real data sources: your email connector, your calendar, your project management tools, external APIs, anything Claude can reach through connectors. The Artifact renders from that live data each time it's opened, rather than from data baked in at creation time.

The result: a dashboard you create once and check repeatedly. A morning briefing page that always shows today's context, not last week's. A project status view that reflects the current state of your tasks without requiring you to generate a new one.

The Cowork context

Live Artifacts are a Cowork feature, not a Chat feature. They require: - Connectors that provide access to real-time data - The Cowork environment where live data access is supported - The desktop app running so the connectors can reach their sources

In Chat, Artifacts are rendered at generation time. In Cowork, live Artifacts can pull current data when opened.

What this means for your workflow design

Think about the Artifacts you've produced in this section. Are any of them things you'd want to check regularly rather than just once? A recurring report format. A status overview. A team-facing briefing.

Those are candidates for live Artifacts in Cowork. You're not throwing away the work you did building them here; you're building the understanding of what a good Artifact looks like before you make it live and connected.

A preview of Section 13

In Section 13, you'll: - Learn the difference between static and live Artifact architecture - Connect an Artifact to a data source (your calendar, your email, a project tool) - Build a dashboard that refreshes when you open it - Pair a live Artifact with a scheduled task so it's always current even before you look at it

The skills you're building in this section, how to structure clear, useful Artifacts, how to review and refine them, how to describe what you want precisely enough to get a useful first draft, all carry forward into live Artifact work.

Try this in Claude (forward-looking exercise)

Look at the Artifacts you've produced in this section. For each one, ask: if I wanted to see a current version of this weekly or daily, what data source would it need to connect to? Make a brief note for each one.

When you reach Section 13, those notes become your starting point for live Artifact design.

Quick summary

Static Artifacts display data as of when they were generated. Live Artifacts (a Cowork feature) connect to real data sources and display current information when opened. The Artifact design and review skills you're building in this section are the foundation for live Artifact work in Section 13. For recurring views and dashboards, live Artifacts eliminate the rebuild-every-time maintenance cost of static outputs.

SECTION 07 · ACT 1 — CHAT

Section 7 of 14

Section 7: Skills, Plugins & Chat Power Features

Section 7: Skills, Plugins & Chat Power Features

By this point in the course, you've built a solid foundation: you prompt well, you work with documents, you use connectors, you've organized your work into Projects, and you can produce useful Artifacts. Section 7 is about taking what you do repeatedly and making it reusable, and about getting more control over long conversations.

Why this section matters

Repetition is the enemy of efficiency. If you've written the same type of prompt fifteen times, it's time to turn it into a Skill. If you've been onboarding each new Project context from scratch, Plugins might be a better answer. And if long Claude conversations have ever gone sideways and you didn't know how to recover, the power features in this section fix that.

What you'll learn

What Skills are and how to create, browse, and install them. A Skill is a reusable instruction set that tells Claude how to approach a specific type of task. Creating a Skill for a repeated task saves setup time and produces more consistent outputs than writing prompts from scratch each time.

Plugins and the plugin marketplace. Plugins bundle Skills, connectors, and knowledge into a single installable package. Where a Skill teaches Claude one behavior, a Plugin installs a whole workflow: a set of Skills, a connector, and the reference knowledge Claude needs to use them. The Plugin marketplace is where you find and install these bundles.

The hierarchy: prompt < Skill < Plugin < Project. Understanding which layer of instruction takes precedence, and which is the right tool for each type of instruction, prevents confusion and ensures Claude behaves as intended.

Power features: Fork, /rewind, /recap, /btw. Four features that give you control over long or complicated conversations: forking to explore a different direction without losing the original, rewinding to a previous point, recapping to get a summary of a long thread, and using /btw to give Claude a side note without it becoming part of the main response.

Output Styles for consistent formatting. How to tell Claude how to format its responses consistently across all conversations, without repeating the instruction every time.

Choosing the right tool for a repeated task. A decision framework for when to use a prompt, a Skill, a Plugin, or a Project instruction.

The mini project

You'll identify a repeated task from your own work and outline how it becomes a reusable Skill: what the task is, what inputs it needs, what the output should look like, and what standing instructions would make it consistent. This exercise forces clarity about what you actually repeat and how to encode that into a reusable form.

How to approach this section

The concepts in this section build on everything in Act 1. If you've been working through the course in order, you have enough Claude experience to recognize what you repeat and what would benefit from a Skill or Plugin. If you haven't, read through and return to this section once you have more usage to draw from.

The power features are best learned by using them once in a real conversation where they'd be helpful. Don't try to memorize them; just know they exist and what they do so you reach for them when the need arises.

Section 7 — Skills, Plugins & Chat Power Features

What Skills Are and How to Use Them

What Skills Are and How to Use Them

A Skill is a reusable instruction set that tells Claude how to approach a specific type of task. Instead of writing a detailed prompt every time you need Claude to do the same thing, you install the Skill once, and Claude knows how to behave whenever you invoke it. Skills are how repeated tasks stop being repetitive.

Why this matters

The best prompts you write are worth more than a single use. If you've crafted an excellent prompt for summarizing client calls, reviewing documents, generating weekly reports, or rewriting content in your brand voice, that prompt is reusable intellectual property. Saving it as a Skill means you never have to reconstruct it, you share it consistently with anyone who needs it, and you can improve it over time without having to remember where you left the original prompt.

The idea in plain English

A Skill has two main components:

An invocation: A name or trigger phrase that activates the Skill. When you type the Skill's name (usually preceded by /) or use the designated trigger, Claude knows to follow the Skill's instructions for this conversation.

An instruction set (SKILL.md): The detailed instructions that tell Claude what to do when the Skill is invoked. This is essentially a standing prompt encoded into a structured format. It can include: what the task is, what inputs to expect, how to handle them, what the output should look like, any constraints, and examples if useful.

From a user perspective, invoking a Skill is faster than writing a prompt. You type /summarize-meeting and Claude knows exactly what format to produce, what to extract, and how to present it, because the Skill's instructions encode all of that.

Types of tasks that benefit from Skills

Skills are most valuable when: - You do the same type of task repeatedly (weekly reports, meeting summaries, content reviews) - The task requires a consistent output format across multiple uses or multiple team members - The task has a complex setup that you don't want to re-explain each time - You want to share a workflow with colleagues without each person having to figure out the prompt

Examples of useful Skills: - Meeting note summarizer (extracts decisions, action items, and open questions) - Email drafter (takes a brief description and produces a draft in your voice) - Document reviewer (checks a document against a specific rubric) - Weekly report generator (formats metrics into a standing report structure) - Content toner (rewrites content to match a brand voice) - Risk identifier (reads a document and extracts potential risks)

How Skills work in Claude

Skills appear in the Skills section of the Claude interface or in the Skills marketplace. You can browse installed Skills, install new ones from the marketplace, and in some configurations create your own.

When a Skill is installed, its name typically appears as a slash command in the chat interface. Typing / brings up a list of available Skills and their descriptions, letting you select the one you need.

Some Skills work in specific contexts (within a Project, with a specific connector) and some work generally across any conversation.

Creating your own Skills involves writing a SKILL.md file that follows a defined structure. Cowork exposes this directly: you can browse, install, and build Skills from within the Cowork interface. This is covered further in the Cowork sections.

Practical example

A communications team produces ten client briefing documents per month. Each one follows the same format: executive summary, key themes, recommended actions. Until now, each team member writes their own prompt, which produces slightly inconsistent outputs that all require editing.

The team creates a "Client Briefing" Skill with a SKILL.md that specifies: the exact sections, the length requirements per section, the tone, and the specific extraction task (the inputs are a set of meeting notes and the client context). They install it in their shared Project.

Now every team member invokes /client-briefing, pastes the meeting notes and client context, and receives a consistently structured briefing document. The QA burden drops because the Skill's instructions encode the team's quality standards.

Workflow design notes

Skills work best for well-defined, repeatable tasks. They're less useful for tasks that require significant variability or judgment in the setup, because the Skill's instructions assume a consistent structure.

One common mistake: building Skills for tasks you don't actually repeat often. The overhead of creating a Skill (writing the SKILL.md, testing it, installing it) is worthwhile for tasks you do weekly or more. For monthly-or-less tasks, a saved prompt document is usually more practical.

Skills also degrade if the underlying task changes. If your report format changes or your output requirements evolve, the Skill needs to be updated to match. Build in a habit of reviewing Skills when you notice their outputs drifting from your current standard.

Try this in Claude

Identify the one task you've done with Claude most frequently in this course. Describe it as if you were writing instructions for a new colleague: what's the input, what's the process, what's the output? That description is the first draft of a Skill. In Section 9 (Cowork), you'll see how to turn it into an actual installable Skill.

Pro tips

- Name Skills clearly and specifically. "Summarize" is too vague. "Client-meeting-summary" tells you exactly when to use it.

• Test a new Skill on five real examples before relying on it. Edge cases reveal gaps in the instructions that weren't apparent in design.
• Keep SKILL.md instructions focused. The more a Skill tries to do, the harder it is to do any one thing consistently. One task per Skill is a good rule of thumb.

Quick summary

A Skill is a reusable instruction set that tells Claude how to handle a specific type of task. It's invoked by name and produces consistent output without requiring a detailed prompt each time. Skills are most valuable for repeated tasks with consistent inputs and outputs. Create them for tasks you do weekly or more; name them specifically; test them before relying on them.

Section 7 — Skills, Plugins & Chat Power Features

Plugins: Installing a Complete Workflow in One Step

Plugins: Installing a Complete Workflow in One Step

A Skill teaches Claude one behavior. A Plugin installs a complete workflow: a set of Skills, a connector or two, and the reference knowledge Claude needs to use them together. Where a Skill is a recipe card, a Plugin is a fully stocked kitchen with the recipe included.

Why this matters

Setting up a sophisticated Claude workflow from scratch takes time: identifying the right connectors, configuring permissions, writing Skills, testing everything together. For common professional workflows, someone has already done that work. The Plugin marketplace is where you find and install those pre-built, tested workflows.

For teams, Plugins also solve the alignment problem: everyone installs the same Plugin and gets the same capabilities and the same baseline behavior, rather than each person building their own approximate version of the same workflow.

The idea in plain English

A Plugin is a bundle that can contain:

Skills: Pre-written instruction sets for specific tasks within the domain the Plugin covers. A legal review Plugin might include Skills for contract review, risk identification, and clause extraction.

Connectors: Authorized connections to specific tools or data sources the Plugin needs. A project management Plugin might connect to Asana or Linear. A sales Plugin might connect to Salesforce.

Knowledge: Reference documents and context built into the Plugin. A financial analysis Plugin might include relevant frameworks, formulas, and industry benchmarks that Claude uses automatically when the Plugin is active.

Instructions: Standing guidance that applies whenever the Plugin is active, shaping how Claude approaches work in this domain.

You install a Plugin in a few clicks. Claude gains all the associated capabilities without you having to configure each element separately.

The Plugin marketplace

The Plugin marketplace is where you browse and install Plugins. Plugins come from: - Anthropic (official Anthropic-developed Plugins for common workflows) - Third-party developers (Plugins built by tool vendors, agencies, or independent developers) - Your organization (custom Plugins built for internal workflows)

When browsing the marketplace, each Plugin shows you what it includes (Skills, connectors, knowledge), what permissions it requires, and who built it.

Before installing a Plugin, check: - What permissions it requests: does it need access to your email? Your file system? Be sure the permission scope is appropriate for what the Plugin does. - Who made it: Anthropic Plugins and established vendor Plugins have different trust levels than anonymous third-party Plugins. - What problem it solves: does your workflow actually need this, or is it impressive-seeming without being useful for your specific work?

How this works in Claude

In the Claude interface, the Plugin section appears in Settings or in the Cowork interface depending on your setup. From there, you can browse the marketplace, view Plugin details, and install with the required permission grants.

Once installed, a Plugin's Skills appear in your /commands list and its connectors become available for use. If the Plugin includes knowledge documents, they're available as context without uploading anything manually.

In a shared Project or team Cowork workspace, Plugins can be made available to everyone with access. This is how teams standardize their Claude capabilities without each person independently configuring from scratch.

Practical example

A small marketing agency wants to set up Claude for all of their content production work. They could: write Skills for each content type, connect their CMS, add their brand guides, write a set of standing instructions. This would take a day.

Instead, they find a "Content Production Suite" Plugin in the marketplace. It includes Skills for blog posts, social media, email campaigns, and landing pages; connects to their CMS via API; includes a brand voice framework; and has standing instructions for consistent formatting. They install it, configure the CMS connection with their credentials, upload their specific brand guide, and are running in an hour.

The Plugin saved them from building from scratch. Their customizations (their specific brand guide, their specific CMS) are overlaid on top of the Plugin's foundation.

Workflow design notes

Plugins are powerful but require careful permission review. When a Plugin requests access to your email, your calendar, your file system, or external services, you're extending trust. Ask: - Does this Plugin actually need this access to do its job? - Who built this Plugin and how much do I trust them? - What happens to the data the Plugin accesses?

For personal use, some permissions are fine to accept. For enterprise use, Plugin installations may require IT review and approval, especially if they access corporate data.

Plugins also need to be kept current. A Plugin built six months ago for a specific tool version may not work correctly after that tool updates its API. Build in periodic checks on Plugin performance, especially for ones that power critical workflows.

Try this in Claude

Browse the Plugin marketplace. Find one Plugin that addresses a type of work you do. Read its description, Skills list, and permission requirements. Even if you don't install it, this exercise familiarizes you with what's available and how Plugins are structured. Note anything that surprises you about what the Plugin includes or what it requires.

Pro tips

- "Try before you commit" isn't fully possible with Plugins, but you can often find reviews or community discussions about widely used Plugins before installing them.
- If a Plugin includes a connector you already have configured, it will typically use your existing connection rather than requiring a new authorization.
- For team Plugin installations, designate one person to evaluate and approve new Plugins rather than letting everyone install independently. This keeps the shared workflow consistent.

Quick summary

Plugins bundle Skills, connectors, and knowledge into installable packages. They're the fastest path to a complete domain-specific workflow. The Plugin marketplace has Anthropic, third-party, and organizational options. Review permissions carefully before installing, especially for Plugins accessing sensitive data. For teams, shared Plugin installations standardize capabilities and reduce individual setup time.

The Instruction Hierarchy: Prompt, Skill, Plugin, Project

The Instruction Hierarchy: Prompt, Skill, Plugin, Project

Claude's behavior in any given conversation is shaped by multiple layers of instruction. Your prompt is the most immediate layer, but it's not the only one. Understanding what each layer does, which overrides which, and which is the right tool for each type of instruction prevents confusion and makes your Claude setup more predictable.

Why this matters

When Claude behaves unexpectedly, it's often because one layer of instruction is conflicting with or overriding another. The instructions in a Project tell Claude to be formal, but a Skill you invoked asks for casual language. A Plugin has standing constraints that conflict with your prompt. Understanding the hierarchy tells you where the conflict is and how to resolve it.

It also helps you make deliberate choices: if you want a behavior to persist across everything you do with Claude, where do you put the instruction? If you want it to apply only within one Project, where does it go?

The hierarchy explained

From most specific (lowest) to most persistent (highest):

Prompt (most immediate, least persistent) What you type in a single message. This is the most specific instruction and applies to this message and this conversation only. It's also the most flexible: you can say anything in a prompt.

Skill (task-specific, invoked explicitly) A Skill's instructions apply when the Skill is invoked. They're more persistent than a prompt but only active when you explicitly invoke the Skill. They define how Claude approaches a specific task type.

Plugin (domain-level, always active when installed) Plugin instructions apply to any conversation where the Plugin is active. If a Plugin's standing instructions shape Claude's behavior in a domain (legal, marketing, financial), those instructions apply throughout that Plugin context without you needing to invoke them.

Project (workspace-level, most persistent) Project instructions apply to every conversation within the Project. They're the highest-level persistent instructions within your Claude setup. They can set context, tone, constraints, and personas that override more specific layer defaults.

How conflicts resolve

When instructions at different layers conflict, the general rule is that more specific instructions take precedence over more general ones.

A prompt that says "be casual" in a Project instructed to "be formal" will typically produce a casual response, because the prompt is the most immediate instruction. If you want the Project instruction to be inviolable, you need to write it as a constraint: "always use formal language, regardless of how the prompt is phrased."

Plugin instructions interact with Project instructions in ways that depend on the specific Plugin. Generally, Plugin instructions that establish domain context (like specialized knowledge frameworks) operate alongside Project instructions rather than competing with them.

A practical map

What you want	Where to put it
One-time specific instruction	Prompt
Behavior for a specific task type (invoked explicitly)	Skill
Always-on domain behavior for a category of work	Plugin
Consistent behavior across all work in a workspace	Project instructions
Global preferences for all Claude interactions	Memory or global settings

How this works in Claude

You don't usually need to think about the hierarchy explicitly during normal work. It matters when: - You're setting up a new Project or Plugin and want to know where to put standing instructions -

You notice Claude behaving unexpectedly and need to diagnose which layer is causing it - You're building an automated Cowork workflow and need to ensure the instructions apply reliably

To diagnose unexpected behavior: start at the prompt level and work up. Is something in your prompt triggering the behavior? In the Skill you invoked? In the Plugin? In the Project? Find the source, then decide whether to modify that layer or override it at a more specific level.

Practical example

A legal team uses Claude with: - A "Legal Review" Plugin that establishes legal analysis conventions and conservative, precise language - A "Client Contracts" Project with client-specific context and confidentiality constraints - A "Risk Identification" Skill for extracting risks from documents - Individual prompts for specific review tasks

The Plugin's precision requirements apply throughout the legal context. The Project's confidentiality constraints apply throughout the client engagement. The Skill's extraction format applies when invoked. The prompt addresses the specific contract at hand.

All four layers work together because they address different things. Problems arise when they conflict: if the Plugin says "never recommend actions" but a prompt asks Claude to "recommend next steps," there's a conflict. The team resolves it by adding a Project-level nuance: "within this Project, recommendations are permitted when clearly labeled as preliminary analysis, not legal advice."

Try this in Claude

Look at a Project you've set up. Identify one behavior you want to apply consistently across all conversations in that Project. Write it as a Project instruction. Then identify one behavior you want only for a specific task type. Write that as a potential Skill instruction. Notice how different the two instructions need to be to sit correctly at each layer.

Pro tips

- When in doubt about which layer to use, start with the prompt. If you find yourself repeating the same prompt instruction in every session, promote it to a Project instruction.
- Plugin and Project instructions can work together well when they address different concerns. Don't try to consolidate everything into one layer.
- If a Plugin instruction is causing problems in a specific Project, you can add an overriding constraint in the Project instructions. This is preferable to modifying the Plugin itself.

Quick summary

Prompt, Skill, Plugin, and Project are four layers of instruction that shape Claude's behavior, from most immediate to most persistent. More specific instructions generally override more general ones. Use the right layer for each type of instruction: prompts for one-time specifics, Skills for task patterns, Plugins for domain context, Projects for workspace consistency. Understanding the hierarchy helps you diagnose unexpected behavior and design more reliable workflows.

Section 7 — Skills, Plugins & Chat Power Features

Power Features: Fork, /rewind, /recap, and /btw

Power Features: Fork, /rewind, /recap, and /btw

Long conversations with Claude can get complicated. They drift off course. They accumulate context that's no longer relevant. They produce a response you want to explore further without committing to a direction. Claude's power features give you control over conversations as they evolve, rather than forcing you to start from scratch or live with a suboptimal thread.

Why this matters

Most Claude users treat conversations as linear: they go forward, they can't go back, and if a conversation goes sideways they start over. The power features break that constraint. They let you branch, rewind, summarize, and annotate in ways that make long working sessions much more productive.

These aren't features you need every session. But when you need them, knowing they exist is the difference between a frustrating conversation and a controlled one.

The four features

Fork

Forking creates a branch from a specific point in the conversation. Claude continues from that point in a new thread, without inheriting the direction the original conversation took after that point.

When to use it: You've reached a useful point in the conversation and want to explore two different directions without choosing between them. You want to try a different approach to a problem without losing the version you have. You want to give someone else a copy of the conversation up to a certain point so they can continue it.

How it works: Look for the Fork option on any message in the conversation (typically in the message options menu). Forking from that message creates a new conversation that starts from the conversation state at that point.

/rewind

Rewinds the conversation to a specified earlier point, removing messages that came after that point.

When to use it: A response took the conversation in the wrong direction and you want to go back and try a different prompt. You made an error in your prompt and the subsequent conversation is built on that error. The conversation drifted and you want to return to the last point where things were on track.

How it works: Type /rewind followed by identification of where you want to return to (often you can click a specific message to rewind to that point). Messages after that point are removed from the thread.

Note: rewinding removes context. If Claude produced something useful after the point you're rewinding to, copy it first.

/recap

Generates a summary of the conversation so far: the key decisions, conclusions, context, and important points. Useful in long conversations where the thread has accumulated so much that it's hard to hold in mind.

When to use it: A long conversation has become hard to navigate. You're returning to a conversation after a break and want to re-orient. You want to share a summary of what you and Claude have worked through with someone who wasn't in the conversation.

How it works: Type /recap. Claude summarizes the conversation to that point. The recap appears in the conversation and can be used as context for the next part of the session.

/btw

Passes a side note to Claude that Claude should consider but that is not part of the main response or conversation flow. Claude registers the information but doesn't necessarily respond to it directly.

When to use it: You want to give Claude a quiet context update without making it a topic of discussion. "By the way, the meeting was rescheduled to Friday" as a note to adjust Claude's context without triggering a response about the rescheduling. You want to clarify something without interrupting the main thread.

How it works: Type /btw followed by the note. Claude processes it as contextual information.

Practical example

A strategy consultant is working through a complex competitive analysis in Claude. The conversation is fifty messages long and covers multiple companies and frameworks. She needs to:

Branch before committing: She's about to ask Claude to draft a recommendation section, but she wants to try both an aggressive and a conservative framing. She forks the conversation at the point before the recommendation, tries the aggressive framing in the original, and explores the conservative framing in the fork. She picks the better one.

-

Recover from a drift: Several messages back, Claude started analyzing a competitor that's actually outside the scope of the analysis. She rewinds to the point before that message, removing the off-scope analysis, and redirects.

-

Manage a long thread: After forty messages, she types /recap to get a summary of the analysis so far. She uses the recap as a brief to start a fresh conversation for the final synthesis phase, rather than carrying all forty messages of accumulated context.

-

Each feature addressed a different problem. Together, they kept a complex working session under control.

Workflow design notes

Power features are particularly useful in Cowork, where conversations can be part of longer multi-step workflows. Being able to rewind, recap, and fork without starting over makes iterative work significantly more manageable.

For team use, Fork is especially valuable: a conversation that reaches a useful state can be forked and shared with colleagues who continue from that point without needing the full conversation history.

Try this in Claude

In your next extended Claude conversation (a working session rather than a quick question), practice one of these features. When the conversation reaches a natural branch point, fork it. Or, if a response sends the conversation in the wrong direction, use /rewind instead of just sending another corrective message. Note whether the feature saved you effort compared to the alternative.

Pro tips

• Copy valuable outputs before rewinding. Rewound messages are gone; you can't retrieve them without redoing the work.
• /recap is useful not just for long conversations but for handing off a conversation to a colleague. The recap provides the context they need to pick up without reading the full thread.
• Fork early when exploring alternatives. It's better to fork before a divergence than to try to merge two directions later.

Quick summary

Fork, /rewind, /recap, and /btw give you control over long Claude conversations. Fork branches from a point to explore different directions. /rewind removes messages after a point and restores the conversation to that state. /recap summarizes the thread so far. /btw passes a side note without making it a conversation topic. Use them to keep working sessions productive rather than losing control as conversations grow long.

Section 7 — Skills, Plugins & Chat Power Features

Output Styles: Teaching Claude How You Want Things Formatted

Output Styles: Teaching Claude How You Want Things Formatted

Claude has default formatting tendencies: how long responses are, whether it uses bullet points or prose, how formal the language is, whether it adds section headers. Those defaults may or may not match how you work. Output Styles let you change them systematically, so Claude formats things the way you want without you needing to specify it in every prompt.

Why this matters

Inconsistent formatting creates editing friction. If Claude sometimes gives you bullet points and sometimes gives you prose for the same type of task, you're making a formatting decision in every session that you shouldn't have to make. If Claude defaults to long, structured responses when you'd prefer concise prose, you're trimming every output.

Output Styles solve this at the source. You specify your formatting preferences once, they apply globally or per-Project, and Claude formats accordingly.

The idea in plain English

Output Styles are formatting instructions that apply persistently rather than per-prompt. They typically cover:

Length: Do you want concise responses or comprehensive ones? Claude defaults to medium length; you can push it toward shorter (key points only, under 200 words) or longer (thorough, with supporting reasoning).

Structure: Do you prefer prose or lists? Headers or no headers? Tables when comparing options or always prose? Claude uses structure when it seems helpful; Output Styles let you be more prescriptive.

Tone: Professional and formal? Direct and conversational? Conservative and hedged? Confident and assertive? This applies to all responses when set globally or to all responses within a Project when set at Project level.

Voice: First person or third person? Active or passive? Technical vocabulary acceptable or plain language only?

How responses start: Claude sometimes opens with "Certainly!" or similar affirmations. If you find those grating, a style setting can eliminate them.

How this works in Claude

Output Styles can be set in a few places:

Global settings: In Settings, look for a response style or custom instructions option. Instructions you set here apply across all your Claude conversations. This is where you store preferences that should always apply.

Project instructions: For Project-specific formatting, your Project instructions serve as the Output Style for that context. "Always respond in prose, no bullet points. Keep responses under 300 words unless I ask for more" in a Project instruction applies to every conversation in that Project.

Memory: If you tell Claude your formatting preferences in conversation and it saves them to memory, those preferences will apply in future sessions where memory is active.

One-time override in a prompt: A specific prompt instruction always overrides an Output Style for that response. If your global style is "concise" but a specific request needs comprehensive detail, asking for it in the prompt overrides the default for that response only.

Practical example

A legal analyst's main complaint about Claude responses: they're too long, they use too many headers, and the hedging language ("it's worth noting that," "however, it should be considered")

makes every response feel padded.

She sets an Output Style through her global instructions: "Write concisely. Avoid hedging language like 'it's worth noting' and 'however, it should be considered.' Do not use headers unless I ask for a structured document. Write in prose. Aim for under 200 words unless I ask for more."

The change is immediate. Responses are shorter, cleaner, and more direct. She edits far less. The formatting preference she encoded once applies to every conversation without her thinking about it again.

Workflow design notes

Output Styles can conflict with each other if set at multiple layers. A global style that says "no bullet points" and a Project instruction that says "use bullets for action item lists" creates a conflict. Resolve it by being specific: "No bullet points except for explicit action item lists" at the global level, or "use bullet points for action items, prose for analysis" at the Project level.

For teams, shared Output Styles within a Project are powerful for consistency. All team members using the same Project receive the same format from Claude, which reduces the editing burden when outputs need to be reviewed or combined.

Try this in Claude

Look at your recent Claude conversations and identify the formatting element you edit most consistently: you always shorten, you always add headers, you always convert bullets to prose. That's your Output Style priority. Write a two-sentence style instruction that would eliminate most of that editing. Add it to your global settings or your most-used Project and test it in the next three conversations.

Pro tips

- Less is more with Output Styles. One or two clear formatting preferences produce consistent behavior. A long list of formatting rules often produces a Claude that's trying to satisfy too many constraints simultaneously.
- Test style changes with a variety of prompts before accepting them globally. A style that works well for analytical responses may not work for creative ones.
- Output Styles are about format, not content. They don't change what Claude knows or how well it reasons; they change how it presents the output.

Quick summary

Output Styles apply formatting preferences globally or per-Project, so you don't repeat formatting instructions in every prompt. They cover length, structure, tone, and voice. Set them in global

settings for universal preferences, in Project instructions for context-specific preferences. One or two clear instructions work better than a long list. They're the complement to prompting: prompts shape what Claude produces; Output Styles shape how it presents it.

Section 7 — Skills, Plugins & Chat Power Features

Choosing the Right Tool for a Repeated Task

Choosing the Right Tool for a Repeated Task

You have a task you do repeatedly with Claude. Should it be a prompt you write fresh each time? A saved prompt template? A Skill? A Plugin? Part of a Project's standing instructions? The answer depends on a few factors, and getting it right saves setup overhead and produces better results. This lesson is a decision framework for that choice.

Why this matters

Over-engineering a task creates maintenance burden. Under-engineering it creates repetitive friction. A task that should be a simple saved prompt template doesn't need to be a Skill. A task that runs daily in an automated workflow probably does. Knowing which level of infrastructure is right for each task prevents both extremes.

The decision framework

Start by asking five questions about the task:

1. How often do you do this task? - Once ever: no infrastructure needed, just a good prompt - Occasionally (monthly or less): a saved prompt template is sufficient - Regularly (weekly or more): a Skill may be worth the setup - Daily or automated: a Skill or Plugin is almost certainly worth it

2. How consistent is the task across uses? - Highly variable (different inputs, different outputs each time): keep it at the prompt level - Consistently structured (same inputs, same output format): a Skill makes it repeatable without re-specifying - Has a consistent context but variable task (same domain, different specific work each time): a Project instruction may be more appropriate than a Skill

3. Do you share this task with others? - Personal only: prompt template or personal Skill is sufficient - Team shares the task: a shared Skill or Plugin installation makes the workflow consistent across team members

4. Does it involve specialized connectors or knowledge? - Uses standard Claude without external tools: Skill is probably the right layer - Requires connectors (email, CRM, files) and domain knowledge: a Plugin brings all the components together

5. Does it need to run without you present? - You're always in the loop: Skill or prompt is fine - Runs on a schedule without you: a Skill packaged into a Cowork scheduled task (Section 10) is the right architecture

A practical map

Situation	Right tool
One-off task with clear scope	Good prompt, written fresh
Task you repeat but it varies each time	Saved prompt template you modify
Consistent task with consistent output format	Skill
Domain workflow with connectors and knowledge	Plugin
Standing context that applies across all work in one area	Project instruction
Scheduled, unattended automation	Skill + Cowork scheduled task

How to migrate up the stack

Most tasks start at the prompt level and move up as their value and frequency become clear. A reasonable progression:

- You do a task with a good prompt a few times. It works well.
- You save the prompt in a document so you can reuse it without rewriting.
- You do it often enough that finding and copying the prompt feels like friction.
- You turn it into a Skill so you can invoke it with a command.
- You connect it to a Cowork scheduled task if it should run automatically.

Most tasks stay at step 2 or 3. Only the truly frequent or automated ones warrant steps 4 and 5. That's fine. Not everything needs to be a Skill.

Practical example

A marketing manager reviews her Claude usage after three months and finds five categories of tasks she's been doing repeatedly:

- **Weekly blog post draft:** Done every week, consistent format, same inputs. Becomes a Skill.

- **Monthly newsletter:** Done monthly, consistent but not frequent enough to invest in a Skill. Stays as a saved prompt template she modifies monthly.
- **Ad hoc client emails:** Variable every time, no consistent format. Stays at the prompt level.
- **Competitive monitoring:** Should be done weekly but she forgets. The right tool is a Cowork scheduled task that runs automatically and leaves the analysis waiting for her. Uses a Skill within a scheduled automation.
- **Brand voice checking:** Applied to everything going out. Becomes a Project instruction in her "Content" Project, not a Skill, because it applies as a constraint rather than a task.

Five tasks, five different right answers.

Try this in Claude

List the five tasks you've done most frequently with Claude in the past month. For each one, apply the five-question framework. What's the right tool for each? Are any of them currently at the wrong level (over-engineered or under-engineered)? What's the one change that would reduce the most friction in your workflow?

Pro tips

- A good signal that a prompt should become a Skill: you've copied and pasted the same prompt more than three times without modifying it significantly.
- A good signal that a Skill should become a Plugin: you find yourself also manually setting up a connector or loading reference documents before every time you use the Skill.
- Don't build infrastructure anticipating future need. Build it when the current friction is real and the task frequency justifies the setup.

Quick summary

Match the infrastructure level to the task's frequency, consistency, ownership, connector needs, and automation requirements. Most tasks belong at the prompt or saved-template level. Frequent, consistent tasks with clear output formats benefit from Skills. Complex domain workflows benefit from Plugins. Stand-alone context should go into Project instructions. Automate only what's genuinely recurring and well-defined enough to run without you present.

SECTION 08 · BRIDGE

Section 8 of 14

Section 8: Quality Control & Responsible Use

Section 8: Quality Control & Responsible Use

This is the section where the course changes gear. Everything in Act 1 was about reviewing text Claude produced for you. Everything in Act 2 will be about authorizing actions Claude takes on your behalf. This section bridges the two.

Why this section matters

The move from "Claude wrote this email" to "Claude sent this email" is small in mechanics and enormous in stakes. When Claude produces text, you review it, edit it, and decide what to do with it. When Claude takes actions, things happen in the world. Reversing them ranges from mildly inconvenient to impossible.

The review habits you build in this section protect you now (before you're automating anything) and scale into the discipline you'll need in Act 2 (when Claude starts taking real actions with real consequences). This isn't a warning to scare you; it's a genuine preparation for the more capable thing you're about to learn.

What you'll learn

Why AI output needs review. Not as a disclaimer, but as a practical understanding of what Claude's error modes look like and why confident-sounding output is not the same as accurate output.

Checking facts, dates, and assumptions. The specific types of claims that most commonly trip up AI output and the verification habits that catch them before they cause problems.

Spotting hallucinations and unsupported claims. What hallucination actually looks like in practice (it's rarely obvious), and the questions you ask to surface it.

Reviewing connected-tool and automation output. When Claude uses a connector to pull real data, the review calculus changes: the data may be accurate but Claude's interpretation, summary, or action based on that data still needs evaluation.

The stakes shift: reading output vs. authorizing actions. A direct examination of the difference between Claude as a drafting assistant and Claude as an agent that acts. Where the line is, why it matters, and what the implications are for your review process.

Building quality-control checklists. Converting review habits into systematic processes. For high-stakes outputs and automated actions, a checklist is more reliable than a habit.

The mini project

You'll create a quality-control checklist tailored to your own work: one section for Claude-generated text you review before sharing, and one section for actions you'll authorize when you reach Act 2. This checklist becomes a standing resource, not an exercise you complete and forget.

How to approach this section

Read it carefully. Not because it's complicated, but because the habits it describes are often neglected in the enthusiasm of getting things done quickly. Speed is one of Claude's most appealing qualities; it's also the quality that most encourages people to skip review. This section makes the case for why that's a false economy.

By the end, you should have a clear mental model of what can go wrong with AI output and a systematic approach to catching it before it becomes your problem.

Section 8 — Quality Control & Responsible Use

Why AI Output Needs Review (And What Goes Wrong When It Doesn't)

Why AI Output Needs Review (And What Goes Wrong When It Doesn't)

Claude is fast, articulate, and frequently right. It is also capable of being confidently, fluently wrong in ways that are not obvious until something breaks. Understanding the specific mechanisms that produce that wrongness, not as a theoretical concern but as a practical reality, is what makes review a habit rather than a formality.

Why this matters

The value of speed is real. Claude compresses hours of work into minutes. The risk is that the compressed work might contain errors that a slower process would have caught. The people who get burned by AI output are not usually the careless ones; they're the busy ones, moving fast

through a stack of work, who reviewed the output with the part of their brain that was expecting it to be right.

Understanding *why* Claude makes mistakes makes it easier to look for the right things when you review.

The mechanics of AI error

Claude generates text by predicting what would be an appropriate and plausible continuation of the input. This process is powerful for producing coherent, relevant output. It creates three specific failure modes that aren't obvious from the outside:

Hallucination: Claude generates information that is plausible-sounding but fabricated. A statistic it hasn't seen but that fits the pattern. A citation that doesn't exist. A name that sounds right. A date that's close but wrong. Claude doesn't know it's doing this; it's not choosing to deceive. It's filling in the most plausible continuation of the text.

Overconfidence: Claude often presents uncertain information with the same confident tone it uses for certain information. There's no "I'm not sure about this" in the delivery unless you specifically ask. A guess and a fact sound the same.

Outdated knowledge: Claude's training has a cutoff date. Events, personnel changes, policy updates, market conditions, and product features that changed after that date are unknown to Claude unless you supply the current information. Claude may answer based on outdated facts without signaling that the information might be stale.

Misinterpretation: Claude can misread what you're asking for, especially when the brief is ambiguous. The output is responsive to what Claude thought you meant, not necessarily what you actually meant. This error is often invisible until you look at whether the output actually serves your purpose.

Interpolation errors: When reasoning from partial information, Claude fills gaps in ways that seem logical but may not be accurate to the specific situation you're in.

What review is actually for

Review is not about finding whether Claude is right or wrong. It's about applying your judgment and ground truth to Claude's output before it gets used. You know things Claude doesn't: what's current, what's specific to your organization, what the actual situation is, what the recipient will actually need.

Review is the step where your knowledge and Claude's generation capability combine to produce something reliable.

This is true for text you'll send to someone else. It becomes more critical for actions Claude takes on your behalf, because an action with an error in it doesn't sit in a draft waiting to be corrected. It happens.

A taxonomy of AI output by review burden

Output type	Review burden	Why
Internal brainstorm notes	Low	Stakes are low; errors are recoverable
Draft for your own editing	Low-medium	You're the next reviewer
External-facing communication	Medium-high	Errors affect others; trust is at stake
Communication claiming specific facts	High	Factual errors travel
Decisions based on AI analysis	High	Errors compound into downstream decisions
Automated actions taken by Claude	Very high	Irreversibility

Practical example

A professional services firm uses Claude to draft client proposals. The first few go through careful review. As the team gets comfortable, they start lightening the review process, checking format but not substance.

Six months in, a proposal goes out with a market size figure that Claude hallucinated. The figure was plausible, formatted correctly, and appeared in a table that was clearly well-organized. No one checked it against a source. The client noticed. They cited it in a conversation as evidence that the firm didn't do its homework.

The error was fixable. The trust damage was not. The review that would have caught it: one check that asked "where does this number come from?"

Try this in Claude

Ask Claude to write a short analysis of something in your domain, something you know well. Identify three specific claims in the output. For each claim, ask yourself: how would I verify this? If you can't, ask Claude where the claim comes from. Note how often the answer is essentially "I generated this from training data" rather than "this comes from a specific source."

This exercise calibrates how much verification effort your domain typically requires.

Pro tips

- Build review into the process, not after it. The most reliable review happens before the output leaves your hands, not after it's been shared.
- "What are you least confident about in this response?" is a useful prompt after any Claude output involving facts. Claude will often flag its own uncertainty when asked.
- For high-stakes outputs, change the reading mode: read it as the skeptical recipient, not as the author who knows what was meant.

Quick summary

Claude generates plausible, fluent output that can be wrong in ways that aren't obvious: hallucinated facts, overconfident claims, outdated information, and misinterpretation of intent. Review is not about doubting Claude; it's about applying your knowledge and ground truth before output becomes action. The review burden scales with the stakes of the output and the reversibility of any subsequent action.

Section 8 — Quality Control & Responsible Use

Checking Facts, Dates, and Assumptions

Checking Facts, Dates, and Assumptions

Factual verification in AI output is not random. Claude's errors concentrate in predictable places: specific numbers, specific dates, specific names, recent events, and claims that were never stated but are implied. Knowing where to look makes verification faster and more reliable than a general sense of "I should check this."

Why this matters

Checking everything is impractical. Checking nothing is reckless. The skill is in knowing which claims carry the most risk if wrong and focusing verification effort there. This lesson gives you that map.

The high-risk claim types

Specific statistics and figures. Numbers that sound precise are often the most confidently wrong things in Claude's output. Market size estimates, percentage figures, survey results, performance

benchmarks. These feel authoritative because they're specific. They're among the most commonly fabricated. Treat every specific number as a verification priority.

Named sources and citations. Claude will cite studies, reports, books, and articles that don't exist. The title and author sound plausible; the study is invented. Never use a citation from Claude without verifying it against a real source.

Dates and timelines. Event dates, launch dates, milestone dates, and timeline references are frequently wrong. This is especially true for anything in the past few years (approaching Claude's training cutoff) or anything that involves specific scheduling.

Personnel and organizational facts. Who leads which organization, which company acquired which, which person holds which role. These change, and Claude's training data captures a snapshot. Verify before referencing in anything external-facing.

Legal and regulatory claims. Statements about what laws require, what regulations permit, or what compliance means in a specific context. These require professional review, not just fact-checking.

Comparative or superlative claims. "The largest," "the most commonly used," "the leading provider." These are often unverifiable generalizations that Claude presents as fact.

Assumptions: the harder check

Facts can often be verified against sources. Assumptions are harder because they're often invisible. Claude builds assumptions into its reasoning without labeling them as assumptions. Finding and checking them requires a different approach.

Ask Claude directly: "What assumptions is this analysis built on?" or "What would need to be true for this recommendation to be valid?" Claude will surface assumptions it embedded in its reasoning that you wouldn't have noticed otherwise.

Common assumption types to check: - **Market and competitive assumptions:** "The market is growing" or "competitors offer X" may be assumptions rather than verified facts. - **Audience assumptions:** Claude may assume things about your customers, readers, or stakeholders that aren't accurate to your specific situation. - **Context assumptions:** Claude may assume the most common version of a situation you described, when your situation is actually a less common variant. - **Recency assumptions:** Claude may reason as if conditions from its training data are still current.

How this works in practice

Build a lightweight verification step into your review workflow. For any output you'll share externally or act on:

- **Scan for numbers and dates.** Highlight every specific figure and date. Decide which ones you'll verify and against what source.
- **Check any citations.** Search for cited sources before including them. If they don't exist, remove the citation.
- **Ask about assumptions.** For analytical or recommendation outputs, send "what assumptions is this built on?" before finalizing.
- **Flag recent events.** Any claim about something that changed in the last year or two warrants a quick check that Claude's information is current.

Practical example

A business analyst asks Claude to draft a competitive analysis section for a market entry brief. Claude produces a well-structured section with market share figures, a competitor ranking, and a trend summary with a specific growth rate.

Before the brief goes to the client, the analyst runs her verification routine:

- Three market share figures: she finds a source for one, can't verify the other two. She replaces the unverifiable figures with approximate ranges and notes "estimated."
- A competitor described as "the market leader": she checks and finds the landscape shifted after Claude's training cutoff. The "market leader" was acquired. She updates the section.
- A 12.4% annual growth rate: she traces Claude's likely source, finds the study exists but the figure cited was for a different market segment. She corrects it.
- "Assumptions" prompt: Claude reveals it assumed the market is primarily North American, when the client's target is Europe. The entire competitive landscape needs to be reoriented.

The verification took twenty minutes and prevented four errors from reaching the client.

Try this in Claude

Take any Claude output that contains specific claims: figures, dates, named sources, or comparative statements. Apply the four-step verification routine above. Note: how many claims survive verification unchanged? How many need modification? How many need to be removed or replaced?

Pro tips

- For fact-dense outputs, ask Claude to produce the output with inline uncertainty flags: "After each factual claim you're not certain about, add [unverified] in brackets." This speeds your review by pre-marking what needs attention.
- Sources that Claude cites from before 2020 are somewhat more reliable than those from 2022-2024, simply because there's been more time for errors in training data to be identified and corrected. Recency of the cited material increases verification priority.

• For regulatory or legal claims: verify with a professional, not with another AI tool. AI fact-checking AI is not a reliable quality control loop.

Quick summary

Facts, dates, and assumptions require targeted verification, not a general "looks right" review. High-risk areas: specific statistics, named citations, dates, personnel facts, and legal claims. Assumptions require a direct prompt to surface. Build a lightweight four-step verification habit for any output you'll share externally or act upon. Verification takes time but prevents the specific category of mistake that damages credibility and decisions.

Section 8 — Quality Control & Responsible Use

Spotting Hallucinations and Unsupported Claims

Spotting Hallucinations and Unsupported Claims

The word "hallucination" has become common in AI discussions, but it's often misunderstood. It doesn't mean Claude is making random errors or malfunctioning. It means Claude is generating plausible-sounding content that is not grounded in verified facts. Learning to spot it is a skill, not an instinct.

Why this matters

Hallucinations are dangerous precisely because they don't look different from accurate information. A fabricated statistic is formatted the same way as a real one. A non-existent study is cited with the same confidence as a real study. The output reads fluently because it was generated to be fluent, not because it's correct.

The practical consequence: you can't rely on "this feels wrong" to catch AI hallucinations. You need a more systematic approach.

What hallucination looks like in professional output

Fabricated specifics with high confidence. "According to a 2023 McKinsey report, 67% of procurement leaders cited..." If you can't find the specific report or the specific figure, assume it's fabricated. Consulting firm reports are a commonly hallucinated category because they're frequently cited in Claude's training data.

Plausible but wrong personnel or organization details. "The CEO of [company] stated..." where the CEO either isn't the right person or the statement can't be verified. Organizational facts change; Claude's training data may reflect a past state.

Accurate-seeming historical details that are subtly wrong. Dates that are off by a year or two. Events attributed to the wrong person. Cause-and-effect relationships that are inverted from what actually happened. These are particularly hard to spot because they're close to correct.

Implicit claims that go further than the evidence. Claude might accurately describe a study's findings but then draw a conclusion from it that the study didn't actually support. The citation is real; the inference is fabricated.

Confident answers to questions Claude can't actually answer. If you ask Claude about something that's highly specific to your context, your company, or recent events, and Claude gives you a detailed answer, be suspicious. It's working from general patterns, not knowledge of your situation.

The verification question

For any specific factual claim that matters, ask yourself: can I point to a source for this? Not "does this seem like it could be true?" but "where would I go to verify this independently?"

If there's no source you could check: - The claim may be true but unverifiable through Claude (it exists in the world but not in a source you can access quickly) - The claim may be Claude's reasonable inference from related information - The claim may be fabricated

All three produce the same-looking output. The verification question distinguishes them.

High-hallucination contexts

Some request types produce higher hallucination rates. Be more vigilant when Claude is:

- **Generating quantitative data** about specific markets, industries, or organizations
- **Citing academic, consulting, or research sources** for specific claims
- **Writing about recent events or developments** (past two years)
- **Describing specifics of technical standards, regulations, or legal frameworks**
- **Filling in details** about your specific organization, clients, or situation that you didn't provide

Lower-hallucination contexts: summarizing a document you provided, reformatting content you gave it, reasoning through a problem with explicit input you supplied, writing creative content that doesn't claim factual accuracy.

The unsupported-claim variant

Hallucination is generating false content. Unsupported claims are a related but distinct problem: Claude makes a claim that may be true but that is not supported by the information provided.

In analytical or recommendation outputs, Claude sometimes draws conclusions that go beyond the evidence. "Based on this data, the market is clearly moving toward X" when the data shows a trend but doesn't clearly establish direction. "This approach is superior" when the comparison wasn't made on all the relevant dimensions.

Unsupported claims are caught by asking: "What in the information I provided (or that you know verifiably) supports this conclusion?" If Claude can't point to specific support, the claim is an extrapolation that should be labeled as such.

Practical example

A communications consultant asks Claude to write a background section for a client pitch, providing some context about the client's industry. Claude returns a well-written section that includes: - A claim about the industry's revenue growth rate: sounds right but specific enough to require checking - A reference to "a growing body of research" showing X: vague generalization, not a claim - A citation: "according to Gartner's 2024 Industry Forecast": needs verification; may not exist - A statement about competitive dynamics: plausible inference from the context she provided

She applies verification: the growth rate figure traces to a real report with a slightly different number (she adjusts). The "growing body of research" she converts to a more specific claim or removes. The Gartner citation doesn't exist in the form Claude described (she removes it). The competitive dynamics statement she labels as her analysis rather than a factual claim.

The section that goes to the client is accurate and appropriately scoped. The version Claude produced was not.

Try this in Claude

Ask Claude to write a brief about a topic in your field that includes specific facts and statistics. Then ask: "For each factual claim in what you just wrote, tell me: how confident are you and what's your source?" Claude's answer will distinguish between claims it's grounding in training data, claims it's inferring, and claims it's genuinely uncertain about. This calibrates your verification priorities.

Pro tips

- Treat specific numbers as the highest-risk element in any AI-generated analysis. Always spot-check at least two or three figures.

• If Claude cites an organization or study you've never heard of, search for it before relying on it.
• Ask Claude to "flag any claims in this output that are based on general patterns rather than specific verifiable sources." Claude will often identify its own hallucination risk areas when asked.
• For documents going to external stakeholders, consider a policy: all statistics require a verifiable source before the document is shared. This forces verification without having to decide case by case.

Quick summary

Hallucinations are plausible-sounding content Claude generates without factual grounding. They look identical to accurate content. High-risk contexts include specific data, citations, recent events, and details about your specific situation. Unsupported claims are a related problem: conclusions that go beyond the evidence. The core verification habit: for any specific claim that matters, identify the source that would verify it. If there isn't one, the claim is either unverifiable or fabricated.

Section 8 — Quality Control & Responsible Use

Reviewing Connected-Tool and Automation Output

Reviewing Connected-Tool and Automation Output

When Claude reads your email and summarizes it, the situation is different from when Claude writes an essay from training data. The data it's working with is real and current. But Claude's interpretation, organization, and conclusions about that data still involve the same generative process. The review you need is different in character, not in importance.

Why this matters

A common mistake when deploying connected-tool workflows: people review Claude's analysis of their real data less carefully than they review its free-form writing, because "Claude is reading my actual calendar so it must be right." This confuses data accuracy with analytical accuracy. The data might be real; the interpretation might still be wrong.

Understanding the specific failure modes of connected-tool output prevents the category of mistake that comes from trusting the pipeline more than the output.

Three types of connected-tool output

Retrieval output: Claude searched or retrieved information from a connected source and is presenting it. Example: "You have three emails awaiting a response from last week."

Analytical output: Claude has read real data and drawn conclusions from it. Example: "Your meeting load has increased 40% over the past month, concentrated on Wednesday afternoons."

Action output: Claude has taken an action using a connected tool. Example: "I've drafted a reply to Sarah Chen's email and scheduled it for 9am." (With write-enabled connectors.)

Each type requires different review.

Reviewing retrieval output

Check completeness, not just content. If Claude searched your email and found three pending replies, ask: are there more? A connector might miss messages in certain folders, messages that arrived after the search window, or messages that match the criteria but aren't labeled in the expected way.

Verify against the source for high-stakes retrieval. If Claude says "your next open calendar slot is Thursday at 2pm," glance at your actual calendar. Connector queries can have edge cases in how they handle time zones, recurring events, or recently updated items.

Be skeptical of negatives. "There are no emails about X in your inbox" is hard to verify. It may be true; it may mean the search didn't surface them. For important retrieval, consider doing the search yourself as well.

Reviewing analytical output

Check the reasoning, not just the conclusion. If Claude analyzes your calendar data and concludes that your most productive days are Fridays, ask why. Does the underlying data actually support that conclusion? Is it measuring the right things?

Watch for interpretation bias. Claude may apply a frame to your data that seems reasonable but doesn't match your actual situation. "Your email response time averages 4 hours" might be accurate, but if it's counting emails you intentionally ignore, the interpretation is off.

Ask what data was excluded. Analytical conclusions depend on what's in the data set and what's not. "Emails about X increased 20% this month" might be accurate if you count all emails, but misleading if you count only external emails. Asking Claude to describe the data set it used prevents misleading conclusions.

Reviewing action output

Action output (covered more in Section 12, Computer Use) requires the most careful review because the action has already happened or is about to happen.

For write-enabled connector actions: Claude might draft and queue an email, create a calendar event, or update a document. Review before confirming. A "confirm before acting" discipline, where Claude drafts but you send, is the right posture for most people until they have very high confidence in a specific workflow.

For actions that affect others: Anything that sends a communication, creates an external record, or modifies shared resources requires review. Errors in these actions have external consequences.

For irreversible actions: Some actions can't be undone. Deleting records, sending external communications, making financial transactions. These should have a hard review gate.

Practical example

A manager sets up a Cowork automation that reviews her inbox every morning and summarizes urgent emails needing a response. After a week, she notices the summary consistently misses emails from her most important client because they come in with a subject line she doesn't recognize as the client.

The automation's retrieval is accurate for the emails it finds. It has a completeness problem: a structural gap in how the connector identifies "urgent." She adds the client's email domain as an additional criterion. The gap is caught not because the automation output was obviously wrong, but because she was consistently checking the actual inbox against the summary.

That checking habit is the review mechanism for retrieval output. Once she trusts the pattern, she can relax it. Until then, she maintains it.

Try this in Claude

Set up a simple connected-tool query (email search, calendar review) and compare the result against the source directly. Find one item the retrieval missed or one conclusion the analysis drew that you'd characterize differently. Understanding the failure mode of your specific connector setup is more useful than theoretical knowledge of what can go wrong.

Pro tips

- For automated summaries (morning digest, weekly report), spot-check them against the source regularly for the first few weeks. This builds confidence in the workflow and catches systematic gaps before you stop checking.
- Ask Claude to describe its data set before presenting analysis: "What data did you use to reach this conclusion?" This forces transparency about scope and helps you catch exclusions.

• Establish a "confirm before acting" rule for any write-enabled connector, especially early in a new workflow's life.

Quick summary

Connected-tool output has real data underneath, but Claude's interpretation and conclusions still require review. Retrieval output needs completeness checks. Analytical output needs reasoning checks. Action output needs a gate before confirmation, especially for irreversible actions. The review character changes (you're checking interpretation rather than fabrication), but the discipline of checking remains essential.

Section 8 — Quality Control & Responsible Use

The Stakes Shift: Reading Output vs. Authorizing Actions

The Stakes Shift: Reading Output vs. Authorizing Actions

There is a clean line in how you use Claude that this section is designed to make visible before you cross it. On one side: Claude produces things you read, edit, and decide what to do with. On the other side: Claude takes actions in the world on your behalf. These are not just different in degree. They're different in kind.

Why this matters

When Claude writes an email draft, you read it. If it's wrong, you fix it or discard it. Nothing happened yet. When Claude sends an email, something happened. The recipient received it, the message is in the record, and reversing it ranges from awkward to impossible.

Every workflow in Act 2 (Cowork) involves Claude acting: reading files, creating documents, sending communications, scheduling events, running scheduled routines. The people who use these workflows most successfully are the ones who internalized this distinction before they started building. The ones who get surprised are the ones who treated "Claude does it" the same as "Claude writes it for me."

The distinction in plain terms

Output you read: - Claude produces text, code, a visual, or an analysis - You read it; you decide what to do - If there's an error, you haven't acted yet - Review is about improving before use

Actions Claude takes: - Claude performs an operation: sends, creates, modifies, schedules, triggers - The operation happens (or is queued to happen) - If there's an error, it's already in the world - Review must happen before the action, not after

This isn't a reason to avoid using Claude for actions. Agentic workflows are valuable exactly because they compress multi-step tasks into things that happen without your constant attention. It's a reason to design those workflows with the right gates and review steps built in.

The reversibility spectrum

Not all actions are equally irreversible. A useful mental model:

Action type	Reversibility	Required gate
Creating a draft	Fully reversible	Light or none
Creating a document or file	Easily reversible	Brief check
Sending a scheduled task result to your own review	Reversible	Review before next step
Updating a shared document	Partially reversible	Review before confirming
Sending an external email	Hard to reverse	Explicit approval
Deleting files or records	Very hard to reverse	Hard gate, explicit confirmation
Making an external API call with effects	Varies	Design-time risk assessment

The appropriate review gate is proportional to the irreversibility. Fully reversible actions can have light review. Irreversible actions should have hard gates.

Designing for the right level of human oversight

The goal in Act 2 is not to review everything (which eliminates the value of automation) or to review nothing (which eliminates the safety of oversight). It's to design each workflow with the review that's proportionate to the stakes.

Patterns for appropriate oversight:

Draft-then-review: Claude creates the output (email draft, document, summary) and leaves it for you to review and send or publish. You do the confirming step. This works for most communication and document workflows.

Act-then-notify: Claude takes the action (creates a calendar event, updates a file) and notifies you. You can undo if needed. Works for reversible actions where the overhead of pre-approval isn't worth it.

Act-within-limits: Claude takes action but only within defined constraints (only reply to emails marked [urgent], only schedule in the next 24 hours). The constraints are the safety gate.

Explicit-approval-required: Claude queues an action and waits for your confirmation. For sensitive or irreversible actions, this is the right posture until you have very high confidence in the workflow.

Where this course is going

The Cowork sections introduce all of these patterns in practice. Scheduled tasks produce output you review before acting on. Dispatch lets you authorize tasks from your phone, which means your phone tap is the confirmation gate. Computer use, the most powerful and least reversible option, has the most explicit review requirements.

What you're building in this section, the habit of thinking clearly about what review is needed and for what kind of action, is the mental model you'll apply throughout Act 2.

Try this in Claude

Think about one task you're considering automating with Claude in Act 2. For that task: 1. What action does Claude take? 2. Is the action reversible? 3. What could go wrong? 4. What review gate would catch that error before it matters?

Write those four answers. They're the basis of the review design for that workflow.

Pro tips

- "What's the worst thing that could happen if Claude gets this wrong?" is a useful question for every action workflow you design. It helps you identify whether your review gate is proportionate to the actual risk.
- Build review steps into automation from the start, not as an afterthought. Adding a gate after an automation is running is harder than designing it in.
- For workflows that involve external communications, start with "draft and notify me" posture and only move to "send directly" after the draft quality has been consistently high across many runs.

Quick summary

Reading Claude's output and authorizing Claude's actions are fundamentally different activities. Output can be reviewed and discarded if wrong. Actions have already happened when you find the error. Design every agentic workflow with review gates proportionate to the irreversibility of the actions involved. Draft-then-review is the default posture for most communication workflows; harder gates are appropriate for irreversible or high-stakes actions.

Section 8 — Quality Control & Responsible Use

Building Quality Control Checklists

Building Quality Control Checklists

A habit is what you do when you remember to. A checklist is what you do when you might not. For AI output that matters, the difference is significant. This lesson is about converting review habits into systematic processes that are reliable regardless of how busy you are.

Why this matters

Review disciplines erode under pressure. When deadlines are close and work is moving fast, the review steps that seemed important in a training environment are the first things to compress. A checklist that's built into the workflow rather than relying on memory is more likely to survive real conditions than a well-intentioned habit.

This is doubly true for automated workflows. If a Cowork automation runs every morning, there's no "I was in a hurry" explanation for missing the review. Either the review gate is built in or it isn't.

What belongs in a quality-control checklist

A good QC checklist covers the specific things that most commonly go wrong with your type of output. It's not a generic "review everything" instruction; it's a targeted list of failure modes to check for.

For **text outputs** (drafts, analyses, summaries): - Are specific facts (numbers, dates, names, citations) verified? - Does the output answer the actual question, not a simpler adjacent question? - Does the tone match the intended audience and relationship? - Are there any claims that would embarrass you if questioned? - Is there anything in this output that only seems right because you want it to be?

For **connected-tool outputs** (email summaries, calendar analysis, data retrieval): - Is the retrieval complete, or might there be items it missed? - Does the analysis reflect the full picture, or just the

most visible pattern? - Have I checked the source for the most important items, not just trusted the summary?

For **automated actions** (scheduled tasks, automated communications, file operations): - Is the scope of the action limited to what was intended? - Is there a draft-review step before anything irreversible happens? - Who is notified if something goes wrong? - When did I last spot-check this workflow against the source?

Building a personal checklist

The best checklist is one built from your own review history: the specific things you've caught after AI generation that needed fixing. Start by thinking about the last five times Claude produced something you had to significantly edit or correct. What was wrong each time? That's your checklist.

A template structure:

```
Before sharing externally:
■ Verified all specific figures against a source
■ Checked any citations actually exist
■ Read as the recipient: is the tone and message right?
■ Confirmed nothing in this output would embarrass me if questioned

Before publishing or distributing widely:
■ All of the above
■ A colleague or second reader has reviewed
■ No claims that require professional review (legal, financial) without that
review

Before authorizing an automated action:
■ The action scope is exactly what I intended
■ I've reviewed the draft/output that will be acted upon
■ I've considered what happens if this is wrong
■ There is a way to undo or recover if needed
```

Customize this to your work. Add items for the specific failure modes you've encountered. Remove items that don't apply.

Checklists for teams

If you're building QC processes for a team using Claude, the checklist serves two functions: a quality gate and a training document. A new team member following the checklist builds review habits faster than one relying on informal guidance.

For team checklists: - Make them specific to the output types the team produces (not generic) - Include examples of what "passes" and what "fails" each check - Assign ownership: who signs off

on what - Review and update the checklist as the team encounters new failure modes

Practical example

A content team produces ten external blog posts per month using Claude for drafts. They implement a three-step QC process:

Writer checklist (before submission for review): - All statistics verified with a source link in the document comments - All company/product facts checked against the current website - The post actually answers the question in the headline (not a related but easier question)

Editor checklist (before publishing): - Tone appropriate for audience and channel - No implied expertise claims the company can't stand behind - A colleague not involved in the writing has read it

Post-publish tracking: - Any corrections needed within 48 hours are logged - Patterns in corrections are reviewed monthly and added to the writer checklist

Three months after implementation, the correction rate drops significantly. The checklists caught the specific things that kept slipping through informal review.

Try this in Claude

Build your personal QC checklist right now. Three columns, three to five items each: text outputs, connected-tool outputs, and automated actions. Populate the first column from your own review history; add items from this lesson for the second and third columns. Keep it in a place you'll actually consult it: in your main Claude Project instructions, in a document you can access quickly, or as a saved note.

Pro tips

- Checklists that are too long don't get used. If your checklist has more than seven items per category, it's a policy document, not a checklist. Tighten it to the things that matter most.
- Review checklists quarterly. The failure modes that mattered six months ago may not be the ones that matter now as your workflow evolves.
- For automated workflows: include the checklist review as part of the automation monitoring, not as a separate task. "Review the output of this weekly report against the QC checklist" should be on your calendar, not just your intentions.

Quick summary

Quality-control checklists convert review habits into reliable processes. Build yours from your own failure history: the specific things that have gone wrong with AI output in your context. Separate

checklists for text output, connected-tool output, and automated actions address different failure modes. Keep them short enough to actually use, specific enough to be useful, and reviewed periodically as your workflows evolve. For teams, checklists serve double duty as quality gates and training tools.

SECTION 09 · ACT 2 — COWORK

Section 9 of 14

Section 9: Meet Cowork — The Agentic Desktop

Section 9: Meet Cowork — The Agentic Desktop

Welcome to Act 2. You've spent eight sections building skill with Claude as a chat assistant: prompting, organizing Projects, working with documents, using connectors interactively. Cowork extends that foundation into something qualitatively different: a desktop environment where Claude can access your files, run on a schedule, receive instructions from your phone, and take actions in applications on your computer.

Why this section matters

The word "agentic" means Claude acts, not just responds. It reads your actual files. It writes to your actual folders. It can connect to your applications and operate them. The same care you bring to any tool that touches your real data and your real work applies here, with higher stakes because the actions have effects.

This section establishes what Cowork is, how it's structured, and what you need to understand before you start building workflows in it. Skipping this section and going straight to automation is the Cowork equivalent of not reading the setup guide before connecting something to your network.

What you'll learn

What Cowork is and how it differs from Chat. Cowork is the desktop app's agentic mode. The key additions over Chat: file system access, the ability to run on a schedule, Dispatch (remote triggering from mobile), and computer use. The Claude model is the same; the surface is more powerful.

The three-tab desktop app: Chat, Cowork, and Code. How the desktop app is organized and what each tab provides. Understanding the surface prevents confusion about which mode you're operating in.

File-system access and the permission dialogs you grant on setup. Cowork can read and write files in folders you authorize. You grant this access through permission dialogs. What you authorize, when, and how to manage those authorizations is covered here.

Global instructions vs. per-folder instructions. Cowork supports two levels of standing instructions: instructions that apply across all Cowork work, and instructions specific to a folder or

workspace. Knowing which to use for which type of guidance prevents conflicts.

Connectors and plugins inside Cowork. The same connectors you configured in Act 1 work in Cowork. Plugins extend both Chat and Cowork capabilities. The configuration is largely shared.

Cowork instructions vs. Project instructions. Projects (from Section 5) and Cowork workspaces are related but different. Understanding the relationship prevents double-configuration and inconsistency.

Review mode vs. action mode. Cowork has settings for how autonomously Claude operates. Review mode keeps you in the loop; action mode lets Claude proceed without explicit confirmation. The right setting depends on the workflow and your confidence in it.

The mini project

You'll set up a functional Cowork workspace: authorize folder access, write global and folder-level instructions, and connect the tools you'll use in the sections ahead. The deliverable is a configured workspace, not just knowledge of the concepts.

How to approach this section

Cowork setup is done in the desktop app, not in a browser. If you haven't installed it, that's the first step. The setup dialogs walk you through folder authorization and connector configuration with permission prompts. Read them carefully; they describe exactly what access you're granting.

By the end of this section, you'll have a working Cowork environment and the conceptual foundation for everything in sections 10 through 14.

Section 9 — Meet Cowork: The Agentic Desktop

What Cowork Is and How It Differs from Chat

What Cowork Is and How It Differs from Chat

Cowork and Chat both use Claude. The similarity ends roughly there. Cowork is a desktop agentic environment designed for Claude to do work on your machine, not just produce text in a window. If Chat is Claude as a knowledgeable assistant who advises you, Cowork is Claude as an assistant who can walk over to your desk, open your files, and do the work.

Why this matters

The mental model you use for Chat will mislead you in Cowork. In Chat, nothing happens until you decide to act on Claude's output. In Cowork, Claude can act. Understanding what that means, practically and consequentially, is the foundation for using Cowork well.

The core differences

File system access. In Chat, you bring files to Claude by uploading them. In Cowork, Claude can access files in folders you've authorized, read them without uploading, write new files, and update existing ones. Your machine's file system is part of Claude's working environment.

Scheduled operations. Chat is interactive: you ask, Claude responds. Cowork can run on a schedule without you being present. You configure a task to run every morning, and it runs every morning, accessing your files and connectors, producing output or taking actions, whether or not you're at your desk.

Dispatch. Cowork accepts instructions from your phone via the Claude mobile app. You send a task from your phone; Cowork executes it on your desktop. This is covered in Section 11.

Computer use. Cowork can control applications on your computer: move the mouse, click buttons, type text, navigate interfaces. This is the most powerful and most carefully governed feature. Covered in Section 12.

Action-taking. Cowork workflows can take actions with real effects: create documents, send emails (with write-enabled connectors), update records, trigger processes. These actions happen whether or not you review them before they execute, depending on how you've configured the workflow.

What stays the same

The Claude model is the same. Your prompting skills, your knowledge of Projects, Artifacts, Skills, and Plugins, all apply in Cowork the same way they do in Chat. The connector infrastructure you configured in Act 1 is available in Cowork without reconfiguration. The quality-control habits from Section 8 apply here with higher importance.

Cowork is not a different Claude. It's the same Claude with a broader set of capabilities and a more consequential operating environment.

The trust calibration question

In Chat, you choose what to do with what Claude produces. In Cowork, you configure what Claude does and then (often) it does it. The appropriate level of trust to extend to Cowork workflows is the central design question for anyone building them.

The answer depends on: - How well-defined the task is (clear scope = higher trust) - How reversible the actions are (reversible = higher trust) - How well you understand the workflow's failure modes (well-understood = higher trust) - How long the workflow has been running reliably (proven track record = higher trust)

Start with less autonomy. Add more as you verify that the workflow behaves as intended.

How this works in Claude

Cowork appears as a tab in the Claude desktop app. When you open Cowork, you're in the agentic environment. The interface is similar to Chat (a conversation area, a sidebar) but with additional elements: folder navigation, workflow configuration, a scheduled tasks view.

The key setup step before any of Cowork's additional capabilities work: authorizing folder access and granting the necessary permissions. The app walks you through this when you first open Cowork.

Practical example

A freelance consultant uses Chat for all her writing, analysis, and communication work. She runs a monthly client report that involves reading notes from five client folders, pulling in calendar data, and producing a progress report for each client.

Before Cowork: she opens each folder, copies relevant content into Claude, asks Claude to produce the report, reviews and sends. About four hours of work.

After Cowork: she builds a workflow that reads her notes folders directly, accesses her calendar connector, produces a draft report for each client, and puts the drafts in a review folder. She reviews and sends. About forty-five minutes.

The work is the same. The workflow is different. Cowork handles the mechanical parts; she handles the judgment parts.

Try this in Claude

Open the Cowork tab in the desktop app. Before touching any settings, explore the interface: where does folder authorization happen? Where are scheduled tasks configured? Where does Dispatch appear? Getting familiar with the surface before configuring it prevents setup mistakes.

Pro tips

- Think of Cowork as earning trust, not assuming it. Start with read-only folder access, then add write access for specific folders when a specific workflow needs it.

• The most useful first Cowork workflow is typically a simple one: read a folder you know well and produce a summary. This verifies folder access and Claude's behavior with your actual files before you build anything more complex.

Quick summary

Cowork is the desktop agentic mode of Claude: same model, broader capabilities. File system access, scheduled operations, Dispatch, and computer use distinguish it from Chat. Actions in Cowork have real effects. The right mental model is Claude as an assistant who can act on your machine, not just advise you. Start with limited scope and expand as you verify workflows behave as intended.

Section 9 — Meet Cowork: The Agentic Desktop

The Three-Tab Desktop App: Chat, Cowork, and Code

The Three-Tab Desktop App: Chat, Cowork, and Code

The Claude desktop app organizes three distinct surfaces into one interface. Understanding what each tab does, and what mode you're in at any given moment, prevents the confusion that comes from expecting Chat behavior in a Cowork context or vice versa.

Why this matters

Each tab operates differently. The same prompt can produce different results depending on which tab you're in, because the context, capabilities, and permissions available to Claude vary by surface. Knowing which tab to be in for which type of work is a practical navigation skill.

The three surfaces

Chat: The conversational AI assistant. Everything covered in Act 1 of this course lives here. Text generation, Projects, Artifacts, connectors for interactive queries, Skills and Plugins. Chat doesn't have access to your file system by default and doesn't run on a schedule.

Cowork: The agentic desktop environment. File system access, scheduled tasks, Dispatch, computer use. This is where you build and run automated workflows. The conversation interface looks similar to Chat, but the underlying capability set is different.

Code: A Claude Code terminal interface for software developers. Command-line-style interaction for code generation, debugging, and repository work. If you're not a software developer, this tab is mostly out of scope for your work. The course covers it briefly in Section 14 as a pointer for learners who want to go further.

Moving between tabs

Each tab maintains its own state: your active conversations in Chat, your configured workflows in Cowork, your code sessions in Code. Switching between tabs doesn't break anything; you return to the same state when you come back.

One practical consequence: if you start a conversation in Chat and realize you needed the file access of Cowork (or vice versa), you typically start over in the right tab rather than migrating the conversation. Take note of which tab you're in before a long working session.

The sidebar is shared

The left sidebar (conversation history, Projects, Skills) is largely shared across tabs, though some elements are tab-specific. Your Claude Projects and conversation history are accessible from both Chat and Cowork. Folder configurations and scheduled tasks are specific to Cowork.

Try this in Claude

Open the desktop app and click through all three tabs. In each tab, note: what's different in the main interface? What options or sections appear that don't appear in the other tabs? This orientation exercise takes five minutes and removes navigation uncertainty for the rest of the Cowork sections.

Quick summary

Chat is the conversational assistant surface. Cowork is the agentic desktop surface with file access and automation. Code is the developer terminal interface. Be in the right tab for your task: Chat for interactive text work, Cowork for file-connected and automated workflows. The sidebar is largely shared; workflow configurations are tab-specific.

Section 9 — Meet Cowork: The Agentic Desktop

File System Access and the Permission Dialogs You Grant on Setup

File System Access and the Permission Dialogs You Grant on Setup

Cowork can read and write files on your computer. That capability powers everything in Act 2 that involves your actual documents, notes, and data. It also means you're extending real, consequential access to an AI system. The permission dialogs you see during setup are not formalities. They describe exactly what you're authorizing.

Why this matters

Most professionals are accustomed to granting app permissions quickly: accept, accept, accept, get to the interface. That habit can lead to overly broad access in Cowork. Understanding what you're granting, at what scope, for what purpose, gives you the control you need to use Cowork securely and confidently.

How folder authorization works

When you first set up Cowork, the app asks you to choose one or more folders that Claude can access. You select a folder from your file system, and the permission dialog describes what access you're granting:

Read access: Claude can read files in this folder. It cannot create, edit, or delete them.

Read and write access: Claude can read, create, and modify files in this folder. Depending on the setting, it may also be able to delete.

Access is folder-scoped. Granting access to a specific folder does not grant access to other folders on your machine. If you authorize your "Work Projects" folder, Claude can access files within it but not files in your Documents folder more broadly.

You can grant access to multiple folders, each with its own permission level. You can also revoke access at any time through the Cowork settings.

A deliberate approach to folder authorization

Rather than granting broad access to accommodate potential future workflows, authorize specific folders for specific purposes:

- A "Cowork Inbox" folder where you put files you want Claude to process
- A "Cowork Outputs" folder where Claude deposits its outputs for your review
- Specific project folders where Claude needs access for defined workflows

This creates a clear boundary between what Claude can touch and what it can't. If you need Claude to access a new folder for a new workflow, you authorize it when you need it.

What the permission dialogs tell you

The setup dialogs will describe: - Which folder is being authorized - What level of access is being granted (read vs. read/write) - Whether delete access is included - Whether subdirectories are included in the scope

Read all of this before confirming. If the scope is broader than your workflow requires, you can often narrow it.

After authorization

Once access is granted, Claude can work with files in authorized folders whenever a Cowork workflow (interactive conversation, scheduled task, or Dispatch command) references those files. It doesn't access them continuously; it accesses them when a workflow runs.

Review your authorized folders periodically (quarterly is a reasonable cadence) and revoke access to folders that are no longer part of active workflows.

Try this in Claude

Before authorizing any folders, decide what you actually need: what workflow will use these files, what folder contains them, and what level of access (read or read/write) is necessary. Write this down. Then authorize accordingly. You'll have a clear record of what you granted and why.

Quick summary

Cowork's file system access requires explicit folder authorization through permission dialogs. Grant access to specific folders for specific purposes rather than broadly. Read the permission dialog carefully: what folder, what access level, what scope. Revoke access when workflows that used it are retired. Start with read access and add write access when a specific workflow requires it.

Section 9 — Meet Cowork: The Agentic Desktop

Global Instructions vs. Per-Folder Instructions

Global Instructions vs. Per-Folder Instructions

Cowork supports two levels of standing instructions: global instructions that apply to all Cowork work, and per-folder instructions that apply to work within a specific folder or workspace. Getting

this right means Claude always has the right context without you having to specify it in every prompt.

Why this matters

Without standing instructions, every Cowork workflow starts cold. You either include all the necessary context in your prompt (inefficient) or Claude makes reasonable-but-wrong assumptions about how to approach the work (risky). The instruction layers eliminate both problems.

Global instructions

Global instructions apply to every Cowork interaction, regardless of which folder you're working in or what task you're running. They're the things you always want Claude to know and always want Claude to do in your Cowork environment.

Good candidates for global instructions: - Your name, role, and organization (general context Claude should always have) - Your review and approval requirements ("always create a draft for my review before taking any action that affects an external party") - Standing safety constraints ("do not delete files under any circumstances without explicit confirmation") - Output format preferences that should apply everywhere - Any hard limits on what Claude can do autonomously

Global instructions are the guardrails for your entire Cowork environment. Err on the side of more safety, less autonomy at this level.

Per-folder instructions

Per-folder instructions apply to work within a specific folder. They're context and constraints relevant to the work in that workspace but not relevant everywhere.

Good candidates for per-folder instructions: - What the folder contains and what its purpose is - Specific output formats or naming conventions for files produced in this workspace - Context about the project or client this folder relates to - Any constraints specific to this type of work - How Claude should handle edge cases in this specific context

Per-folder instructions override global instructions only for the specific folder they're set for. They supplement rather than replace global instructions.

A practical configuration pattern

Global instructions (short, safety-focused):

> *I'm a [role] at [organization]. In all Cowork work, always create drafts before sending anything external. Always notify me of any action taken. Do not delete files without explicit confirmation.*

> *Output files should be clearly named with date and type.*

Per-folder instructions (specific, context-rich):

> *This folder contains client deliverables for [client name]. Work here is confidential. Documents follow [style guide] format. Output files should go in the /drafts subfolder. Final approvals require my explicit confirmation before delivery.*

Try this in Claude

Draft your global instructions before touching any Cowork workflows. What are the three things that should always be true about how Claude operates in your Cowork environment? Write those as global instructions. Then think about your first intended Cowork folder: what context and constraints are specific to it? Write those as per-folder instructions.

Quick summary

Global instructions set universal guardrails for all Cowork work. Per-folder instructions add specific context and constraints for individual workspaces. Configure global instructions first, keep them safety-focused, and add per-folder instructions as you build out specific workflows. The combination gives Claude the context to work effectively without you repeating it in every prompt.

Section 9 — Meet Cowork: The Agentic Desktop

Connectors and Plugins Inside Cowork

Connectors and Plugins Inside Cowork

The connectors and plugins you configured in Act 1 work in Cowork without re-configuration. But how they're used changes: in Chat, you invoke them interactively. In Cowork, they can be accessed by scheduled tasks and Dispatch commands without you being present. This lesson maps out what that means in practice.

Why this matters

A connector that works well in interactive Chat use might behave differently in an automated Cowork context. The same connector that accurately retrieves your inbox when you're watching can sometimes behave unexpectedly when it runs at 6am on a schedule. Understanding how connectors work in automation contexts prevents gaps between what you expect and what happens.

Connectors in automated workflows

When a Cowork scheduled task or Dispatch command uses a connector, it accesses the connector with the permissions you granted during setup. The connector reaches its source (email, calendar, CRM), retrieves or writes data, and returns results to Claude.

Key considerations:

Permissions scope: The connector uses the permissions you granted at setup time. If your email connector has read-only access, it can read but not send, even in an automated context.

Authentication: Connectors authenticate with their source services. If a service requires periodic re-authentication (token refresh, re-authorization), an automated task that runs when authentication has expired will fail. Check for re-authentication requirements when setting up automation.

Rate limits: Some services limit how often their APIs can be called. Very frequent automated tasks that use connectors may hit rate limits. Most professional workflows don't encounter this, but it's worth knowing.

Data freshness: Connectors pull current data when invoked. A scheduled task that runs at 7am retrieves inbox state as of 7am. If you're checking the output at 9am, it reflects 7am state, not 9am.

Plugins in Cowork

Plugins installed in Chat are available in Cowork. Plugin Skills appear in the same slash-command interface. Plugin connectors work the same way.

One practical distinction: some Plugins are designed for interactive Chat use (respond to a query, produce output in the conversation). Others are designed for workflow use (process a document, update a record, run a standing analysis). In Cowork automation, you generally want the latter type.

When selecting Plugins for Cowork automation, check whether the Plugin documentation describes automated or workflow use cases, not just interactive Chat use.

Try this in Claude

Open your Cowork connector settings and verify that the connectors you configured in Act 1 are visible and authorized. For any connector you plan to use in automated workflows, check: does it have the right permission scope? Does it require periodic re-authentication? This verification takes five minutes and prevents the silent failure of a workflow that can't access its data source.

Quick summary

Act 1 connectors work in Cowork without re-configuration. In automated contexts, check permission scope, authentication status, and any rate limits. Plugins available in Chat are available in Cowork; prefer Plugins designed for workflow use in automation contexts. Verify connector authorization before building automated workflows that depend on them.

Cowork Instructions vs. Project Instructions

You now have two types of standing instructions available: Project instructions (from Section 5) and Cowork instructions (global and per-folder, from this section). They serve different purposes and live in different places. Understanding the relationship prevents double-configuration and conflicting guidance.

The key distinction

Project instructions live in Chat's Project system. They apply to conversations within that Project in the Chat environment. They're available across chat sessions but don't automatically apply to Cowork automated tasks unless the Cowork task explicitly references the Project.

Cowork instructions live in the Cowork environment. They apply to Cowork work: scheduled tasks, Dispatch commands, interactive Cowork conversations. They're not visible to Chat unless you're working in a Cowork-connected conversation.

Think of them as belonging to different surfaces that can work with the same content but have separate instruction sets.

When you need both

For work that happens both in Chat and in Cowork, you may want to configure instructions in both places:

- A client Project in Chat (for interactive conversations, document uploads, analytical work) with Project instructions
- A corresponding Cowork folder configuration with Cowork instructions (for automated tasks that read that client's files and produce outputs)

The instructions don't need to be identical, but they should be consistent: the same constraints, the same context, the same output standards.

When one is sufficient

If a workflow only happens in Chat (interactive conversations, manual document review), Project instructions are sufficient. Cowork instructions are not needed.

If a workflow only happens in Cowork (fully automated file processing, scheduled reports), Cowork instructions are sufficient. Project instructions may not add value.

If a workflow happens in both, configure both.

Avoiding conflicts

A practical check: if Claude behaves unexpectedly in Cowork, look for conflicts between your Cowork global instructions, your per-folder instructions, and any Project instructions that might also be in scope. Identify which layer is producing the unexpected behavior and resolve it at that layer.

Try this in Claude

Look at your most-used Chat Project. Would you want the same context available in a Cowork automated workflow? If yes, plan where those Cowork instructions would live (global or per-folder) and note any differences between what the Project version and the Cowork version would say. This exercise makes any needed configuration explicit before you start building automation.

Quick summary

Project instructions live in Chat and govern Chat conversations. Cowork instructions live in Cowork and govern automated and agentic work. For workflows that span both surfaces, configure instructions in both places consistently. Check for conflicts when Claude behaves unexpectedly; resolve at the source layer.

Section 9 — Meet Cowork: The Agentic Desktop

Review Mode vs. Action Mode

Review Mode vs. Action Mode

Cowork lets you configure how much autonomy Claude has in each workflow. Review mode keeps you in the loop: Claude produces output and waits for your confirmation before acting. Action mode lets Claude proceed: Claude produces output and acts without waiting. The right setting depends on the workflow and your confidence in it.

Why this matters

Defaulting to action mode because it's more efficient is the mistake that causes real problems in Cowork. The time saved by removing review steps is small compared to the time cost of fixing an autonomous action that went wrong. Start with review, earn your way to action.

Review mode

In review mode, Claude: - Produces drafts, outputs, and proposed actions - Presents them for your review - Waits for explicit confirmation before anything consequential happens (sending, writing, deleting, acting)

Review mode is appropriate when: - A workflow is new and you haven't verified its output quality - The actions are irreversible (sending emails, deleting files) - The task involves external parties who would be affected by errors - You haven't built confidence in this specific workflow yet

Action mode

In action mode, Claude: - Produces outputs and takes actions without waiting for your confirmation - Notifies you of what it did (depending on configuration) - Proceeds to the next step in the workflow

Action mode is appropriate when: - The workflow has run many times and produced consistently correct output - The actions are reversible or low-stakes (writing to a draft folder, creating internal files) - The workflow has appropriate constraints that limit what Claude can do autonomously - You have monitoring in place to catch unexpected behavior

Transitioning from review to action

The responsible path: run every new workflow in review mode first. Verify outputs across multiple runs. Identify edge cases. Only move to action mode once you've seen the workflow handle the range of inputs it will encounter in practice.

A rough threshold: if a workflow has produced correct, review-ready output across ten to twenty runs without requiring significant correction, it's a candidate for action mode, assuming the actions are recoverable if something unexpected happens.

For workflows with irreversible actions, consider keeping a review gate permanently, regardless of how reliable the workflow is. The time cost of that review gate is worth the protection.

Try this in Claude

For the Cowork workspace you're setting up, decide: what is the default mode for your first workflows? Write it into your global instructions: "All workflows should start in draft/review mode. Action mode requires explicit per-workflow approval." This default prevents accidental action mode on workflows you haven't verified.

Quick summary

Review mode keeps you in the loop before consequential actions. Action mode lets Claude proceed without confirmation. Start in review mode, always. Move to action mode after a workflow has demonstrated consistent reliability across multiple runs. Keep review gates for irreversible actions regardless of workflow reliability. Write your default mode preference into your global Cowork instructions so new workflows start safely.

SECTION 10 · ACT 2 — COWORK

Section 10 of 14

Section 10: Automation — Scheduled & Recurring Tasks

Section 10: Automation — Scheduled & Recurring Tasks

The most reliable way to do something consistently is to set it up once and have it run automatically. That's what this section is about: configuring Claude tasks that execute on a schedule without requiring you to ask for them every time.

Why this section matters

There is a category of work that is valuable every time it's done but tedious to remember to do. Your daily briefing. Your weekly report. A recurring data check. A regular synthesis of what came in since the last run. These tasks have a consistent structure, consistent inputs, and consistent outputs. They're exactly what scheduling is designed for.

Once a scheduled task is working, it produces output every day or every week whether or not you have capacity to think about it. The good weeks and the bad weeks both get the briefing. That's the compounding value of automation.

What you'll learn

The /schedule skill and the Scheduled Tasks sidebar. How to set up and manage scheduled tasks in Cowork, and where to see them in the interface.

Cadences: hourly, daily, weekly, weekdays-only. The time patterns available and when each is appropriate.

Keep-awake, and missed runs that auto-execute when the computer wakes. What happens when the computer is off when a task is scheduled to run, and how to configure recovery behavior.

Per-run access to files, connectors, and plugins. What each scheduled run can access, and how to ensure it has the resources it needs.

Daily-digest and weekly-report patterns. Two of the most commonly useful automation templates and how to configure them.

The mini project

You'll design and set up a morning digest automation: a scheduled task that runs each weekday morning, reviews your inbox and calendar, identifies priorities, and leaves a briefing in your review folder. You'll configure the cadence, the prompt, and the folder output.

How to approach this section

Start with the simplest possible scheduled task: a single task that reads one thing and writes one thing. Verify it runs correctly, produces the right output, and handles edge cases (empty inbox, no new calendar items) before building anything more complex. Complexity in automation is earned by reliability at simpler levels.

The scheduled task patterns in this section are ones that consistently deliver value for professionals managing significant information volume. They're starting points, not recipes to follow rigidly. Adjust the timing, the prompt, and the output format to match how you actually work.

Section 10 — Automation: Scheduled & Recurring Tasks

The /schedule Skill and the Scheduled Tasks Sidebar

The /schedule Skill and the Scheduled Tasks Sidebar

Scheduling a Cowork task involves two things: writing a complete prompt for what Claude should do, and telling Cowork when to run it. The /schedule skill is how you create a scheduled task from the Cowork conversation interface. The Scheduled Tasks sidebar is how you manage them after they're set up.

Why this matters

Understanding the scheduling interface means you can set up automations quickly and correctly, and monitor them without uncertainty about whether they're running as intended.

The /schedule skill

The /schedule skill is invoked by typing /schedule in the Cowork conversation interface. It starts a guided setup flow for creating a new scheduled task. You'll specify:

The task prompt: What Claude should do when the task runs. This is the most important element. It should be written as a complete, self-contained instruction that Claude can follow without any additional context from you.

The cadence: When the task runs. Options typically include specific times (daily at 7am, weekdays at 8:30am) and intervals (every four hours, every Monday at 9am).

The resources: Which folders, connectors, and plugins the task should have access to when it runs.

The output: Where results should go. A folder where Claude writes output files. A notification. A combination.

After setup, the task appears in the Scheduled Tasks sidebar and begins running on its configured schedule.

Writing a complete scheduled task prompt

The most important thing to understand about scheduled task prompts: Claude has no opportunity to ask clarifying questions when the task runs. Everything it needs must be in the prompt.

A good scheduled task prompt specifies: - What to read or access (which folders, which connectors, what data) - What to do with it (what analysis, what synthesis, what generation) - What the output should look like (format, structure, length) - Where to put the output (which folder, what file name pattern) - How to handle edge cases (no new items, data that doesn't fit the expected pattern)

A prompt that would work for an interactive conversation often fails as a scheduled task because it relies on back-and-forth that doesn't exist in automation.

The Scheduled Tasks sidebar

The Scheduled Tasks sidebar shows all your configured tasks, their cadences, their last run status, and their next scheduled run. From here you can: - Enable or disable tasks without deleting them - View the log of recent runs - Edit the task prompt or cadence - Delete tasks that are no longer needed

Build a habit of reviewing the sidebar periodically: are all tasks running as expected? Are any failing silently? Are any producing output you're no longer reviewing?

Try this in Claude

Before creating your first scheduled task, write the prompt as if you were going to run it manually right now. Run it manually and verify the output is what you want. If it is, schedule it. This "manual first" pattern prevents the frustration of scheduling a task and then discovering the prompt doesn't produce what you expected.

Quick summary

The /schedule skill guides you through scheduled task setup. Write complete, self-contained task prompts that don't rely on back-and-forth clarification. The Scheduled Tasks sidebar is your ongoing management interface: review it regularly for task status and output quality. Always test a task manually before scheduling it.

Cadences: Choosing the Right Schedule for Your Task

A scheduled task runs on a cadence. The cadence determines when it runs, how often it runs, and what time window it operates in. Choosing the right cadence is not just a scheduling decision; it's a workflow design decision.

The available cadences

Hourly: The task runs every hour. Appropriate for tasks where freshness matters at this granularity: monitoring for incoming items that need rapid triage, checking a data source that updates frequently. Be thoughtful about hourly tasks: they generate output (or try to) 24 times a day. If no one is reviewing the output, hourly is probably too frequent.

Daily: The task runs once per day at a specified time. The most common cadence for briefing and summary tasks. "Every day at 7am" or "every day at end-of-business" works well for tasks that synthesize a day's worth of activity.

Weekdays only: The task runs on weekdays but not weekends. Appropriate for work-context tasks where weekend runs would produce empty or irrelevant output. Most professional briefings and report tasks should use weekdays-only rather than daily to avoid Saturday morning output that no one reads until Monday.

Weekly: The task runs once per week on a specified day and time. Appropriate for weekly synthesis, review, and planning tasks. A weekly report that runs Sunday evening to be ready for Monday morning review is a common and useful pattern.

Custom intervals: Some scheduling configurations allow specific day-of-week and time combinations, like "every Monday and Thursday at 8am." Useful for tasks tied to recurring team rhythms.

Matching cadence to value

The right cadence is the one that matches how often the output is actually useful. More frequent is not better; unnecessary runs produce output that piles up unreviewed, which either creates overhead or trains you to ignore the automated outputs.

A useful check: if this task ran right now and produced a result, would you actually want that result? If the answer is "yes, at 7am each weekday but not at 11pm on a Sunday," configure it accordingly.

Edge cases in cadences

What happens when a task runs and there's nothing to process? (Empty inbox, no new calendar items.) Your task prompt should include a handling instruction: "If there are no new items, produce a brief note saying no items found rather than an empty output."

What happens when output is the same two days running? A daily briefing that finds no new email and no calendar changes produces the same result as yesterday. That's acceptable as long as the output clearly reflects the current state rather than being stale.

Try this in Claude

For the morning digest you're building in this section's mini project, decide: what cadence? Daily or weekdays-only? What time, specifically? What should the task do if there's nothing new to report? Write those decisions into your task prompt before scheduling.

Quick summary

Cadences range from hourly to weekly. Choose the one that matches how often the output is genuinely useful, not the most frequent option available. Weekdays-only is usually right for work context tasks. Include edge case handling in prompts for situations where there's nothing to process. More frequent is not better if output exceeds your review capacity.

Section 10 — Automation: Scheduled & Recurring Tasks

Keep-Awake and Missed Runs

Keep-Awake and Missed Runs

Scheduled tasks run on your computer. If your computer is off, asleep, or not connected when a task is scheduled to run, the task doesn't run on schedule. Cowork's keep-awake and missed-run features handle this situation.

Why this matters

A morning digest that's supposed to run at 7am doesn't help much if your computer is closed until 8am. Understanding how missed runs work prevents the confusion of tasks that seem to have stopped running.

Keep-awake

The keep-awake setting prevents your computer from sleeping while Cowork is active and scheduled tasks are pending. With keep-awake enabled, your computer stays awake (screen may still dim) to allow scheduled tasks to run at their configured times.

Use keep-awake if: - Your tasks run at times when your computer might otherwise sleep (overnight, during long meetings) - Task timeliness matters more than power consumption - You have tasks running at specific times when you need the output to be ready

Turn off keep-awake if: - Your tasks run during hours when your computer is normally active - Power consumption or hardware longevity is a concern - Your task schedule is flexible enough that missed-run recovery is acceptable

Missed-run recovery

When a scheduled task misses its run time (computer was off or asleep), Cowork can execute the missed run when the computer wakes. This is the "auto-execute on wake" behavior.

With this setting enabled: when your computer wakes, Cowork checks for missed tasks and runs them. If your 7am briefing missed its run because your laptop was closed, it runs when you open the laptop at 8am.

Consideration for time-sensitive tasks: A morning briefing run at 8am instead of 7am is usually still useful. A time-sensitive alert run three hours late may not be. Configure missed-run recovery based on how time-sensitive the output actually is.

Consideration for stacking: If multiple tasks missed their runs while the computer was off, they may all run at once when it wakes. This can produce a burst of output. For tasks with light compute requirements, this is fine. For tasks that make many connector calls, it may create momentary load.

Practical configuration

For most professional users, a practical configuration is: - Keep-awake: enabled during work hours (Cowork running, computer active) - Missed-run recovery: enabled for morning briefings and daily digests - Specific tasks: configure individually based on time-sensitivity

Try this in Claude

For each scheduled task you plan to set up, decide: does it need to run at the exact scheduled time, or is it acceptable for it to run when the computer next wakes? Document that decision in your task setup notes. This informs whether missed-run recovery is the right behavior or whether you need the task to be more precisely timed (which requires the computer to be active at run time).

Quick summary

Keep-awake prevents your computer from sleeping while tasks are pending. Missed-run recovery executes overdue tasks when the computer wakes. Use keep-awake for time-sensitive tasks that must run at specific times. Enable missed-run recovery for tasks where timeliness is flexible. Configure both based on actual task requirements, not defaults.

Section 10 — Automation: Scheduled & Recurring Tasks

Per-Run Access to Files, Connectors, and Plugins

Per-Run Access to Files, Connectors, and Plugins

Each scheduled task run has access to exactly what you've configured it to have access to. Files in authorized folders, connectors you've enabled, plugins that are installed. Understanding this access model prevents both the frustration of tasks that fail because they can't reach a resource, and the risk of tasks that have broader access than necessary.

Why this matters

A scheduled task that fails because it can't reach a connector is frustrating but safe. A task that has access to folders or connectors it doesn't need creates unnecessary risk. Access configuration is a workflow design decision, not just a setup step.

Folder access

Scheduled tasks can read from and write to folders you've authorized in Cowork. A task needs: - **Read access** to any folder it reads input from - **Write access** to any folder it deposits output into

If a task needs to read from your "Client Notes" folder and write to your "Reports" folder, both folders need to be authorized with the appropriate access levels.

A practical pattern: create an "Inputs" folder and an "Outputs" folder in your Cowork workspace. Scheduled tasks read from Inputs (or from specific project folders) and write to Outputs (or a review subfolder). This creates a clear separation between source material and generated output.

Connector access

Scheduled tasks can use connectors you've configured in Cowork. If a task needs to read email, the email connector must be authorized. If a task needs to read your calendar, the calendar connector must be authorized.

Check connector status before setting up tasks that depend on them. A task that uses an expired connector authorization will fail silently or produce incomplete output.

Plugin access

Installed plugins are available to scheduled tasks. If a task uses a Skill from a plugin, the plugin must be installed and active. Plugin-provided connectors must be authenticated.

Access scoping

Configure each task with the minimum access it actually needs. A task that reads your inbox and produces a text summary doesn't need write access to your file system. A task that generates a report doesn't need your CRM connector.

Minimal access reduces the blast radius if a task misbehaves: a task that can only write to one specific folder can only cause problems in that folder.

Try this in Claude

For the morning digest task you're building, list every resource it needs: which folders (read), which folder for output (write), which connectors, which plugins. Verify each is authorized before you schedule the task. This checklist prevents the "task ran but failed to access resources" failure mode.

Quick summary

Scheduled tasks access exactly what you configure: authorized folders, active connectors, installed plugins. Configure each task with minimum necessary access. Verify all required

resources are authorized before scheduling. A task that fails to access resources is immediately apparent; a task with excessive access creates risk you won't see until something goes wrong.

Section 10 — Automation: Scheduled & Recurring Tasks

Daily Digest and Weekly Report Patterns

Daily Digest and Weekly Report Patterns

Two automation patterns consistently deliver the most value to professionals managing significant information volume. This lesson gives you the templates for both.

The daily digest

A daily digest is a morning briefing: a summary of what's in your inbox, what's on your calendar, and what needs your attention today. It runs on a schedule and produces a file waiting for you when you start work.

What it typically includes: - High-priority email summary (flagged, unread, from key contacts) - Today's calendar in brief - Any items pending more than a defined number of days - Any time-sensitive items in monitored folders

Sample task prompt:

```
Morning digest task. Run every weekday at 7:00am.

Access: email connector (read), calendar connector (read), [output folder]
(write)

Task:
1. Read my inbox. Identify emails that arrived since yesterday 7am that are: (a)
unread from my key contacts [list or define], (b) flagged, or (c) appear
time-sensitive based on subject. Summarize each in one sentence.

2. Read today's calendar. List all events with: time, title, and any relevant
notes.

3. Identify anything in my inbox where someone is waiting for a reply I haven't
sent in more than 3 days.

4. Write the output to [output folder]/daily-digest-[date].md with sections:
Email Priorities, Today's Calendar, Pending Replies.
```

```
If there are no items in a category, note "Nothing to report" for that section.
```

Output pattern: A dated markdown file in a review folder. You open it when you start work. You scan it, act on what needs action, and move on. The file is your starting context for the day.

The weekly report

A weekly report synthesizes a defined period into a standing document: what happened, what's pending, what's coming. It runs on a consistent schedule and produces output ready for your weekly review.

What it typically includes: - Summary of significant activity in monitored areas - Status of ongoing projects based on folder contents - Calendar review: meetings completed, outcomes if noted - Looking ahead: next week's key commitments

Sample task prompt:

```
Weekly report task. Run every Friday at 4:00pm.

Access: email connector (read), calendar connector (read), [project folders]
(read), [output folder] (write)

Task:
1. Summarize email activity from this week: key threads completed, significant
decisions made, items still unresolved.

2. Review my calendar for this week: key meetings, any notable outcomes mentioned
in meeting notes files.

3. Check [project folders] for any files modified this week and note what
changed.

4. Review next week's calendar for significant commitments I should be prepared
for.

5. Write the output to [output folder]/weekly-report-[date].md with sections:
This Week Summary, Ongoing Projects, Looking Ahead.
```

Tailoring to your workflow

These templates are starting points. The right digest for a VP of Sales looks different from the right digest for a research analyst. Adjust the data sources, the summary format, and the output structure to match how you actually work.

The most important variable: what would make you more effective at the start of each day or week? Design toward that.

Try this in Claude

Build the morning digest template for your work. Use the sample above as a starting point and adapt it: what connectors do you have? What information is most valuable to you first thing? What do you wish you knew every morning that you currently have to spend time finding? Write the prompt, run it manually, and evaluate the output before scheduling.

Quick summary

Daily digests and weekly reports are the two most consistently valuable automation patterns for knowledge workers. Daily digests synthesize inbox, calendar, and pending items into a morning briefing. Weekly reports synthesize the week's activity and look ahead. Start with the templates and adapt to your specific sources and needs. Run manually first, then schedule.

SECTION 11 · ACT 2 — COWORK

Section 11 of 14

Section 11: Dispatch — Run Cowork from Your Phone

Section 11: Dispatch — Run Cowork from Your Phone

Dispatch connects your phone to your desktop. You send an instruction from the Claude mobile app; your desktop Cowork environment receives it, executes the task, and leaves the result waiting for you. It's the remote control for your Cowork workspace.

Why this section matters

The limitation of scheduled tasks is that they run on a fixed schedule, not when you actually need them. Dispatch removes that constraint. You're in a meeting, an idea comes up, and you need a quick analysis of a file on your desktop. You're traveling and need a document drafted from notes in your work folder. You're between calls and want to know what's in your inbox before the next one starts. Dispatch handles all of these by turning your phone into a trigger.

What you'll learn

What Dispatch is. A continuous conversation between your mobile Claude app and your desktop Cowork environment. Instructions sent from mobile are received and executed by the desktop app.

The fast reach methods. Connectors (the fastest path, direct API access) and the Chrome extension (for web apps that don't have connectors). When to use each.

Writing a complete upfront prompt. Dispatch tasks need the same completeness as scheduled tasks: Claude can't ask follow-up questions while the task runs. Everything it needs must be in the message you send from your phone.

The keep-awake requirement. Dispatch requires your desktop to be awake and Cowork to be running. How to manage this for reliable remote access.

Picking up finished work. Where to find the results of Dispatch tasks when you return to your desktop.

Security for remote access. What the security model is for Dispatch and what you should consider before using it for sensitive work.

The mini project

You'll design a message-from-mobile workflow: a specific instruction you'd realistically send from your phone that reads from your local files, pulls from a connector, and produces a finished file you can review at your desk. The design includes the prompt, the resources it accesses, and the expected output.

How to approach this section

Dispatch is the Cowork feature most dependent on setup working correctly. The desktop needs to be running, authorized, and connected. Test Dispatch in a low-stakes context before using it for anything important. Send a simple test instruction, verify it runs, and confirm you can find the output. Then build more complex workflows.

The mental model shift from scheduled tasks: scheduled tasks are push (Cowork pushes output to you on a schedule). Dispatch is pull (you pull Cowork into action when you need it). Both patterns are useful; which to use depends on whether the task should happen on a fixed schedule or on demand.

Section 11 — Dispatch: Run Cowork from Your Phone

What Dispatch Is

What Dispatch Is

Dispatch is a continuous, persistent conversation thread that links your Claude mobile app to your desktop Cowork environment. Messages you send in this thread from your phone are received by Cowork on your desktop, executed with full file and connector access, and responded to in the same thread.

Why this matters

Without Dispatch, Cowork is desktop-only. You can only trigger work when you're sitting at your computer. With Dispatch, you can trigger Cowork from anywhere your phone goes. The desktop does the work; you just have to tell it what to do.

How it works

The Dispatch thread appears in both the Claude mobile app and in Cowork on your desktop. It's a single conversation that spans both surfaces:

- You open the Dispatch thread in the Claude mobile app
- You send a message: "Review the files in my Client X folder and summarize what's changed since last Monday"
- Your desktop Cowork environment receives the message (if the desktop is running and awake)
- Cowork reads the files, generates the summary, and responds in the same thread
- You receive the response in the mobile app

The conversation is persistent. You can review past Dispatch exchanges, see what was requested and what was produced, and continue previous threads. This is what the "one continuous conversation" framing means: it's not a session that starts and ends. It's a thread you return to.

What Dispatch can access

When a Dispatch task runs, it has access to whatever your Cowork environment is authorized to access: the folders you've granted permission for, the connectors you've set up, the plugins you've installed. Dispatch doesn't add new access; it allows you to invoke existing Cowork capabilities remotely.

When Dispatch doesn't work

Dispatch requires your desktop to be running with Cowork active. If the desktop is off, closed, or if Cowork isn't running, Dispatch messages won't be executed until the desktop is active again (at which point they may execute as missed runs, depending on configuration).

This is a real operational consideration. Dispatch is most reliable if your desktop typically stays on and Cowork stays open during working hours. For off-hours use, keep-awake and missed-run settings determine what happens.

Try this in Claude

Set up the Dispatch thread by opening the Claude mobile app and finding the Dispatch option. Send a simple test message: "What files are in my [one authorized folder]?" Verify the response arrives. This test confirms the end-to-end connection is working before you rely on it for anything important.

Quick summary

Dispatch is a persistent conversation thread connecting your mobile Claude app to your desktop Cowork environment. Messages trigger Cowork tasks with full file and connector access. It requires the desktop to be running. The thread persists across sessions, giving you a history of what you've requested and what was produced.

The Fast Reach Methods: Connectors and the Chrome Extension

The Fast Reach Methods: Connectors and the Chrome Extension

When a Dispatch task needs to access external data or tools, it has two main paths: direct connector access and the Chrome extension. Understanding which to use and when is the difference between tasks that run in seconds and tasks that run in minutes.

Connectors: the fast path

Connectors provide direct API access to external services: email, calendar, CRM, project management tools. When a Dispatch task uses a connector, Claude accesses the service's API directly. There's no browser involved, no interface to navigate, no visual rendering. The data comes back immediately.

Connector-based tasks typically complete in seconds to a minute, depending on data volume. They're the right choice whenever a connector exists for the service you need to access.

Example: "Check my email for anything from Meridian Capital and summarize the last three messages." This uses the email connector directly. Claude queries the email API, retrieves the messages, and returns a summary. No browser required.

The Chrome extension: for web apps

Some services don't have connectors, or have connectors that don't expose the specific data you need. For these, the Chrome extension allows Cowork to access a web app through the browser.

The Chrome extension: - Requires the Chrome browser to be running with the extension installed - Navigates to the target web app on your behalf - Reads page content or interacts with the interface - Returns data to the Cowork task

Chrome extension tasks are slower than connector tasks (seconds to several minutes) because they involve browser navigation and rendering. They're also more fragile: web app interfaces change, and a task that worked last week may need updating after the app updates.

Use connectors whenever they exist. Use the Chrome extension when they don't, understanding the additional latency and maintenance requirement.

Choosing the right method

For any external data source, ask: 1. Is there a connector for this service? If yes, use the connector. 2. Does the connector expose the specific data I need? If no connector or the connector is insufficient, consider the Chrome extension. 3. Does the data exist in a local file I've already authorized? If yes, use file access instead of an external call.

Try this in Claude

For the Dispatch workflow you're designing in this section's mini project, identify each external data source it needs to access. For each, determine: connector or Chrome extension? Verify that connectors are set up and authorized before relying on them in a Dispatch task.

Quick summary

Connectors provide fast, direct API access to external services. The Chrome extension accesses web apps through the browser when connectors aren't available. Connectors are faster and more reliable; prefer them whenever they exist. Chrome extension access is slower and more maintenance-intensive. Match the access method to what's available for each data source.

Section 11 — Dispatch: Run Cowork from Your Phone

Writing a Complete Upfront Prompt for Dispatch

Writing a Complete Upfront Prompt for Dispatch

A Dispatch prompt is not a conversation opener. It's a complete instruction set that Claude executes without any back-and-forth. The quality of what comes back is directly proportional to how complete and clear the prompt is when you send it from your phone.

Why this matters

The difference between a Dispatch task that does exactly what you needed and one that does something adjacent to what you needed is almost always in the prompt completeness. Prompts written on a phone, quickly, between meetings, tend to be short and vague. The context you're

holding in your head doesn't make it into the message. Claude doesn't have that context.

The three elements of a complete Dispatch prompt

What to read (inputs): Every source Claude should access. Be specific about which folders, which connectors, what time window. - "Read all files in my Client X project folder modified in the last 7 days" - "Check my calendar for the next 48 hours" - "Search my email for messages with 'invoice' in the subject received this week"

Where to look (scope): Any constraints on scope or focus. - "Focus on the meeting notes files, not the proposal drafts" - "Only consider emails from external senders, not internal team messages" - "Specifically look at the risk sections of each document"

What format to return (output): What the finished output should look like and where it should go. - "Write a one-page summary to /Cowork Outputs/client-x-summary-[today's date].md" - "Respond in this conversation with a bulleted list, no more than 10 items" - "Create a draft email and put it in /Drafts/ready-to-send.md"

Template for Dispatch prompts

Before you're in the situation of typing from your phone under time pressure, draft Dispatch prompt templates for your common use cases. Keep them in a note where you can copy-paste with minimal editing.

Template structure:

```
Read: [specific sources]
Focus on: [scope constraints]
Produce: [output description]
Put output: [destination]
If nothing found: [edge case handling]
```

Practical example

Vague Dispatch message (produces mediocre output):

> *"Check the client folder and give me a summary"*

Complete Dispatch message (produces useful output):

> *"Read all files in /Client Projects/Hartwell Technologies/ modified in the last 14 days. Focus on meeting notes and status updates, not the background research documents. Write a 200-word status summary covering: what's been completed, what's in progress, and any open questions. Save to /Cowork Outputs/hartwell-status-[date].md."*

The second message takes about thirty seconds longer to type (or can be templated). The output is directly usable.

Try this in Claude

Write three Dispatch prompt templates for the Dispatch tasks you're most likely to use. Save them somewhere accessible from your phone. The next time you need to send a Dispatch task, you'll copy the template and modify the variables rather than drafting from scratch.

Quick summary

Dispatch prompts need to be complete before you send them: what to read, what scope to apply, and what output to produce and where. Templates prepared in advance make complete prompts practical to send from a phone under time pressure. Vague prompts produce adjacent-but-not-quite outputs; complete prompts produce directly usable results.

Section 11 — Dispatch: Run Cowork from Your Phone

The Keep-Awake Requirement

The Keep-Awake Requirement

Dispatch requires your desktop to be awake and Cowork to be running. This is not a bug or a limitation to be worked around; it's the operational reality of the system. Managing it deliberately is part of making Dispatch reliable.

Why this requirement exists

Cowork runs on your machine. Unlike a cloud service, it doesn't have a server that receives your Dispatch messages when your computer is off. Your desktop is the processing environment. When it's asleep, it can't receive instructions or execute tasks.

Practical implications

For in-office work: If you're at your desk most of the day and your computer is on, Dispatch works reliably. Your most valuable use cases are "I need something before my next meeting" tasks sent from your phone while walking between rooms or during a break.

For travel: Before leaving on a business trip, consider whether you want Dispatch to be available from your phone during the trip. If yes, you need to leave your desktop on and Cowork running. If it's a laptop you're taking with you, Dispatch isn't useful for tasks that require your desktop files.

For overnight or weekend tasks: If you want a Dispatch task to run at midnight or on a Saturday, your desktop needs to be on and awake. Keep-awake settings can manage this, but they also mean your computer runs continuously.

Keep-awake configuration

In Cowork settings, the keep-awake option prevents your computer from sleeping while Cowork is active. With this enabled: - Your computer stays awake (display may still dim) - Dispatch tasks are received and executed whenever you send them - Scheduled tasks run at their configured times

The tradeoff is power consumption and hardware wear from continuous operation. For a desktop computer in an office, this is usually acceptable. For a laptop on battery power, be more deliberate about when keep-awake is active.

A practical pattern

Many professionals configure Cowork to keep awake during work hours (start time to end time, weekdays) and allow sleep outside those hours. Dispatch tasks sent outside work hours queue and execute when the computer wakes. This balances availability with reasonable operating hours.

Try this in Claude

Decide: during what hours do you want Dispatch to be reliably available? Configure keep-awake accordingly. For the first week, send a few test Dispatch tasks at different times and verify they execute correctly. This gives you real operational experience of the system's reliability under your specific setup.

Quick summary

Dispatch requires a running, awake desktop with Cowork active. Keep-awake settings prevent sleep during configured hours. Balance availability against power and hardware considerations. Establish operating hours for Dispatch and configure accordingly rather than assuming the default is correct.

Section 11 — Dispatch: Run Cowork from Your Phone

Picking Up Finished Work on the Desktop

Picking Up Finished Work on the Desktop

Dispatch tasks produce output. That output needs to go somewhere you can find it. Establishing a clear, consistent output location and format before you start using Dispatch heavily prevents the experience of returning to your desk and not knowing where to look.

Why this matters

The value of a Dispatch task isn't just that it ran. It's that the output is waiting in a useful form where you can immediately act on it. An output that's hard to find or requires decoding to interpret is only partially useful.

Output options

A dedicated review folder. The most common and recommended pattern: Dispatch tasks write output to a specific folder (e.g., "Cowork Outputs" or "Review Queue") that you check when you return to your desk. Files are dated and named clearly. You open the folder, review the outputs, act on what needs action.

A response in the Dispatch thread. For shorter outputs, Claude can respond directly in the Dispatch conversation thread. You return to the thread in the mobile app or on the desktop and read the response there. This works for summaries, answers, and short outputs. It doesn't work well for long documents or files you need to edit.

A specific file path. For tasks that should update or create specific files, you can specify the exact path. "Write to /Projects/Hartwell/weekly-notes.md" produces output at the exact location where you expect it.

Naming conventions for output files

For a review folder that accumulates output over time, clear naming conventions prevent confusion: - Include the task type: briefing, summary, draft, analysis - Include the date: YYYY-MM-DD format sorts chronologically - Be specific about the subject: client name, topic, context

Example names: - `2026-05-30-morning-briefing.md` - `hartwell-status-update-2026-05-30.md` - `draft-reply-meridian-invoice-question.md`

Review habit

When you return to your desk after triggering one or more Dispatch tasks, your first step is the review folder (or the Dispatch thread, depending on your output pattern). Review outputs, act on

what needs action, and clear reviewed items.

If outputs are accumulating unreviewed, either your cadence is too frequent, the tasks are producing outputs that aren't useful, or your review habit needs strengthening.

Try this in Claude

Create a "Cowork Outputs" folder in your file system. Add it to your Cowork authorized folders with write access. Update your Dispatch task prompts to write output there. Verify that the next Dispatch task produces a file in that folder with a sensible name.

Quick summary

Dispatch task output needs a clear, consistent destination. A dedicated review folder is the most practical pattern for accumulated output. Name files clearly with date and subject. For short outputs, in-thread responses work. For longer outputs and documents, file-based output is more practical. Establish the output pattern before relying on Dispatch for real work.

Section 11 — Dispatch: Run Cowork from Your Phone

Security for Remote Access and Connected Tools

Security for Remote Access and Connected Tools

Dispatch extends Claude's reach from your phone to your desktop, through your connectors, to your files. That's a powerful combination, and it's one worth thinking carefully about before you rely on it for sensitive work.

Why this matters

Remote access to a computer that has access to your email, your files, and your professional data is a meaningful security surface. Understanding it doesn't mean being afraid of it; it means making deliberate choices about what's accessible and under what conditions.

The security model

Dispatch works through the Claude mobile app's authenticated session. Your Claude account credentials are what authorize the Dispatch connection. The same account that has access to your Cowork environment is the account that controls Dispatch.

This means: - Anyone with access to your Claude account can send Dispatch instructions - The mobile device you use for Dispatch needs to be secured (lock screen, device encryption) - Account credential security (strong password, two-factor authentication) is the primary security control

If you're concerned about account compromise: enable two-factor authentication on your Claude account. This is the highest-leverage security action.

Access scoping

The data accessible through Dispatch is the data your Cowork environment is authorized to access: the folders you've granted permission for and the connectors you've configured. Dispatch doesn't add access; it allows you to invoke existing access remotely.

To scope Dispatch access: - Only authorize folders in Cowork that you're comfortable being accessible via Dispatch - For folders with particularly sensitive content, consider whether they need to be in the Cowork authorized set at all - Review connector access: does your email connector need write (send) access, or is read sufficient for your workflows?

For enterprise and regulated environments

If you work in an environment with data handling requirements (finance, healthcare, legal, government), Dispatch (and Cowork generally) may be subject to IT policy review before you use it with work data. Check with your organization's IT or compliance team before connecting work data to Cowork.

Consumer Claude plans have different data handling terms than enterprise plans. If your organization has an enterprise Claude agreement, use those credentials rather than personal credentials for work data.

A practical security checklist

- Two-factor authentication enabled on the Claude account
- Mobile device secured (lock screen, biometrics)
- Only sensitive folders authorized in Cowork that genuinely need to be
- Connector access scoped to what's actually needed (read vs. read-write)
- IT policy reviewed if working with regulated or confidential business data

Try this in Claude

Review your current Cowork folder authorizations and connector configurations with fresh eyes: is anything authorized that doesn't need to be? Is any connector permission broader than your workflows require? Make any adjustments. This audit takes fifteen minutes and reduces your

unnecessary exposure.

Quick summary

Dispatch security rests primarily on Claude account security (use two-factor authentication) and device security (lock screen, encryption). Access scope is determined by what your Cowork environment is authorized to access. Review folder and connector permissions and keep them minimal. For regulated environments, check IT policy before connecting work data. Security through scope limitation is more reliable than hoping for the absence of incidents.

Section 12: Computer Use — Letting Claude Operate Your Apps

Section 12: Computer Use — Letting Claude Operate Your Apps

Computer use is Cowork's most powerful and most carefully governed feature. It allows Claude to control your mouse and keyboard, navigate application interfaces, and interact with software as if a human were operating it. For apps without connectors and without a web version, it's the only way to automate them.

Why this section matters

Most of what Cowork does, it does through clean API calls: connectors reach services directly, file access is file system reads and writes. Computer use is different. It operates through the visual interface, which means it's slower, less precise, and more sensitive to interface changes than API-based approaches.

Understanding when computer use is the right tool (and when it isn't) is the practical skill this section builds.

What you'll learn

Computer use as last-resort reach. The philosophy: use connectors and the Chrome extension first. Use computer use when no other approach is available.

Research-preview status. Computer use is off by default in Cowork. You enable it explicitly in Dispatch settings. Understanding why it's off by default shapes how you approach it.

Per-app approval. Before Claude can operate an application, you grant it permission to do so. The permissions required and how to grant them.

When to prefer faster alternatives. The decision framework for choosing between computer use, connectors, and the Chrome extension.

Risks and the extra review. Computer use output deserves more careful review than connector output. The reasons and the implications.

The mini project

You'll plan a computer use task for a local application: choose a real application you use that has no connector, define what you'd want Claude to do in it, document the approval steps, and write the review process you'd apply to the output. The deliverable is a design document, not a running workflow. The design exercise forces the thinking that should precede any computer use deployment.

How to approach this section

Computer use is a capability to understand before you use, not one to explore by experimentation. The risks of an automation that goes wrong inside a desktop application are real: changes to files you didn't intend, operations in applications that have real-world consequences. Read this section carefully, plan deliberately, and start with low-stakes applications when you first try it.

If your entire workflow can be handled through connectors and the Chrome extension, you may never need computer use. That's fine. The section gives you the knowledge to make that determination.

Section 12 — Computer Use: Letting Claude Operate Your Apps

Computer Use: The Last Resort That's Still Worth Knowing

Computer Use: The Last Resort That's Still Worth Knowing

The name "computer use" describes what it does accurately: Claude uses your computer. It sees your screen, moves the cursor, clicks buttons, types text. For applications that exist only on your desktop with no web version and no API connector, it's the only way Claude can interact with them.

Why this matters

Connectors and the Chrome extension cover a wide range of applications. But legacy desktop software, proprietary internal tools, and applications with no web presence or public API are common in real professional environments. Finance teams using specific desktop tools. Operations teams with legacy systems. Professionals who use specialized desktop applications for their industry. Computer use is how Cowork reaches all of those.

The philosophy: connectors first

Every time you consider computer use, ask first: is there a connector? Is there a web version accessible through the Chrome extension? Can the data be extracted into a file that Claude can process without controlling the application?

If any of those alternatives work, use them instead of computer use. They're faster, more reliable, less fragile, and less likely to produce unexpected behavior.

Computer use is last-resort, not first-choice. "No other method works" is the correct bar for reaching for it.

What computer use looks like

Claude operates the application by: - Taking screenshots to see what's on screen - Moving the mouse to specific positions - Clicking elements - Typing text into fields - Reading on-screen content

From Claude's perspective, it's navigating a visual interface the same way a human would. From the application's perspective, it looks like normal user input.

This means computer use is slow (each action involves a screenshot-interpret-act cycle), sensitive to visual changes (if the interface updates, the instructions may fail), and requires that the application be visible on screen.

When computer use is genuinely the right answer

- Desktop-only applications with proprietary data formats
- Legacy software with no API or web equivalent
- Applications that require specific desktop workflows (file management, printing, specialized tooling)
- Situations where screen-based data is the only accessible form of the data you need

Try this in Claude

Identify one application in your work toolkit that has no connector, no useful web version, and that you interact with in repetitive, mechanical ways. That's your computer use candidate. Hold it in mind as you work through the rest of this section.

Quick summary

Computer use lets Claude control desktop applications by operating the mouse and keyboard. Use it only when connectors, Chrome extension, and file-based approaches don't work. It's slower, less reliable, and more fragile than API-based alternatives. Connectors first; computer use last.

Research Preview Status: Off by Default

Research Preview Status: Off by Default

Computer use is a research preview feature in Cowork. It's not enabled by default. You opt in deliberately, in Dispatch settings, after understanding what you're enabling.

Why it's off by default

Defaulting to off reflects the seriousness of the capability. Claude operating your mouse and keyboard in desktop applications can: - Make changes to files you didn't intend - Trigger actions in applications with real-world consequences - Interact with applications that hold sensitive data - Produce behavior that's hard to observe and harder to reverse

Research preview status means: this feature is available and functional, but it's still being refined. Behaviors that work correctly in most cases may not work correctly in all cases. Use it deliberately, with review, and with an understanding that edge cases exist.

What enabling it means

When you enable computer use in Dispatch settings: - Claude can be given instructions that involve operating desktop applications - Additional permission dialogs will appear for specific applications before Claude is authorized to operate them - Screen recording and accessibility permissions are typically required (covered in the next topic)

Enabling computer use doesn't mean Claude starts operating applications without instruction. It means the capability is available when you explicitly ask for it.

How to enable

In the Cowork Dispatch settings panel, find the Computer Use section and toggle it to enabled. The interface will walk you through any initial permissions required. If you're on a managed device, IT permissions may be required before the necessary system permissions can be granted.

A note on research preview updates

Because computer use is a research preview, its behavior and interface may change as Anthropic refines it. If you build workflows that depend on computer use, be prepared for occasional updates that require workflow adjustments.

Try this in Claude

Open Dispatch settings and locate the Computer Use toggle. Read the description and any guidance provided there. Decide: based on the applications in your work environment and the workflows you're considering, is computer use something you want to enable now? If yes, proceed with the permission requirements in the next topic.

Quick summary

Computer use is off by default and requires explicit enablement in Dispatch settings. It's a research preview feature that works in most cases but has edge cases. Enabling it makes the capability available; it doesn't trigger automatic use. Additional per-application permissions are required after enabling.

Section 12 — Computer Use: Letting Claude Operate Your Apps

Per-App Approval and Required Permissions

Per-App Approval and Required Permissions

Before Claude can operate a specific application, you grant it permission to do so. This per-application approval is separate from the global computer use enable. It's a deliberate, application-specific decision.

Why per-app approval matters

Blanket approval for Claude to operate all applications would be an overly broad grant. Per-app approval means you consciously decide: yes, Claude can operate this specific application. This limits exposure and ensures you've thought about each application before authorizing it.

System permissions required

Computer use requires two system permissions that are not granted by default:

Accessibility permissions: Allows Cowork to interact with application interfaces (read interface elements, send click and keyboard events). On macOS, this is in System Settings > Privacy & Security > Accessibility. On Windows, accessibility-related permissions are handled through the app's permission request dialogs.

Screen recording permissions: Allows Cowork to see your screen (take screenshots to understand the current state of the interface). On macOS: System Settings > Privacy & Security > Screen Recording. On Windows: similar permissions dialogs.

Both are required for computer use to function. Without accessibility permissions, Claude can't send input to applications. Without screen recording, it can't see what's on screen.

These permissions have broad implications: once granted, they allow Cowork to see everything on your screen and interact with any accessible application. For enterprise devices, these permissions may require IT approval.

Per-application authorization

After granting system permissions, you authorize specific applications when you first ask Claude to operate them. The authorization flow is part of the Dispatch interface when you send a computer use instruction.

Keep a mental (or written) list of which applications you've authorized. Periodically review whether those authorizations are still appropriate.

Try this in Claude

If you've decided to enable computer use, walk through the permission requirements for your operating system before your first use. Ensure you understand what each permission grants before approving it. If you're on a managed device, check with IT before granting these permissions.

Quick summary

Computer use requires accessibility and screen recording system permissions, plus per-application authorization. These are broader permissions than connector access. Grant them deliberately, per application. On managed devices, check IT policy first. These permissions allow Cowork to see your screen and interact with applications; grant them only after understanding that scope.

Section 12 — Computer Use: Letting Claude Operate Your Apps

When to Use Connectors or Chrome Extension Instead

When to Use Connectors or Chrome Extension Instead

Computer use is the capable but slow cousin in the Cowork toolkit. For every task where an alternative approach works, that alternative is better. This lesson is the decision framework for choosing correctly.

The speed comparison

Connector-based access: Seconds to a minute. Direct API calls retrieve or write data without visual interface interaction.

Chrome extension access: Seconds to a few minutes. Browser-based, faster than computer use, slower than connectors.

Computer use: Minutes. Each action involves a screenshot, interpretation, and action cycle. Complex tasks with many steps can take significantly longer.

For a task that runs once manually, these differences are acceptable. For a scheduled task running daily or a Dispatch task you send when time-pressed, they matter.

When connectors win

Use a connector whenever: - A connector exists for the service you need - The connector provides access to the specific data you need - You need reliable, consistent performance at scale

Web-based services with public APIs (email, calendar, CRM, project management, cloud storage) almost all have connectors available or buildable. Before reaching for computer use for any cloud service, verify whether a connector exists.

When the Chrome extension wins

Use the Chrome extension when: - The service is web-based but no connector is available - You need to interact with a web application's interface rather than just read data from it - The data is only accessible through the web interface

The Chrome extension is more fragile than connectors (web interfaces change) but faster than computer use. It's the right choice for the middle tier of web-accessible applications without connectors.

When computer use is actually necessary

Computer use is appropriate when: - The application is desktop-only with no web version - No connector exists and no Chrome extension path is viable - The application interface is the only way to reach the data or trigger the action - The task is worth the overhead of slower execution and greater fragility

The decision tree

- Is there a connector? Use it.
- Is it web-accessible? Use the Chrome extension.
- Can the data be extracted to a file Claude can process? Use file access.
- If none of the above work: consider computer use.

Try this in Claude

For the computer use candidate application you identified earlier in this section, work through the decision tree. Can a connector work? Can Chrome extension work? Can file-based access work? Only if all three are genuinely infeasible should you proceed with computer use.

Quick summary

Connectors are fastest and most reliable. Chrome extension is second. Computer use is last. Work through the decision tree before choosing computer use: connector, then Chrome extension, then file access, then (only if nothing else works) computer use. The speed and reliability differences are real and compound when tasks run repeatedly.

Section 12 — Computer Use: Letting Claude Operate Your Apps

Risks and the Extra Review Computer Use Output Deserves

Risks and the Extra Review Computer Use Output Deserves

Computer use output deserves more careful review than other types of Claude output. Not because Claude is malicious, but because computer use involves more points of potential failure than connector-based or file-based approaches, and because errors in desktop application automation can be harder to catch and harder to reverse.

Why the extra review

Visual interpretation errors: Claude interprets your screen from screenshots. If the interface is complex, densely populated, or updates between screenshots, Claude may misread what's there. An action taken on a misinterpreted screen state can produce incorrect results.

Interface fragility: If an application updates its interface between when you designed the workflow and when it runs, elements may have moved, renamed, or disappeared. Claude may proceed with an outdated interpretation, taking actions on the wrong element.

State dependency: Computer use workflows often depend on the application being in a specific state. If the application's state is different from expected (a dialog open that shouldn't be, a loading state that hasn't resolved), actions may not go where intended.

Compounding errors: In a multi-step computer use task, an error at step 3 doesn't stop the workflow. Steps 4 through 10 execute against the incorrect state produced by step 3. The further into the workflow an error occurs undetected, the more work may need to be undone.

The review posture for computer use

Review the action log, not just the output. Claude should report what actions it took during a computer use task. Review that log for unexpected actions, not just the final output. An action taken in the wrong application or at the wrong location is a problem regardless of what the output says.

Compare against a known good state. After a computer use task, compare the application state to what you expected. Did the right thing change? Did anything change that shouldn't have?

Always start in review mode. For any new computer use workflow, require Claude to stop and describe its intended actions before taking them. Review the plan before execution. Move to unattended execution only after the workflow has consistently produced correct plans that you've verified match intent.

Keep reversibility in mind. Before starting a computer use task, consider: if something goes wrong, how do I undo it? For tasks with reversible effects (temporary files, drafts, non-committed changes), this is manageable. For tasks with irreversible effects (permanent deletions, committed transactions, external sends), maintain hard review gates.

The appropriate use pattern

New computer use workflow: 1. Describe the task to Claude 2. Ask Claude to plan the steps before executing 3. Review the plan 4. Execute with logging enabled 5. Review the action log and output 6. Verify the application state matches intent 7. Repeat until the workflow is stable, then gradually reduce review overhead for low-risk steps

Try this in Claude

For your planned computer use task, write the review process before you design the automation. What will you check after it runs? What does "it worked correctly" look like? Having the success criteria defined before running the task makes the review faster and more reliable.

Quick summary

Computer use has more failure modes than connector-based approaches: visual interpretation errors, interface changes, state dependencies, and compounding errors in multi-step tasks. Review the action log, not just the output. Compare application state to expected state. Start in review mode. Keep reversibility in mind. The extra review overhead is proportionate to the extra risk.

SECTION 13 · ACT 2 — COWORK

Section 13 of 14

Section 13: Live Artifacts — Self-Updating Dashboards & Tools

Section 13: Live Artifacts — Self-Updating Dashboards & Tools

Static Artifacts are useful snapshots. Live Artifacts are tools that stay current. This section covers the upgrade from "Claude made this once" to "Claude keeps this current," and the design thinking that separates a live artifact that actually gets used from one that sounds impressive but isn't.

Why this section matters

The limitation of static Artifacts became clear in Section 6: every time the underlying data changes, the Artifact is stale. A metrics dashboard from last Tuesday isn't a dashboard; it's a record of last Tuesday. A project status page that reflects the state from when you last manually updated it isn't a status page; it's a document.

Live Artifacts solve this by connecting to real data sources: your email, your calendar, your project tools, your files. When you open a live artifact, it reads current data and renders current state. The dashboard is always now, not the last time you rebuilt it.

What you'll learn

Live artifacts vs. static artifacts. The architectural difference and why it matters for which type to build.

Connecting an artifact to MCP servers and connectors. How live artifacts access real-time data from the tools you've already connected.

Auto-refreshing dashboards, briefings, and reports. The patterns that work well as live artifacts and the ones that don't.

The Live Artifacts tab and the refresh control. How to access, manage, and manually refresh live artifacts in Cowork.

Pairing live artifacts with scheduled tasks. The combination that keeps a live artifact current even before you open it.

The mini project

You'll build a live artifact that pulls from your connectors (calendar, email, and/or notes) and displays a current personal dashboard. The deliverable is a working artifact that shows different content on two different days, proving that the live data connection is functioning.

How to approach this section

Live artifacts are more complex to build than static ones. They require working connectors, careful prompt design, and testing against real data. Don't attempt a complex multi-source dashboard as your first live artifact. Start with one data source, verify the connection works, and add sources once the basic pattern is proven.

The section builds on everything that came before it in Cowork: connectors from Sections 4 and 9, Artifact design from Section 6, and the file access and scheduling patterns from Sections 9 and 10. It's the synthesis of the Act 2 capabilities into their most powerful combined form.

Section 13 — Live Artifacts: Self-Updating Dashboards & Tools

Live Artifacts vs. Static Artifacts

Live Artifacts vs. Static Artifacts

A static artifact is a rendered output that reflects data at the moment Claude generated it. A live artifact is a rendered output that fetches current data when opened. Same visual form, fundamentally different behavior. Choosing the right type for a given use case is the first design decision.

Why this matters

Choosing the wrong type is not a minor inconvenience. A dashboard built as a static artifact requires rebuilding every time the data changes. A report built as a live artifact when the data doesn't change is unnecessarily complex. Understanding the distinction means you make the right architectural choice before investing time in building.

What makes an artifact "live"

A live artifact contains code that, when the artifact is opened or refreshed, makes calls to data sources through Cowork's connector infrastructure. The artifact renders from the results of those calls, not from data baked in at generation time.

Technically: a live artifact's HTML or JavaScript includes calls to `window.cowork.callMcpTool()` or similar interfaces that reach out to connectors (email, calendar, files, external APIs) and return current data. The artifact renders that data each time it runs.

A static artifact contains data embedded in its code at generation time. Opening it shows the same data regardless of when it was last updated.

When to use each type

Use a static artifact when: - The content doesn't change (a decision framework, a checklist template, a one-time analysis) - The output is for a specific moment in time (a report for a specific quarter, a proposal) - You want a permanent record of how things looked at a specific date - The artifact will be shared externally (live artifacts depend on Cowork; external recipients can't run them)

Use a live artifact when: - You'll open this repeatedly and want current data each time - The underlying data changes frequently (daily email, weekly metrics) - The artifact is a personal or team tool, not an external deliverable - Rebuilding the artifact manually to get current data would be frustrating

The trade-off in complexity

Live artifacts are more complex to build, test, and maintain than static ones. They require working connector configurations, handling for connector errors or empty data states, and testing against real data at multiple points in time.

Static artifacts are simpler: Claude generates them once, you review them, you use them.

Don't default to live just because it sounds more impressive. Default to the type that's appropriate for how the artifact will be used.

Practical examples

Artifact type	Static or live?	Reason
Q2 performance report	Static	Specific to Q2; data doesn't change
Daily calendar view	Live	You want today's calendar, not Monday's
Decision framework template	Static	The framework itself doesn't change
Inbox priority tracker	Live	Your inbox changes throughout the day

Artifact type	Static or live?	Reason
Competitive analysis	Static	A moment-in-time document
Weekly team metrics dashboard	Live	Data updates weekly; you want current

Try this in Claude

Look at the Artifacts you produced in Section 6. For each one, ask: would I want to open this again next week and see current data, or is the value in the specific data from when I made it? Any artifact where you'd want current data is a live artifact candidate.

Quick summary

Static artifacts embed data at generation time. Live artifacts fetch current data when opened. Use static for permanent records and external deliverables. Use live for personal and team tools you'll open repeatedly. Live artifacts are more complex to build; choose them when the recurring data freshness justifies the investment.

Section 13 — Live Artifacts: Self-Updating Dashboards & Tools

Connecting an Artifact to MCP Servers and Connectors

Connecting an Artifact to MCP Servers and Connectors

A live artifact's data comes from connectors accessed through Cowork's MCP (Model Context Protocol) infrastructure. Building a live artifact means writing an artifact that makes connector calls at render time. This lesson covers how that connection works in practice.

Why this matters

The mechanism of the connection is what you need to understand to design live artifacts that work reliably. A live artifact that can't reach its data source returns empty or broken content. Understanding the connection model helps you design for reliability.

How the connection works

Live artifacts in Cowork can use a global function `window.cowork.callMcpTool(toolName, args)` to call connector tools. When the artifact runs, this function reaches out to the Cowork environment, which routes the request to the appropriate connector, retrieves the data, and returns it to the artifact.

The artifact then renders that data: formats it, displays it, and presents it to you in whatever visual form the artifact uses.

The practical flow: 1. You open the live artifact 2. The artifact's JavaScript calls `window.cowork.callMcpTool()` with specific tool names and parameters 3. Cowork calls the connector API 4. The connector returns current data 5. The artifact renders the data

Available connector tools

The tool names available depend on which connectors you have configured. Common examples: - Email connectors expose tools for inbox retrieval, thread reading, search - Calendar connectors expose tools for event listing, availability checking - File connectors expose tools for file reading and listing - Project management connectors expose tools for task and project retrieval

When building a live artifact, you need to know which tool names are available from your configured connectors. Claude knows these when it builds the artifact within your Cowork environment.

Design for data failure

Connectors can fail: authentication expires, the service is temporarily unavailable, the query returns no results. Well-designed live artifacts handle these cases gracefully:

- Show a "data unavailable" message rather than a broken interface
- Show the last successful data with a timestamp if real-time data is unavailable
- For empty results, show "nothing to display" rather than blank sections

Ask Claude to include error handling when it builds your live artifact. A live artifact that shows "Calendar connector unavailable, please check your connection" is much more usable than one that shows a blank calendar or an uncaught JavaScript error.

Also available: askClaude

Live artifacts can also call `window.cowork.askClaude(prompt, data)` to run lightweight AI inference on the data they've retrieved. This is useful for: summarizing a set of emails before displaying them, classifying items, generating a brief interpretation of the data.

This adds a second layer to the artifact: data retrieval from connectors, then light AI processing of that data, then display.

Try this in Claude

Ask Claude to build a simple live artifact that uses one connector: "Build a live artifact that shows my next three calendar events, updating each time I open it. Use the calendar connector. If the connector is unavailable, show an error message." Review the code it generates. Look for the `callMcpTool` calls. Verify the error handling is present.

Quick summary

Live artifacts connect to connectors through `window.cowork.callMcpTool()`. When the artifact opens, it calls the connector, gets current data, and renders it. Design for failure: handle connector errors, empty results, and authentication issues gracefully. The `askClaude` function is available for light AI processing of retrieved data before display.

Section 13 — Live Artifacts: Self-Updating Dashboards & Tools

Auto-Refreshing Dashboards, Briefings, and Reports

Auto-Refreshing Dashboards, Briefings, and Reports

Three patterns consistently deliver value as live artifacts: personal dashboards, briefing pages, and recurring reports. Each has a specific design that makes it genuinely useful rather than technically impressive but practically ignored.

Why this matters

The difference between a live artifact you check daily and one you open twice and forget about is almost entirely in the design: whether it answers a question you actually have, displays information at the right level of detail, and loads reliably every time.

Pattern 1: The Personal Dashboard

A personal dashboard answers the question "what do I need to know right now?" It shows current state across the dimensions that matter most to you: inbox priority, calendar for the day, active project status, pending decisions.

Design principles: - One screen, no scrolling needed (or minimal scrolling) - Information organized by urgency, not by source - Action-oriented: each item should suggest what to do with it - Loads fast: three to five connector calls maximum

Good content sources: Email inbox (urgent/flagged items), today's calendar, file system (files modified recently), project management connector.

Sample prompt for building:

> *"Build a live personal dashboard that: (1) shows my top 5 priority emails from today using the email connector, (2) shows today's calendar events, (3) shows any files in my 'Action Required' folder. Use a clean layout with three sections. Handle connector errors gracefully."*

Pattern 2: The Briefing Page

A briefing page answers "what happened since I last checked?" It synthesizes recent activity from multiple sources into a coherent narrative, often with AI summarization.

Design principles: - Time-bounded: since yesterday, since last week, since you last opened it - Synthesized, not just listed: use `askClaude` to produce brief summaries rather than raw connector data - Readable in under three minutes

Good content sources: Email summaries, calendar review, any project or document activity.

What makes it work: The `askClaude` function. Raw email data displayed as a list is not a briefing. Email data processed through a brief summarization prompt becomes one.

Pattern 3: The Recurring Report

A recurring report answers "how are we tracking against what we care about?" It shows metrics, status indicators, and trend data that you check on a regular cadence.

Design principles: - Consistent format so changes are immediately visible - Traffic light indicators (green/yellow/red) for quick scanning - Historical context where it adds value (this week vs. last week)

Good content sources: Project management tools, CRM, file-based metrics you maintain.

What makes it work: The format must stay constant; only the data changes. If the format shifts between visits, you lose the ability to scan quickly for changes.

Choosing between patterns

Ask what question the artifact should answer: - "What do I need right now?" → Dashboard - "What happened?" → Briefing - "How are we doing?" → Report

A single live artifact that tries to answer all three questions is usually too complex and too slow. Build separate artifacts for separate questions.

Try this in Claude

Pick one of the three patterns and build a first version for your work context. Start simpler than you think you need to: one or two data sources rather than five. Verify it loads correctly and displays useful data. Add sources once the basic pattern works.

Quick summary

Three live artifact patterns work consistently well: personal dashboards (current state), briefing pages (recent activity, AI-summarized), and recurring reports (metrics and status). Each answers a different question. Design each for its specific purpose rather than combining all three into one. Start simple and add complexity only after the basic version proves reliable.

Section 13 — Live Artifacts: Self-Updating Dashboards & Tools

The Live Artifacts Tab and the Refresh Control

The Live Artifacts Tab and the Refresh Control

Live artifacts are accessed through the Live Artifacts tab in Cowork. Unlike static artifacts that live in conversation history, live artifacts have a dedicated management surface. The refresh control lets you manually trigger a data reload without waiting for an automatic update.

Why this matters

Knowing where to find your live artifacts and how to manage them is practical housekeeping. The Live Artifacts tab is also where you see which artifacts are current, which have stale data, and which have connection problems.

The Live Artifacts tab

The Live Artifacts tab (in the Cowork sidebar or main navigation, depending on the interface version) shows all your configured live artifacts. Each artifact appears with: - Its name and description - Last refresh time (when data was last fetched) - Status indicator (current, stale, error) - Open and refresh controls

Opening an artifact from this tab loads it with a fresh data fetch. Opening it from a conversation history entry may load a cached version; use the tab for the current view.

The refresh control

Each live artifact has a refresh button. Pressing it triggers a new round of connector calls and re-renders the artifact with the latest data.

When to use manual refresh: - You've just taken an action (replied to an email, added a calendar event) and want to see the updated state - The artifact's automatic refresh interval hasn't triggered yet but you need current data - The artifact shows a stale data warning and you want to update it

The view header in a live artifact also includes a refresh button, so you don't need to return to the tab to refresh.

Managing live artifacts

From the Live Artifacts tab you can: - Rename artifacts (give them more descriptive names than the auto-generated ones) - Delete artifacts you no longer use - View the artifact's underlying code (for debugging) - Duplicate an artifact to create a variant

Build a habit of reviewing your live artifacts list periodically. Artifacts you're not using accumulate like unused bookmarks: they don't cause problems, but they add clutter and make finding the useful ones harder.

Caching behavior

Live artifacts may cache their last data set so they display something immediately while fetching current data in the background. This prevents the "blank screen while loading" experience. The cache shows the last successful fetch; the fresh data replaces it once loaded.

If you see data that looks stale, check the "last refreshed" timestamp before assuming the connector is broken.

Try this in Claude

After building your first live artifact, find it in the Live Artifacts tab. Note the last refresh time. Press the refresh button and observe whether the data updates. If you have a connector that reflects real-time data (email or calendar), make a change (add a calendar event, receive an email) and refresh to verify the artifact reflects the new state.

Quick summary

Live artifacts are managed through the Live Artifacts tab, which shows status, refresh time, and controls. The refresh button triggers a new data fetch when you need current data before automatic refresh. Manage the list actively: delete unused artifacts, name them clearly. Cache behavior means you see data immediately while fresh data loads in the background.

Pairing Live Artifacts with Scheduled Tasks

A live artifact fetches current data when you open it. A scheduled task produces processed output on a timer. Combining them creates something more useful than either alone: a live artifact that's always current, with pre-processed output waiting for you before you even open it.

Why this matters

Live artifacts that require significant data processing take time to load: connector calls, AI summarization, data synthesis. If you open your morning briefing artifact and wait ninety seconds for it to load, the experience degrades. If a scheduled task ran that processing at 6:30am, the artifact loads in seconds because the heavy lifting is already done.

The pairing also handles the gap between "artifact data" and "artifact insight." Raw connector data is retrievable by the artifact. Synthesized insights, structured summaries, and AI-produced interpretations need a processing step. The scheduled task is that processing step.

The architecture

Without scheduled task: Open artifact → Artifact calls connectors → Connectors return raw data → Artifact renders (possibly slow, possibly raw)

With scheduled task: Scheduled task runs at 6:30am → Reads connectors, processes with Claude, writes results to file → You open artifact at 7am → Artifact reads pre-processed file → Artifact renders quickly with clean output

The artifact still connects to live data, but some of that data is the pre-processed file rather than raw connector output. This is hybrid architecture: some data is real-time (calendar, for accurate timing), some data is pre-processed (email synthesis, which is better done once in the background).

When to add a scheduled task to a live artifact

Add a scheduled task when: - The artifact's load time is too slow due to data processing overhead - The artifact uses AI summarization that takes too long to run at open time - The artifact synthesizes data from many sources and the synthesis step is intensive - You want the output to be ready before you open it, not computed when you do

Skip the scheduled task when: - The artifact loads quickly enough already - The data needs to be completely real-time (a task queue, a chat inbox) - The scheduled task cadence doesn't align with your artifact use pattern

Designing the pair

When designing a live artifact with a scheduled task:

- The scheduled task writes its processed output to a specific file in your Cowork outputs folder
- The live artifact reads that file as one of its data sources
- The artifact also reads real-time connectors for data that benefits from freshness (calendar, current file state)
- The artifact renders from the combination

Name the output file clearly: `morning-briefing-latest.md` or `weekly-digest-current.md`. The artifact reads the latest version of that file each time it opens.

Practical example

A morning briefing live artifact is paired with a 6:30am scheduled task:

Scheduled task (6:30am): Reads email since yesterday, summarizes with Claude, writes to `/Cowork Outputs/morning-briefing-latest.md`

Live artifact (opens at 7am): Reads `morning-briefing-latest.md` (the pre-processed email summary) + reads today's calendar in real time + renders both together

Load time: fast. Content: current email summary + today's live calendar.

Try this in Claude

If your live artifact has a slow-loading section (usually AI summarization of email or documents), plan a scheduled task that pre-processes that section and writes the result to a file. The artifact reads the file instead of generating the summary at load time. Measure the load time difference.

Quick summary

Pairing live artifacts with scheduled tasks separates slow preprocessing (done on schedule) from fast rendering (done at open time). The scheduled task runs early, processes data with AI, writes results to a file. The live artifact reads that file plus any real-time connector data needed. The result: fast-loading artifacts with pre-processed insights waiting when you open them.

Section 14 of 14

Section 14: Managed Agents, Capstone & Graduating to Claude Code

Section 14: Managed Agents, Capstone & Graduating to Claude Code

The final section of this course brings together everything you've learned, points to the specialized vertical capabilities that extend what Cowork can do, and gives you a clear picture of what comes next if you want to go further.

Why this section matters

The course has covered the foundations of Claude Chat and the agentic capabilities of Cowork. Section 14 does three things: it introduces Managed Agents and vertical Plugins for specialized professional domains; it frames the capstone as a deliberate system design exercise; and it orients you toward Claude Code and the developer ecosystem for learners who want to go deeper.

What you'll learn

Managed Agents and vertical bundles. Anthropic and third parties provide pre-built agentic systems for specific professional domains: Legal, Financial Services, Small Business, Marketing Operations. These are purpose-built, governed AI agents with the context, tools, and review workflows appropriate for their domain.

When a plugin or managed agent beats a DIY workflow. Not every workflow should be built from scratch. The decision framework for when to install a pre-built solution versus when to build your own.

A short orientation to Claude Code and Channels. For learners who want to go beyond non-coding use into developer-level Claude capabilities. What's available, what it requires, and what it enables.

Designing a complete Claude system. The capstone framework: how to bring privacy, review, and success criteria together into an end-to-end workflow design that's ready to deploy and evaluate.

The capstone mini project

You'll design one end-to-end workflow that uses at least five features across Chat and Cowork. The design includes: the use case, the features it uses, the privacy rules that govern it, the review steps at each stage, the success criteria, and the final deliverable. This is a full system design, not a technical implementation exercise. It demonstrates that you can think clearly about how AI workflows are constructed, governed, and evaluated.

How to approach this section

Section 14 is both conclusion and orientation. The Managed Agents topic expands your awareness of what's already built. The capstone project synthesizes what you've learned. The Claude Code orientation opens a door for learners who want to continue.

Take the capstone seriously. Designing a real system you'd actually deploy is worth more than completing an exercise. If you've been building toward a specific workflow throughout the course, this is where you document and finalize it.

Section 14 — Managed Agents, Capstone & Graduating to Claude Code

Managed Agents and Vertical Bundles

Managed Agents and Vertical Bundles

Not every professional AI workflow needs to be built from scratch. For established professional domains, pre-built Managed Agents and vertical Plugin bundles bring purpose-designed capabilities, appropriate governance, and domain-specific knowledge in a single installation. This lesson covers what they are, what domains they serve, and when to use them instead of building your own.

Why this matters

A legal professional setting up a contract review workflow faces a real design challenge: what constraints are appropriate? What review gates are required? What liability considerations shape how outputs should be described? A Managed Agent for legal work has those decisions already made, by people who have thought carefully about the domain. Building the same workflow from scratch means making those decisions yourself, often without the domain expertise to make them well.

Managed Agents and vertical bundles reduce the design burden for specialists working in governed domains. They're not replacements for professional judgment; they're scaffolding built to

support it.

The domains currently covered

Vertical bundles are available (or in development) for several professional domains:

Legal: Contract analysis, clause extraction, risk identification, regulatory review. Governed with appropriate disclaimers, designed to support professional legal review rather than replace it.

Financial Services: Financial document analysis, regulatory compliance review, report generation. Designed with the data handling and accuracy standards appropriate for regulated financial work.

Small Business: Operations support, customer communication, financial tracking, scheduling. Covers the breadth of tasks a small business owner manages without a specialist team.

Marketing Operations: Campaign planning, content production, performance analysis, audience insights. Integrates with marketing tools and workflows.

Each vertical bundle typically includes: domain-specific Skills, appropriate connectors, reference knowledge for the domain, and governance instructions that shape how Claude approaches work in that professional context.

The governance difference

A well-designed Managed Agent for a professional domain includes guardrails that a DIY workflow might not: - Appropriate disclaimers for outputs that require professional sign-off - Constraints on what the agent will and won't do autonomously - Review requirements built into the workflow, not bolted on after - Domain-appropriate error handling

For regulated professions (law, finance, healthcare), these guardrails aren't optional niceties. They're baseline requirements for responsible use. Using a Managed Agent designed for the domain means inheriting those guardrails rather than having to design them from scratch.

Practical example

A small law firm is considering using Claude for contract review. Option A: build a Cowork workflow from scratch, configure connectors, write Skills, design review gates. Requires the firm's IT or an AI-savvy team member to design the workflow. The quality of the governance depends on how well that person understood the requirements.

Option B: install the Legal Managed Agent. Pre-built for contract review, with appropriate professional disclaimers, review requirements, and constraints. The firm's lawyers can start using it without designing the governance model.

For most firms, Option B is the right starting point. They can customize and extend from there once they understand how the agent behaves.

Try this in Claude

Browse the Plugin marketplace and look at what vertical bundles are available for your professional domain or adjacent ones. For one bundle that interests you, read the description carefully: what does it include, what does it constrain, what permissions does it require? Note whether there are capabilities it explicitly limits that you would need to consider before deploying it in your organization.

Quick summary

Managed Agents and vertical bundles provide pre-built agentic systems for specific professional domains. They include domain-appropriate Skills, connectors, knowledge, and governance. For regulated or specialist professions, they offer appropriate guardrails that DIY workflows require you to design yourself. They're starting points, not final solutions, but for most professionals they're better starting points than building from zero.

Section 14 — Managed Agents, Capstone & Graduating to Claude Code

When to Use a Pre-Built Plugin or Agent vs. Building Your Own

When to Use a Pre-Built Plugin or Agent vs. Building Your Own

Throughout this course, you've built skills for designing Claude workflows. This lesson is about knowing when not to build and when to start with something that already exists. The decision framework is straightforward, but the instinct to build can make it feel more complicated than it is.

Why this matters

Building a Claude workflow from scratch takes time and produces a result that reflects your current understanding of the problem. A pre-built Plugin or Managed Agent reflects the cumulative understanding of its developers, including edge cases you haven't encountered yet. For common professional problems, the pre-built solution is often better than the first version you'd build.

The decision framework

Use a pre-built Plugin or Managed Agent when:

The domain is specialized and governed. Legal, financial, healthcare, compliance. These domains have requirements that go beyond "produce good output." Pre-built agents for these domains typically include appropriate constraints that you'd need to research and design yourself.

The problem is well-defined and common. If thousands of professionals face the same workflow need, someone has probably built a well-tested solution. Contract review, expense report processing, meeting scheduling, customer communication. Standard problems have standard solutions worth evaluating before reinventing.

Speed of deployment matters more than customization. A pre-built solution you can use tomorrow beats a custom solution you can use next month, for most use cases.

Governance and reliability matter more than flexibility. Pre-built solutions from established providers have been tested against a wider range of inputs than your first version will be.

Build your own when:

Your workflow is genuinely unique. If your process doesn't map to any existing solution, you have to build. Industry-specific internal processes, proprietary methodology-based workflows, organization-specific automation patterns.

You need integration with internal systems. Pre-built solutions are built for standard external tools. If your workflow depends on proprietary internal systems with no public API, you'll be building custom connectors regardless.

You need specific control over every decision in the workflow. Some organizations require full visibility into every prompt, every constraint, and every behavior in their AI workflows. Pre-built solutions are opaque to that level. Custom builds are transparent.

The pre-built solution is close but not quite right. Sometimes the right answer is "start with the pre-built solution, customize where needed." That's also valid.

The hybrid approach

For many professionals, the best answer is: start with a pre-built Plugin or Managed Agent, understand how it works, and customize where the defaults don't fit your specific context. This gives you the governance head-start of a pre-built solution with the flexibility to adapt it.

Use the Plugins system to install the bundle, review the Skills and instructions it includes, and modify the ones that don't match your situation. This is faster than building from scratch and more tailored than using the default.

Try this in Claude

Map your three most time-consuming recurring workflows. For each one: is there a Plugin or Managed Agent that addresses it? If yes, evaluate it against the "use pre-built" criteria above. For the one where pre-built seems most appropriate, evaluate whether the governance model is right for your context.

Quick summary

Use pre-built Plugins and Managed Agents for common, well-defined problems, especially in governed domains. Build your own for unique workflows, proprietary system integrations, or situations requiring full transparency. The hybrid approach, starting with pre-built and customizing, is often the fastest path to a well-governed, tailored workflow. The instinct to build everything from scratch is worth questioning when good alternatives exist.

Section 14 — Managed Agents, Capstone & Graduating to Claude Code

A Short Orientation to Claude Code and Channels

A Short Orientation to Claude Code and Channels

This course has focused on non-coding use of Claude: Chat, Cowork, connectors, artifacts, and automation without programming. For learners who want to go further, Claude Code and Claude Channels are the next layer of capability. This lesson is an honest orientation: what they are, what they require, and whether they're the right next step for you.

What Claude Code is

Claude Code is a command-line tool that runs in a terminal. It's designed for software developers who want Claude to work directly with codebases: reading files, writing code, running tests, making changes across multiple files, and reasoning about complex technical problems.

Unlike Chat, which you interact with through a web or desktop interface, Claude Code works in the terminal environment where developers already work. It can read your project files directly, run commands, and make changes in ways that would require manual copying in Chat.

What it requires: You need to be comfortable working in a terminal. You need to have a software project to work with (Claude Code is designed for code, not general document work). You need familiarity with developer concepts like repositories, file structures, and command-line tools.

What it enables: Significantly more powerful code generation, debugging, and refactoring than Chat. Automated implementation of features across multiple files. Code review that considers the full context of a project rather than a single file.

Who it's for: Software developers, technical professionals who write code regularly, and technically-inclined users who want to automate code-adjacent tasks.

What Claude Channels are

Channels are a developer feature for building applications and integrations that use Claude's capabilities programmatically. They're built on the Claude API and are used by developers building products, internal tools, and integrations.

Channels are not a user-facing product; they're a developer building block. If you're considering building a product that uses Claude, or integrating Claude into an internal system, Channels are the appropriate path. If you're a knowledge worker using Claude for your own professional productivity, Channels are probably not your next step.

Should you go further?

If you've completed this course and want more:

You're primarily interested in productivity, not code: The next steps are in Plugins, Managed Agents, and building more sophisticated Cowork workflows. Deeper use of the features in this course, not a different technical track.

You write code regularly and want more from Claude Code: Claude Code's documentation and getting-started guides are the right next resource. The concepts in this course (prompting well, using context effectively, reviewing outputs) transfer directly.

You want to build something with Claude: The Claude API documentation and Channels documentation are the starting point. This requires software development skills.

You're curious about the technical side but not ready to commit: Browse Claude Code's documentation without installing it. Understanding what it can do helps you make an informed decision about whether it's worth learning.

Try this in Claude

Search for "Claude Code getting started" and read the overview documentation. Not to install it, but to understand what it does and who it's for. After reading, you'll have a clearer sense of whether it's the right next step or whether deeper Cowork mastery is more valuable for your situation.

Quick summary

Claude Code is a terminal-based tool for developers working with codebases. Channels are developer building blocks for creating Claude-powered applications. Both require technical background to use effectively. For most non-developer professionals completing this course, the right next step is deeper mastery of the Cowork capabilities covered here, not a move to developer tools. Claude Code is the right next step for learners who write code regularly and want Claude integrated into their development workflow.

Section 14 — Managed Agents, Capstone & Graduating to Claude Code

Designing a Complete Claude System: The Capstone Framework

Designing a Complete Claude System: The Capstone Framework

You've reached the end of the course. This final lesson isn't a summary of what you've learned; it's a framework for putting it to use. A complete Claude system is a workflow that someone could evaluate, audit, and improve. This lesson shows you how to design one.

Why this matters

Understanding Claude's capabilities individually is useful. Combining them into a coherent, governed, evaluable system is what makes Claude transformatively valuable to your work. The capstone exercise is the difference between "I've learned about these features" and "I've designed a system that uses them responsibly and effectively."

The five dimensions of a complete Claude system

A well-designed Claude system addresses five dimensions explicitly:

1. The use case: what problem this solves

Be specific about the problem. "Save time" is not a use case. "Reduce the time from client call to delivered meeting notes from four hours to thirty minutes" is a use case. Specificity makes it possible to know whether the system is working.

The use case should include: the current process, the pain point, the expected improvement, and who benefits.

2. The features: what capabilities are involved

Map the features from this course onto your use case. Which combination of capabilities does this workflow require? Typical systems use five or more features. A complete system might involve: a Project (for context), a connector (for data access), a Skill (for the core task), a Cowork scheduled task (for automation), and an Artifact (for the output).

Be explicit about each feature and why it's in the design.

3. The privacy rules: what data is involved and how it's protected

Every Claude system that handles real data needs explicit privacy rules. These address: - What data the system accesses (be specific: which folders, which connectors) - What data the system produces and where it goes - What data should never be in the system (client PII, regulated data, confidential information) - How long outputs are retained and by whom they can be accessed

Privacy rules aren't a legal exercise; they're a practical decision about acceptable risk. Write them down so they can be audited and updated.

4. The review steps: where human judgment is required

Map the review gates explicitly. For every action the system takes, answer: - Who reviews this before it proceeds? - What does a "pass" look like? - What does a "fail" look like (and what happens when it fails)? - What's the escalation path for edge cases?

For fully automated workflows, the review happens when the output is checked; specify the cadence and who's responsible.

5. The success criteria: how you know it's working

Define what success looks like before you deploy. Success criteria should be: - Measurable: specific metrics, not impressions - Time-bounded: evaluated at a specific interval (weekly, monthly) - Honest about what they're measuring: output volume is not the same as output quality

Example success criteria: - Meeting notes produced within 30 minutes of call end, 90% of the time - No factual errors in client deliverables over a 30-day period - Time saved per week, as measured by before/after comparison

The capstone deliverable

Your capstone is a written system design that addresses all five dimensions. It can be a document, a structured Artifact, a Project instruction set, or any format that makes the design clear and reviewable. Length is less important than specificity: a two-page design with clear answers to all

five dimensions is more valuable than a ten-page document that's vague about review steps or success criteria.

The design process

Work backwards from the use case: 1. Start with the problem: what takes too long, produces inconsistent results, or doesn't happen reliably? 2. Define the success state: what would need to be true for this to be solved? 3. Map the features: which Claude capabilities address each part of the problem? 4. Design the review gates: at each stage where an error would matter, what's the gate? 5. Write the privacy rules: what's the data handling scope? 6. Define the success criteria: how will you know in 30 days whether this is working?

An example system design sketch

Use case: Weekly client status reports currently take 90 minutes per client. They require reading meeting notes from the past week, synthesizing progress against the project plan, and drafting a clear narrative. I have 12 active clients. 90 × 12 = 18 hours per month on this one task.

Features: Project (client context and report template), email connector (read), calendar connector (read), Cowork scheduled task (runs Friday afternoons), file access (meeting notes folders), Artifact (report output format).

Privacy rules: Meeting notes and client project files in authorized Cowork folders. Report drafts go to my review folder before any client-facing use. Client names and project details never enter Claude memory or general Projects.

Review steps: Every draft report reviewed by me before sending. Any report with specific commitments or financial figures gets a second read. Reports go out from my email, not automatically.

Success criteria: Report drafts delivered to review folder by 4pm Friday, every week. Average review time under 15 minutes per report (down from 90). Zero reports sent with factual errors over the first 60 days.

Try this in Claude

Write your capstone system design. Use a real use case from your work. Be specific about all five dimensions. When you've finished, review it against this question: could someone else read this design and understand not just what it does but why it's built this way, and whether it's working? If yes, you have a complete system design.

Quick summary

A complete Claude system addresses five dimensions: the use case (what problem), the features (what capabilities), the privacy rules (what data, how protected), the review steps (where human judgment is required), and the success criteria (how you know it works). The capstone exercise is designing a real system across all five dimensions. Specificity is what makes the design evaluable and improvable. This is the foundation for using Claude not just effectively but responsibly at scale.
